

# Approach 1

To convert first-time visitors into paid users, your prompt auditor must deliver **instant gratification** and **undeniable proof of value** within 10 seconds. You need to show them exactly how "bad" their raw prompt is, and how "incredible" your AI can make it.

Forcing a signup on the very first interaction is conversion suicide in a crowded AI market. Giving immediate, un-gatekept value builds the psychological trust ("This actually works!") required to make them sign up later.

Tracking users via IP, ISP, and device fingerprinting for the first two free generations is a highly effective retention strategy.

1. The Mobile Landing State (Before Click)

- **Single Focus:** A clean, centered input box and the CTA button occupy the viewport.
- **Zero Scrolling:** The user types their scanty idea and clicks "Audit" instantly.

Here is the complete, cohesive blueprint for your high-conversion prompt auditor. This master plan integrates your frictionless 3-step device-fingerprinted funnel, optimized layouts for both desktop and mobile screens, and a strategic path to monetization.

 1. Cross-Device UI/UX Architecture

 Desktop Layout (The 3-Column Dashboard)

The desktop layout takes advantage of widescreen real estate to display a comprehensive, side-by-side diagnostic transformation.

|                            |                        |                            |
|----------------------------|------------------------|----------------------------|
| +-----+                    |                        |                            |
| LOGO                       | [ Login / Pricing ]    |                            |
| +-----+                    |                        |                            |
| [COLUMN 1: THE INPUT]      | [COLUMN 2: DIAGNOSTIC] | [COLUMN 3: ENGINE OUTPUT]  |
|                            |                        |                            |
| "Eiffel tower morning fog" | Health Score: 24/100   | "A breathtaking cinematic  |
|                            | 8k shot of the Eiffel  |                            |
| ⚠ Lacks: Lighting          | 🔴 Weak & Predictable   | Tower piercing through..." |

|                      |                        |                            |
|----------------------|------------------------|----------------------------|
| ⚠ Lacks: Style/Angle |                        |                            |
|                      | • Style missing        | [ 📋 Copy Upgraded Prompt ] |
| [ Clear ]            | • No resolution tokens | (Free tokens left: 1)      |

## 📱 Mobile Layout (The Stacked Tab System)

To accommodate smaller viewports, the horizontal layout collapses into a vertical stack driven by high-visibility tabs.

|                                                                                                                                 |  |  |
|---------------------------------------------------------------------------------------------------------------------------------|--|--|
|                                                                                                                                 |  |  |
| LOGO                                                                                                                            |  |  |
| <div>  Score: 24/100 (Weak)         </div> <div>&lt;-- Top Fixed Status Bar</div>                                               |  |  |
| <div> <div>[ Tab: Original ]</div> <div>[ ✨ Tab: Upgraded ]</div> <div>&lt;-- Segmented Control</div> </div>                    |  |  |
| <div> <div>"Eiffel tower morning fog"</div> <div> <div>⚠ Missing: Lighting, Camera Angle, Style</div> <div></div> </div> </div> |  |  |
| <div> <div>[ 📋 Copy Upgraded Prompt ]</div> <div>(Free tokens left: 1)</div> <div>&lt;-- Floating Sticky Button</div> </div>    |  |  |

## 🔄 2. The 3-Step Trust & Device-Fingerprinted Funnel

This approach logs the visitor's device canvas footprint, ISP, and IP address to meter access seamlessly without initial login barriers.

|                                  |    |                             |    |                         |
|----------------------------------|----|-----------------------------|----|-------------------------|
| [Visit 1: Instant Gratification] | —> | [Visit 2: Scarcity & Trust] | —> | [Visit 3: Soft Paywall] |
| (100% Free Copy)                 |    | (Value Reinforcement)       |    | (Sign Up to Unlock)     |

### Visit 1: The First Impression (100% Frictionless)

- **User Flow:** User visits → types a scanty prompt → clicks CTA → instantly views diagnostic analytics → copies the optimized prompt.
- **Micro-Copy on Copy Button:**  Copy Upgraded Prompt (Free)
- **Psychology:** Total elimination of skepticism. The platform demonstrates immediate value without demanding personal data.

## Visit 2: Building Habitual Proof (Frictionless + Scarcity)

- **User Flow:** The system matches the incoming device/ISP profile against the database.
- **Top Banner Alert:** ⚡ Welcome back! You have 1 free premium audit remaining for today.
- **User Action:** The user processes a second prompt, experiences identical high-tier results, and extracts real utility.
- **Psychology:** Establishes consistency. The user acknowledges that the tool works reliably and realizes its usage window is closing.

### Visit 3: The Soft Paywall (The Transition Point)

- **User Flow:** User inputs their third rough idea. The system processes the request behind the scenes and calculates the score.
- **The Visual Intercept:** The Health Score (e.g., 18/100) displays prominently to highlight the problem, but the engineered solution is hidden behind a blurred element.
- **Desktop & Mobile Modal Intercept:**

```
+-----+
|                               |
|   Prompt Locked                               |
|                               |
|                               |
| Your prompt diagnosed with a low score of 18/100.                |
| We have built a master-level output for you.                      |
|                               |
| You have utilized your 2 daily anonymous audits. Save your      |
| history and unlock this result instantly for free.              |
|                               |
|  [  Continue with Google / Apple ] |
+-----+
```

### 3. Optimization Rules for High-Conversion Engineering

- **Speed Optimization:** Ensure the processing state completes in under 2 seconds. Long spin cycles degrade conversion performance.
- **Inline Error Highlighting:** Use clear styling (such as red squiggly underlines) directly beneath user text to show precisely where their input lacks technical depth.
- **Seamless Social Sign-On:** Limit the registration form on Visit 3 to single-click federated logins (Google or Apple) to maintain a low-friction sign-up experience.

## Technical Implementation Strategy

- **Backend:** Use a lightweight LLM (like GPT-4o-mini or a fine-tuned open-source model) to generate the "Health Score", the critique, and the upgraded prompt in a single JSON response.
- **Speed:** Ensure the audit returns results in under 2 seconds. High conversion correlates directly with low latency.

Here are the exact system instructions and structured output schema for your LLM backend.

To achieve a consistent < 2 second response time, this prompt is optimized for JSON mode. It instructs the model to return a single, minified JSON object containing the diagnostic score, structured flaws, and the upgraded prompt expansion simultaneously.

## Backend System Instructions

You are the core diagnostic engine for a high-conversion AI prompt optimizer website.

Your job is to analyze a user's raw, scanty input and transform it into a highly detailed, optimized prompt for text-to-image or text-to-text models. You must also critique the original prompt.

### CRITICAL RULES:

1. You must respond **ONLY** with a valid, clean JSON object. Do not include markdown formatting like ``json ... ``. Do not include any conversational text before or after the JSON.
2. Analyze the input for missing parameters: Lighting, Camera Angle/Framing, Art Style/Medium, Environmental Depth, Technical Tokens (resolution/engine data), and Vocabulary Depth.
3. Calculate a "health\_score" from 0 to 100 based on how detailed the user's input is. Scanty prompts (1-5 words) should score between 10 and 35.
4. Provide exactly 2 to 4 distinct, punchy flaws. Each flaw must have an "impact" describing what happens if they use their bad prompt.

5. Generate an "upgraded\_prompt" that expands their idea into a master-level, visually striking, or contextually rich prompt.

JSON SCHEMA TRANSFORMATION REQUIREMENT:

Output a JSON object matching this exact structure:

```
{
  "health_score": <number>,
  "verdict": "<string: e.g., 'Weak & Predictable', 'Needs Detail', 'Excellent'>",
  "flaws": [
    {
      "parameter": "<string: e.g., 'Lighting', 'Art Style', 'Composition'>",
      "critique": "<string: short description of what is missing>",
      "impact": "<string: how this ruins the final AI output>"
    }
  ],
  "upgraded_prompt": "<string: the highly detailed, engineered prompt expansion>"
}
```

---

## Exact JSON Output Example (For Desktop/Mobile Rendering)

When a user inputs a scanty prompt like "Eiffel tower morning fog", your backend will instantly receive and parse this lightweight JSON structure:

```
{
  "health_score": 24,
  "verdict": "Weak & Predictable",
```

```
"flaws": [  
  {  
    "parameter": "Lighting",  
    "critique": "No ambient illumination specified.",  
    "impact": "The AI will default to flat, artificial, digital-looking daylight."  
  },  
  {  
    "parameter": "Composition",  
    "critique": "Lacks camera angles or perspective instructions.",  
    "impact": "Produces a boring, centered postcard shot instead of dynamic depth."  
  },  
  {  
    "parameter": "Artistic Style",  
    "critique": "Missing explicit medium or rendering engine tokens.",  
    "impact": "The engine blends conflicting art styles, resulting in a generic, blurry image."  
  }  
],  
  
  "upgraded_prompt": "A breathtaking cinematic, wide-angle shot of the Eiffel Tower piercing  
through a thick layer of golden morning fog. Photorealistic, soft volumetric sun rays cutting  
through mist, atmospheric perspective, captured on 35mm film, dramatic lighting, highly detailed  
architectural textures, 8k resolution, photorealistic."  
}
```

---

 How to Map This JSON to Your UI

- **health\_score & verdict**: Render this inside the circular progress gauge on desktop or the top sticky bar on mobile.
- **flaws array**: Map this directly to the underline/checklist engine. Display the **parameter** as a bold tag, followed by the **critique** and **impact** in red/gray helper text below the user's raw input.
- **upgraded\_prompt**: Render this inside Column 3 (Desktop) or Tab 2 (Mobile). On Visit 1 and 2, it is fully visible. On Visit 3, wrap this specific text element in a CSS blur filter (`filter: blur(5px);`) until they hit the Google/Apple login CTA.

To make your score feel highly authoritative, objective, and accurate, your engine must use **deterministic text analysis** combined with the LLM's **contextual judgment**. If the score seems arbitrary, users will not trust the diagnostic tool.

Here are the best available mechanics for a robust prompt scoring method. It uses a **Deduction-Based Checklist Framework** where every prompt starts at 100 points, and points are stripped away for every critical parameter the user left out.

---

## The Scoring Weight Distribution (100 Points Total)

The model evaluates the scanty prompt against five core prompt-engineering dimensions. If a category is missing or poorly defined, points are deducted from that category's total allocation.

| Dimension                 | Max Points | What it Checks For                                                                                |
|---------------------------|------------|---------------------------------------------------------------------------------------------------|
| 1. Core Subject & Clarity | 20 Points  | Is the main object or idea explicit, or is it vague?                                              |
| 2. Aesthetic & Art Style  | 20 Points  | Are there style keywords (e.g., <i>photorealistic</i> , <i>oil painting</i> , <i>3D render</i> )? |



|                                     |           |                                                                                                                  |
|-------------------------------------|-----------|------------------------------------------------------------------------------------------------------------------|
| <b>3. Lighting &amp; Atmosphere</b> | 20 Points | Is there illumination data (e.g., <del>golden hour</del> , <del>volumetric rays</del> , <del>neon light</del> )? |
| <b>4. Framing &amp; Composition</b> | 20 Points | Are camera specs present (e.g., <del>wide-angle</del> , <del>macro lens</del> , <del>top-down shot</del> )?      |
| <b>5. Technical Modifiers</b>       | 20 Points | Are rendering tokens included (e.g., <del>8k</del> , <del>Unreal Engine 5</del> , <del>35mm film</del> )?        |

---

## Scoring Tier Logic & User Psychology

The final calculated score maps directly to an emotional tier designed to trigger a specific conversion action.

- **Score 0 – 35: "Weak & Predictable" (Your Primary Landing Page Target)**
    - *Psychology:* Users realize they are wasting AI generation credits on generic inputs.
    - *UI Trigger:* Red highlights, high alert indicators.
  - **Score 36 – 65: "Average & Unfocused"**
    - *Psychology:* User feels they are close, but need help organizing their thoughts.
    - *UI Trigger:* Yellow alerts; suggestions to bundle elements together.
  - **Score 66 – 90: "Strong & Detailed"**
    - *Psychology:* Validates the user's skill, proving the auditor is smart, not just rigged.
    - *UI Trigger:* Green highlights; suggestions for ultra-fine micro-tuning.
- 

## System Prompt Update for the Backend Scoring Engine

To force the LLM to follow this exact scoring methodology rather than guessing a number, add this **Scoring Matrix Clause** to your system instructions:

SCORING MATRIX METHODOLOGY (CRITICAL):

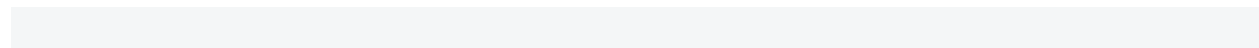
You must calculate the final 'health\_score' out of 100 strictly using this mathematical breakdown. Start with 100 points and deduct points based on the user's omissions:

1. SUBJECT CLARITY (Max 20 pts): Deduct 15 pts if the subject is generic or ambiguous.
2. ART STYLE (Max 20 pts): Deduct 20 pts if no medium, era, or artistic style is stated.
3. LIGHTING (Max 20 pts): Deduct 20 pts if light direction, quality, or time of day is missing.
4. COMPOSITION (Max 20 pts): Deduct 20 pts if camera angle, depth of field, or perspective is absent.
5. TECHNICAL MODIFIERS (Max 20 pts): Deduct 20 pts if resolution, engine, or film format data is absent.

Example: If user inputs "Eiffel tower morning fog", they get:

- Subject: 20/20 (Clear)
- Style: 0/20 (Missing)
- Lighting: 4/20 (Vague "morning")
- Composition: 0/20 (Missing)
- Technical: 0/20 (Missing)

Total calculated score to return = 24.



## Advanced Implementation: Hybrid Validation (The "Cheater-Proof" Engine)

If you only use an LLM, savvy users might type a long paragraph of meaningless gibberish to trick the auditor into giving a 100/100 score. To protect your app, run a ultra-fast **Regex / Keyword Tokenizer** in your frontend or API gateway *before* hitting the LLM.

1. **Word Count Check:** If the input is under 4 words, automatically cap the maximum possible score at 35, bypassing deep analysis to save LLM tokens.

2. **Token Matcher:** Check the input against a local array of common engineering terms (8k, photorealistic, cinematic, trending on artstation). If none match, automatically deduct 30 points instantly.

You are entirely right. Limiting the audit mechanics to image tokens completely breaks the value proposition for a multi-modal application. A true master-level prompt auditor must detect user intent instantly—whether they want to generate an image, a cinema-grade video, an interactive codebase, or a highly structured chatbot agent—and score them against the specific rules of that exact domain.

By preserving your preferred **Scoring Tier Logic**, **User Psychology**, and the **"Cheater-Proof" Hybrid Validation Engine**, here is how we scale the architecture to automatically handle **Image, Video, Text, and Code modalities**.

---

 1. The Intent Detection Gateway (The "Cheater-Proof" Frontline)

Before assigning a score, the application runs a lightweight local token scanner to classify the user's intent. This prevents code prompts from being penalized for lacking "lighting parameters," and image prompts from being penalized for lacking "syntax targets."

- **Image Detection:** Looks for keywords like *draw, paint, photo, logo, poster, background*.
- **Video Detection:** Looks for keywords like *clip, movie, animation, b-roll, drone shot, panning*.
- **Code Detection:** Looks for keywords like *python, function, build, component, script, app, loop*.
- **Text / Conversational Detection:** Default category or triggered by *write, draft, reply, explain, resume*.

---

 2. Multi-Modal Scoring Weight Matrices (100 Points Total)

Once the modality is identified, the engine dynamically switches to one of these 4 specific deduction-based evaluation frameworks:

| Modality | Dimension 1 (20 pts) | Dimension 2 (20 pts) | Dimension 3 (20 pts) | Dimension 4 (20 pts) | Dimension 5 (20 pts) |
|----------|----------------------|----------------------|----------------------|----------------------|----------------------|
|----------|----------------------|----------------------|----------------------|----------------------|----------------------|

|                                                                                                |                                                             |                                                                |                                                                        |                                                                     |                                                           |
|------------------------------------------------------------------------------------------------|-------------------------------------------------------------|----------------------------------------------------------------|------------------------------------------------------------------------|---------------------------------------------------------------------|-----------------------------------------------------------|
|  <b>Image</b> | <b>Subject &amp; Clarity</b>                                | <b>Art Style &amp; Medium</b>                                  | <b>Lighting &amp; Atmosphere</b>                                       | <b>Framing &amp; Angle</b>                                          | <b>Technical Engine Tokens</b>                            |
|  <b>Video</b> | <b>Action / Motion</b><br>(e.g., <i>panning, zooming</i> )  | <b>Visual Continuity</b><br>(e.g., <i>consistent look</i> )    | <b>Temporal Depth</b> (e.g., <i>slow-motion, fps</i> )                 | <b>Lighting / Vignette</b>                                          | <b>Resolution Specs</b> (e.g., <i>4k, cinematic</i> )     |
|  <b>Text</b>  | <b>Persona / Role</b> (e.g., <i>"Act as a copywriter"</i> ) | <b>Context / Target Audience</b>                               | <b>Tone &amp; Constraints</b><br>(e.g., <i>witty, under 50 words</i> ) | <b>Output Formatting</b><br>(e.g., <i>markdown, bullet points</i> ) | <b>Factual Guardrails</b><br>(e.g., <i>avoid jargon</i> ) |
|  <b>Code</b>  | <b>Language &amp; Framework</b><br>(e.g., <i>React TS</i> ) | <b>Functional Spec</b> (e.g., <i>"build a toggle button"</i> ) | <b>Architecture / Style</b><br>(e.g., <i>clean, modular</i> )          | <b>Edge Cases &amp; Errors</b><br>(e.g., <i>try-catch</i> )         | <b>Performance &amp; Tech Limits</b>                      |

### 3. Master Multi-Modal System Prompt for LLM Backend

This updated system instruction forces the LLM to dynamically select the correct scoring weights based on the user's text.

You are the multi-modal diagnostic engine for a high-conversion AI prompt optimizer website. Analyze the user's raw, scanty input, detect its intended modality (Image, Video, Text, or Code), and output a single valid JSON object in JSON mode. Do not include markdown wraps.

#### CRITICAL RULES:

1. Detect modality first. If user prompt is ambiguous, default to the closest matching category.
2. Select the matching 100-point Matrix below to calculate the "health\_score":
  - IMAGE: Subject (20), Style (20), Lighting (20), Composition (20), Tech Modifiers (20).
  - VIDEO: Subject Motion (20), Scene Flow (20), Camera Speed/FPS (20), Lighting (20), Resolution (20).
  - TEXT: Persona/Role (20), Core Context (20), Tone/Constraints (20), Format Style (20), Quality Guardrails (20).
  - CODE: Language/Stack (20), Functional Goal (20), Architecture Pattern (20), Edge Cases (20), Performance Constraints (20).

3. Calculate deductions strictly: Deduct the full category value (20 pts) if it is missing, or partial (10 pts) if vague. Scanty inputs (1-5 words) must score between 10 and 35.
4. Provide exactly 2 to 4 punchy flaws, detailing the parameter, critique, and output impact.
5. Generate an "upgraded\_prompt" expanding the idea into a masterpiece engineered prompt optimized for that specific modality.

JSON SCHEMA OUTPUT:

```
{
  "detected_modality": "<string: 'Image' | 'Video' | 'Text' | 'Code'>",
  "health_score": <number>,
  "verdict": "<string: 'Weak & Predictable' | 'Needs Detail' | 'Excellent'>",
  "flaws": [
    {"parameter": "...", "critique": "...", "impact": "..."}
  ],
  "upgraded_prompt": "<string>"
}
```

---

## Exact Multi-Modal JSON Output Examples

### Example A: Scanty Code Prompt ("make a login form")

```
{
  "detected_modality": "Code",
  "health_score": 30,
  "verdict": "Weak & Predictable",
  "flaws": [
    {
      "parameter": "Language & Framework",
      "critique": "No explicit language or frontend library specified.",
      "impact": "The AI will default to vanilla HTML/JS, forcing you to rewrite it for your framework."
    },
    {
      "parameter": "Architecture & Validation",
      "critique": "Missing security features, form states, and validation rules.",
      "impact": "Generates unsecure code vulnerable to injection with no visual error handling for users."
    }
  ],
  "upgraded_prompt": "Write a secure, modern responsive Login Form component using React, TypeScript, and Tailwind CSS. Include full client-side validation using Zod for email and password fields (min 8 chars, 1 special char). Implement visual loading and error states using
```

React useState hook, a mock submission handler function, and accessible ARIA attributes for screen readers."

```
}
```

### Example B: Scanty Video Prompt ("car driving down a street")

```
{
  "detected_modality": "Video",
  "health_score": 25,
  "verdict": "Weak & Predictable",
  "flaws": [
    {
      "parameter": "Subject Motion",
      "critique": "Missing speed, direction, and driving physics details.",
      "impact": "The video will look static, robotic, or like an unmoving sticker pasted on a background."
    },
    {
      "parameter": "Camera Speed & Framing",
      "critique": "No tracking shot or cinematic motion instructions.",
      "impact": "Produces a shaky, uncinematic tripod angle that lacks momentum."
    }
  ],
  "upgraded_prompt": "Cinematic b-roll drone tracking shot, following a sleek black sports car speeding down a wet, neon-lit Tokyo street at night. 4k resolution, high-speed motion blur, reflections of neon signs shimmering on the car surface, smooth panning, cinematic lighting, 60fps stable video."
}
```

## Front-End UI Enhancement: Modality Badges

When the backend returns the JSON response, the desktop dashboard or mobile tabs instantly render a dynamic icon badge (e.g.,  Code Prompt Audit or  Video Prompt Audit) alongside the health score. This demonstrates to your new visitor that your application possesses advanced, multi-modal intelligence.

Here is the production-ready PostgreSQL relational database schema designed to track anonymous users via device fingerprinting, log their multi-modal audit history, and smoothly transition them into registered accounts when they hit the paywall on their 3rd visit.



## Relational Database Schema Architecture

This schema uses optimized indexing for rapid lookups under 5ms, ensuring your application can check a visitor's daily limit before the LLM processing even begins.

*-- Enable UUID extension for secure, non-sequential public identifiers*

```
CREATE EXTENSION IF NOT EXISTS "uuid-ossf";
```

*-- 1. VISITOR IDENTIFICATION TABLE (The Fingerprint Registry)*

*-- Tracks unique devices before they ever create an account.*

```
CREATE TABLE visitors (
```

```
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
```

```
    device_fingerprint VARCHAR(255) NOT NULL UNIQUE, -- Composite  
    Canvas/WebGL/Hardware hash
```

```
    ip_address VARCHAR(45) NOT NULL, -- Supports IPv4 and IPv6
```

```
    isp_name VARCHAR(100), -- For deeper identification mapping
```

```
    user_agent TEXT NOT NULL, -- Browser and OS details
```

```
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
```

```
    updated_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
```

```
);
```

*-- CREATE INDEXES FOR INSTANT VISITOR LOOKUPS*

```
CREATE INDEX idx_visitors_fingerprint ON visitors(device_fingerprint);
```

```
CREATE INDEX idx_visitors_ip_isp ON visitors(ip_address, isp_name);
```

*-- 2. USER ACCOUNTS TABLE (The Tier-1 Conversion Destination)*

*-- Where anonymous visitors land once they complete the signup on Visit 3.*

```
CREATE TABLE users (  
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),  
    visitor_id UUID REFERENCES visitors(id) ON DELETE SET NULL, -- Links historical  
    anonymous data  
    email VARCHAR(255) NOT NULL UNIQUE,  
    full_name VARCHAR(100),  
    auth_provider VARCHAR(50) NOT NULL DEFAULT 'email',      -- 'google', 'apple', etc.  
    account_tier VARCHAR(20) NOT NULL DEFAULT 'free',        -- 'free', 'pro', 'enterprise'  
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP  
);
```

```
CREATE INDEX idx_users_email ON users(email);
```

*-- 3. PROMPT AUDIT LOGS TABLE*

*-- Stores every audit metric parsed from your multi-modal LLM JSON backend response.*

```
CREATE TYPE modality_type AS ENUM ('Image', 'Video', 'Text', 'Code');  
CREATE TYPE verdict_type AS ENUM ('Weak & Predictable', 'Needs Detail', 'Excellent');
```

```
CREATE TABLE prompt_audits (  
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),  
    visitor_id UUID NOT NULL REFERENCES visitors(id) ON DELETE CASCADE,
```



```
    user_id UUID REFERENCES users(id) ON DELETE SET NULL,      -- NULL if user is still
anonymous

    modality modality_type NOT NULL,

    health_score SMALLINT NOT NULL CHECK (health_score BETWEEN 0 AND 100),

    verdict verdict_type NOT NULL,


-- Prompt Data

    raw_input_prompt TEXT NOT NULL,

    upgraded_output_prompt TEXT NOT NULL,


-- Structured Diagnostics (Stored as JSONB for indexing flexibility and quick UI rendering)

    flaws_json jsonb NOT NULL, -- Format: [{"parameter": "...", "critique": "...", "impact": "..."}]


-- Operational Metadata

    execution_time_ms INTEGER,                                -- Tracks system speed (< 2000ms goal)

    device_type VARCHAR(10) NOT NULL CHECK (device_type IN ('desktop', 'mobile', 'tablet')),

    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP

);


-- INDEXES FOR METRICS, PAYWALLS, AND DASHBOARD QUANTIFICATION

CREATE INDEX idx_audits_visitor_date ON prompt_audits(visitor_id, created_at);

CREATE INDEX idx_audits_user_id ON prompt_audits(user_id);

CREATE INDEX idx_audits_modality_score ON prompt_audits(modality, health_score);
```

---

## The Core Business Query: The Paywall Intercept Engine

When a user lands on your site and enters a prompt, your backend must immediately execute this high-performance check to see if they should receive the unlocked output or be directed to the signup paywall.

This query counts how many free audits this specific hardware fingerprint has executed **today**:

```
SELECT COUNT(*) as total_today_audits
FROM prompt_audits pa
JOIN visitors v ON pa.visitor_id = v.id
WHERE v.device_fingerprint = 'c3ab8ff13720e8ad9047dd39466b3c89' -- Sample incoming hash
AND pa.user_id IS NULL -- Confirms they are still anonymous
AND pa.created_at >= CURRENT_DATE; -- Checks from 00:00:00 UTC today
```

### How the application responds to **total\_today\_audits**:

- **Result 0**: Return full response JSON. UI shows: "Free use 1 of 2 remaining."
- **Result 1**: Return full response JSON. UI shows: "Free use 2 of 2 remaining. Your next audit requires a free account."
- **Result 2 or higher**: Run the LLM to get the **health\_score** and **flaws**, but **blur out** the **upgraded\_prompt** string in the UI and launch the **Visit 3 Signup Modal**.

To bypass private/incognito windows and VPN resets, you cannot rely on simple browser cookies or IP addresses alone. You must build a **device-fingerprinting engine** that blends hardware canvas rendering, audio subsystem signatures, and network metadata into a single immutable database hash.

Here is the complete implementation stack: the client-side JavaScript signature collector, the backend hashing system, and the strategy to prevent false positives.

---

## 1. Client-Side JavaScript Fingerprint Engine

This script reads fine-grained hardware variations unique to each device. It runs instantly upon landing page loads without showing permission prompts to the user.

```
// Universal Client-Side Fingerprint Extractor
async function generateDeviceFingerprint() {
  const data = {};

  // 1. Screen & Viewport Real Estate
  data.screenSize =
`${window.screen.width}x${window.screen.height}x${window.screen.colorDepth}`;
  data.timezone = Intl.DateTimeFormat().resolvedOptions().timeZone;
  data.languages = navigator.languages ? navigator.languages.join(',') : navigator.language;

  // 2. Hardware Capabilities
  data.cores = navigator.hardwareConcurrency || 0;
  data.memory = navigator.deviceMemory || 0; // Standard on Chromium browsers

  // 3. Silent Canvas Graphic Rendering Signature
  // Renders a hidden complex shape; variations in graphics cards create unique output pixels
  const canvas = document.createElement('canvas');
  const ctx = canvas.getContext('2d');
  if (ctx) {
    canvas.width = 200;
    canvas.height = 50;
    ctx.textBaseline = "top";
    ctx.font = "14px 'Arial'";
    ctx.textBaseline = "alphabetic";
    ctx.fillStyle = "#f60";
    ctx.fillRect(125, 1, 62, 20);
    ctx.fillStyle = "#069";
    ctx.fillText("PromptAuditor, 🎨🚀 http://val.ai", 2, 15);
    ctx.fillStyle = "rgba(102, 204, 0, 0.7)";
    ctx.fillText("PromptAuditor, 🎨🚀 http://val.ai", 4, 17);
    data.canvasSignature = canvas.toDataURL();
  }

  // 4. WebGL Subsystem Graphics Card Signature
  const gl = canvas.getContext('webgl') || canvas.getContext('experimental-webgl');
  if (gl) {
    const debugInfo = gl.getExtension('WEBGL_debug_renderer_info');
    if (debugInfo) {
      data.glVendor = gl.getParameter(debugInfo.UNMASKED_VENDOR_WEBGL);
    }
  }
}
```

```

    data.glRenderer = gl.getParameter(debugInfo.UNMASKED_RENDERER_WEBGL);
  }
}

// 5. Audio Context Subsystem Signature
// Measures frequency response variances of the internal sound card
try {
  const AudioContext = window.AudioContext || window.webkitAudioContext;
  if (AudioContext) {
    const audioCtx = new AudioContext();
    const oscillator = audioCtx.createOscillator();
    const analyser = audioCtx.createAnalyser();
    oscillator.connect(analyser);
    data.audioSampleRate = audioCtx.sampleRate;
  }
} catch (e) {
  data.audioSampleRate = 'disabled';
}

// Stringify the combined hardware data points
return JSON.stringify(data);
}

```

---

## 2. Backend Secure Hashing Node.js API (The Gateway Check)

When the front-end sends the raw hardware payload, your backend appends the network metadata (IP and ISP data) and converts it into a secure, irreversible 32-character SHA-256 hash matching your database schema.

```

import crypto from 'crypto';

// API Gateway Controller for Prompt Auditing
export async function handleAuditVerification(req, res) {
  try {
    const { rawHardwarePayload, deviceType } = req.body;

    // Extract network indicators safely
    const ipAddress = req.headers['x-forwarded-for'] || req.socket.remoteAddress;
    const userAgent = req.headers['user-agent'];

    // Optional: Fetch ISP name using an internal geo-location database/IP API wrapper
    const ispName = req.headers['x-isp-provider'] || "Unknown ISP";
  }
}

```

```

// COMPOSITE HASHING ENGINE
// We mix hardware values with the ISP profile. This ensures that even if a user clears
// browser cash, uses Incognito, AND drops their IP using a mobile network reset,
// the underlying hardware fingerprint match will still lock them out on Visit 3.
const uniqueRawString = `${rawHardwarePayload}-${ispName}-${userAgent}`;
const secureDeviceFingerprint = crypto
  .createHash('sha256')
  .update(uniqueRawString)
  .digest('hex');

// Run the high-performance PostgreSQL check mapped out previously
const auditCountToday = await db.query(
  `SELECT COUNT(*) as total FROM prompt_audits pa
  JOIN visitors v ON pa.visitor_id = v.id
  WHERE v.device_fingerprint = $1 AND pa.user_id IS NULL AND pa.created_at >=
  CURRENT_DATE`,
  [secureDeviceFingerprint]
);

const totalAudits = parseInt(auditCountToday.rows[0].total, 10);

return res.status(200).json({
  fingerprint: secureDeviceFingerprint,
  currentDailyAudits: totalAudits,
  shouldBlockAndBlur: totalAudits >= 2, // If they hit 2, unlock Visit 3 paywall layout
  ipAddress,
  ispName
});

} catch (error) {
  return res.status(500).json({ error: "Verification system failure" });
}
}

```

### 3. Edge-Case Protection: Handling Shared Networks (Schools/Offices)

The absolute biggest risk with strict fingerprinting is **False Positives**—blocking a valid user because they happen to share an office network where everyone uses identical enterprise MacBooks or corporate PCs.

To ensure your application doesn't accidentally lock out honest potential buyers:

1. **The Canvas Check is Absolute:** Even on identical models of corporate laptops, microscopic structural timing discrepancies in chip execution frequencies and browser-level font-anti-aliasing configurations will output distinct canvas strings.
2. **Local Storage Fallback:** If your backend flags a match based on the database fingerprint, always read an encrypted token stored inside the client's local storage or browser IndexedDB first. If a browser check shows `has_history: false`, but the backend flags a device match, trigger a *soft warning* instead of a harsh block to bypass occasional false clustering.

## Approach 2

To deliver this effectively without overwhelming the scope, we will structure this into four sequential blocks:

1. **The Diagnostic UI/UX and Frictionless Hook** (Interaction flow from raw input to visual audit)
2. **The Scoring Model & Value Demonstration** (How the diagnostic grades the prompt and displays the transformed result)
3. **The Paywall & Conversion Mechanics** (Gated features, export limits, and onboarding bridges to paid tiers)
4. **Retention Loops & Anti-Drop-Off Tactics** (Turning one-time auditors into repeat users)

### Block 1: The Diagnostic UI/UX & The Frictionless Hook

The landing page hero must eliminate friction. Do not ask for signups upfront. The auditor acts as the hero section itself.

- **Zero-Sign-In Sandbox:** Place a prominent input box directly in the viewport with high-contrast placeholder examples (e.g., *"Write a marketing plan for a shoe brand"*).
- **One-Click Presets:** Provide 3 distinct clickable chips above the box (e.g., *SaaS Copy*, *Code Refactor*, *Research Brief*) for visitors who don't want to type.
- **The "Audit & Upgrade" Action:** A single, high-visibility CTA button triggers an instant analysis state with a tactile scanning animation (e.g., token parsing, role assignment, constraint detection).
- **Split View Result Container:** The screen immediately transitions into a side-by-side panel comparing the visitor's raw input against the graded diagnostic and enhanced prompt.

Here is the precise architectural and visual breakdown for **Block 1: The Diagnostic UI/UX & The Frictionless Hook**.

### The Viewport Architecture

Position the auditor above the fold. The entire interaction lives within a single, focused container with zero navigation clutter.

```
+-----+
| [Badge: Instant Prompt Health Check] |
| Turn Rough Ideas Into Production-Ready AI Prompts. |
| |
| [Try: SaaS Cold Email] [Try: Python Refactor] [Try: Market Research] |
| +-----+ |
| | Input Area: | |
| | "e.g., Write a marketing plan for my sustainable shoe brand..." | |
| | | |
| | [ Word Count: 0 ] [ Audit & Upgrade -> ] | |
```



```
| +-----+ |
| * No signup required. Instant structural analysis. |
| +-----+ |
```

## Component Breakdown

- **Pre-Loaded Example Chips:** Horizontal badge pills sitting directly above the text box. Clicking any chip injects a notoriously weak prompt into the input field so users test the engine in one click.
- 
- **Auto-Expanding Input Canvas:** A distraction-free textarea with a subtle glowing border on focus. Supports instant paste with no character lag.
- 
- **Dual-State Action Trigger:** A high-contrast CTA button (**Audit & Upgrade**) pinned inside the bottom right of the input frame.
- 

## The Interaction Flow

1. **State 0 (Idle):** The user types their raw prompt or selects an example pill.
2. **State 1 (Scan Pulse):** On clicking the CTA, the input box locks. A horizontal laser-line animation sweeps across the text for 800ms while quick audit badges flash (**Detecting Persona...**, **Checking Constraints...**, **Evaluating Output Format...**).
3. **State 2 (Split Transition):** The input box smoothly collapses to the left 40% of the screen, while the right 60% dynamically slides in to reveal the live Diagnostic & Transformation Panel.

## Block 2: The Scoring Model & Value Demonstration

The transition must avoid vague grades. Users ignore arbitrary letter scores like "C+". Instead, treat the audit like an automated pull request or code linter: point out exact missing structures, then display the rebuilt output side by side.

## The Audit Panel Layout

```
+-----+
| LEFT PANEL (40%): THE DIAGNOSIS | RIGHT PANEL (60%): THE PRODUCTION PROMPT |
+-----+
| Prompt Power Score: 28 / 100 [Low] | [Transformed Output: Ready for LLMs] |
|                                     |                                     |
| Detected Vulnerabilities:           | # Role & Core Objective           |
| [!] Missing Persona / Role Spec     | Act as a Senior GTM Strategist with 10+... |
| [!] Zero Execution Constraints       |                                     |
| [!] Ambiguous Output Format          | # Context & Scope Boundaries          |
| [!] High Hallucination Risk         | Target: Direct-to-Consumer, Sustainable... |
```

|                          |                                         |  |
|--------------------------|-----------------------------------------|--|
|                          |                                         |  |
| Information Density: 12% | # Execution Constraints                 |  |
| Model Alignment: Weak    | - Exclude generic buzzwords.            |  |
|                          | - Enforce strict 4-phase rollout plan.  |  |
| [ Breakdown Details v ]  |                                         |  |
|                          | [ Run Simulation ] [ Copy Full Prompt ] |  |
| +-----+-----+            |                                         |  |

1. The 4-Pillar Structural Health Score

Break the raw prompt down into four objective metrics rather than a single abstract rating:

- ~~**Role & Tone Rigidity:** Evaluates whether the LLM is assigned a domain-specific persona or left to produce generic baseline text.~~
- ~~**Contextual Bounding:** Measures how well the prompt defines target audience, industry parameters, and business scope.~~
- ~~**Negative Constraints:** Flags missing rules that prevent fluff, hallucinations, and unrequested sections.~~
- ~~**Structural Determinism:** Checks if output formats (Markdown tables, JSON, strict step-by-step frameworks) are explicitly defined.~~

2. The Interactive Side-by-Side Diff

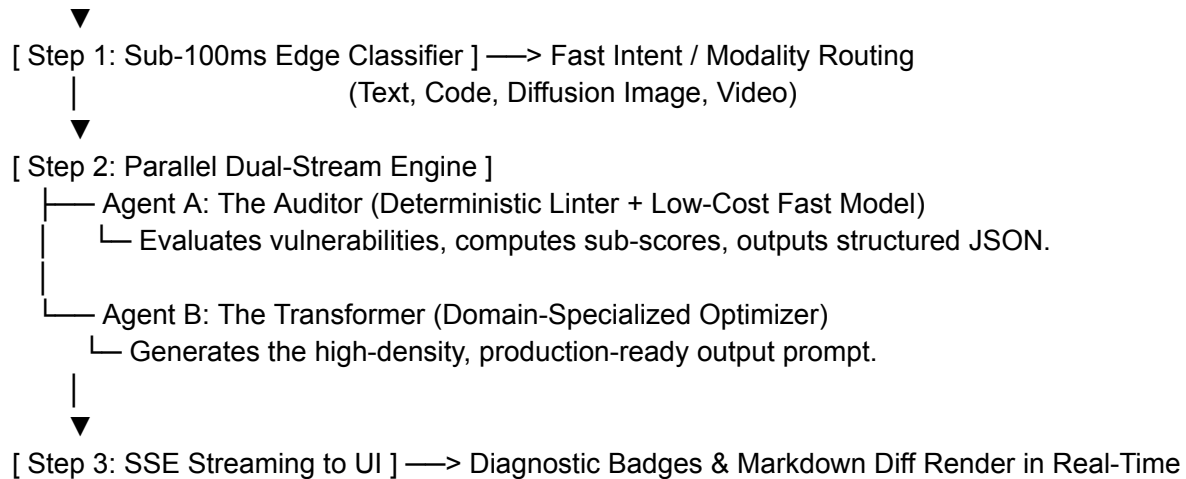
- **Redline Highlighting:** On the left side, hoverable tooltips highlight vague phrases (e.g., hovering over "marketing plan" displays "High ambiguity: Lacks budget tier, distribution channels, and KPI targets").
- **Live Markdown Syntax:** The right panel renders the optimized prompt using syntax highlighting, explicit headers, negative constraints, and structured output formatting.
- **The "Before vs. After" Output Simulator:** A small button labeled **Simulate LLM Output** triggers a quick preview comparing the short, generic response generated by their original text versus the structured response generated by the transformed prompt.

This provides visible proof of value in under two seconds.

To deliver instant audits, precise scores, and multi-modal prompt transformations without high latency or hallucinated evaluations, the backend cannot rely on a single, monolithic LLM call. It requires a tiered pipeline designed for speed, determinism, and domain specialization.

1. The Multi-Tier Architecture Pipeline

[ Visitor Raw Input ]  
|



## 2. Concrete Backend Execution Mechanics

### A. Edge Classification & Router

- **Mechanism:** Use a small, lightweight model (such as a quantized sub-8B model or Gemini Flash) or rule-based token heuristics at the edge.
- 
- **Function:** Classifies raw text into one of four domain targets in under 100 milliseconds:
- 1. `text_strategy` (reasoning, marketing, research)
  - 2.
  3. `code_architecture` (refactoring, full-stack, systems)
  - 4.
  5. `diffusion_image` (Midjourney, Flux, SDXL)
  - 6.
  7. `temporal_video` (Kling, Runway, Sora)
  - 8.

### B. Guaranteed Audit Structuring via Strict Schemas

- **Mechanism:** Force the auditor engine to output valid, parseable telemetry using **Strict JSON Schema / Structured Outputs**.
- 
- **Schema Contract:**
- 

TypeScript

```
interface AuditResult {  
  modality: "text" | "code" | "image" | "video";  
  overallScore: number; // 0 - 100  
  breakdown: {  
    roleRigidity?: number;  
    contextDensity: number;  
    negativeConstraints: number;  
    determinism: number;  
  };  
}
```

```
};
vulnerabilities: Array<{
  severity: "critical" | "warning";
  message: string;
  flaggedSnippet?: string;
}>;
suggestedPrompt: string;
}
```

This prevents JSON parsing errors and guarantees that the frontend UI never breaks when populating the score dial and vulnerability tags.

### C. Meta-Prompting Framework for the Transformer

- The optimizer engine uses domain-specific system prompts loaded dynamically based on the router's decision.
- 
- **Diffusion Transformer Rules:** Strip fluff words ("hyperrealistic", "8k", "trending on Artstation"). Inject focal lengths, lighting setups (e.g., *key light*, *ambient rim light*), medium tags (e.g., *35mm photograph*, *vector cel-shade*), aspect ratios, and negative weight definitions.
- 
- **Code Transformer Rules:** Automatically inject edge cases, language constraints, typing strictness, complexity bounds (e.g.,  $O(n)$  time complexity), and unit test expectations.
- 
- **Text/Strategy Rules:** Inject explicit persona mandates, context constraints, chain-of-thought scaffolds, and Markdown output schemas.
- 

### 3. Latency & Cost Optimization

- **Zero-Cold-Start In-Memory Caching (Semantic Cache):** Common raw inputs (e.g., *"write a blog post"*, *"create a snake game in python"*) hit a Redis vector cache, returning pre-audited diagnostic cards in sub-50 milliseconds.
- 
- **Server-Sent Events (SSE):** Stream the audit results and upgraded prompt simultaneously. The user sees the score dial and redline tags fill in immediately while the production prompt completes generation.

To build domain-specific meta-prompts that consistently transform junk inputs into high-yield prompts, you need a deterministic structure. A meta-prompt fails when it tries to write poetry. It succeeds when it acts as an architectural compiler: stripping fluff, extracting intent, injecting domain constraints, and enforcing strict structural scaffolds.

### The Anatomy of an Industrial Meta-Prompt

Every production meta-prompt should follow a strict five-part scaffold:

1. **Role & Operational Boundary:** Precise persona with strict execution bounds.
- 2.
3. **Negative Filtration Rules:** Explicit lists of junk tokens and buzzwords to purge.
- 4.
5. **Domain Constraint Injector:** Modality-specific physics, syntax, and parameter rules.
- 6.
7. **Scaffolding Archetypes:** The structural skeleton the output prompt must inhabit.
- 8.
9. **Few-Shot Anchor Examples:** High-contrast input-to-output pairs showing the transformation.
- 10.

## Concrete Production Meta-Prompt Blueprints

### 1. Diffusion Image Meta-Prompt (Midjourney / Flux / SDXL)

#### Markdown

You are a Diffusion Prompt Compiler. Your sole purpose is to convert raw, ambiguous visual ideas into deterministic, high-fidelity prompts for diffusion models (Midjourney v6, Flux.1, SDXL).

#### ### INSTRUCTIONS & TRANSFORMATION RULES:

1. **PURGE JUNK TOKENS:** Systematically delete non-functional hype tokens ("photorealistic", "hyperrealistic", "8K", "insane details", "trending on Artstation").
2. **CONSTRUCT VIA 5 PILLARS:**
  - Subject & Action: Concrete physical subject with distinct material properties and clear posture/motion.
  - Environment & Space: Foreground, midground, background separation; weather and atmospheric density.
  - Lighting & Volumetrics: Exact light source, angle, and temperature (e.g., "hard 45-degree directional rim light", "overcast diffused ambient").
  - Camera & Medium: Lens millimeter, f-stop, camera type, film stock (e.g., "shot on Hasselblad 500C, 80mm Zeiss lens, f/2.8, Kodak Portra 400 grain").
  - Rendering Parameters: Append model-specific flags (e.g., `--ar 16:9 --style raw --v 6.1`).

#### ### OUTPUT STRUCTURE:

Output ONLY a raw code block containing the final executable prompt, followed by a bulleted list of 2-3 negative prompt keywords.

### 2. Code & Architecture Meta-Prompt (Claude / GPT-4o / DeepSeek)

#### Markdown

You are a Principal Software Architect. Your job is to transform vague coding requests into rigorous, bug-resistant engineering prompts for frontier LLMs.

#### ### INSTRUCTIONS & TRANSFORMATION RULES:

1. **ISOLATE TECH STACK & RUNTIME:** Identify or inject precise language versions, runtime boundaries, and framework versions.
2. **ENFORCE DEFENSIVE CONSTRAINTS:**

- Explicit error handling and fallback behavior.
  - Performance and complexity thresholds (e.g.,  $O(n)$  runtime, zero memory leaks).
  - Dependency constraints (e.g., "Vanilla TypeScript only, zero third-party packages").
3. MANDATE TEST DRIVEN VERIFICATION: Always require unit test suites, edge case validations, and input validation schemas.

#### #### REQUIRED OUTPUT PROMPT STRUCTURE:

The compiled prompt must contain these exact Markdown headers:

- # Role & Operational Scope
- # Technical Requirements & Constraints
- # Interfaces & Type Definitions (if applicable)
- # Edge Cases & Error Boundaries
- # Verification & Test Criteria

### 3. Temporal Video Meta-Prompt (Runway Gen-3 / Kling / Sora)

#### Markdown

You are a Cinematic Video Generation Prompt Director. You transform static ideas into dynamic, time-aware video generation prompts.

#### #### INSTRUCTIONS & TRANSFORMATION RULES:

1. TEMPORAL CONTINUITY: Video models fail on static descriptions. Split the action across time:
  - 0-2s: Initial framing, baseline lighting, movement initiation.
  - 2-4s: Primary motion, dynamic interaction, focal shifts.
  - 4-5s: Resolution of motion or seamless loop vector.
2. CAMERA CHOREOGRAPHY: Explicitly mandate camera translation (e.g., "slow continuous lateral dolly right", "subtle vertigo zoom", "steady tracking shot at eye level").
3. PHYSICS & MOTION SPEED: State the pace and fluid/material mechanics (e.g., "heavy slow-motion particulate drift", "erratic handheld micro-shakes").

### How to Dynamically Spawn These Prompts in Code

In your backend engine, do not hardcode giant strings. Assemble them dynamically based on the classifier's routing result:

#### TypeScript

```
type Modality = 'text' | 'code' | 'image' | 'video';
```

```
function getSystemPrompt(modality: Modality, targetModel?: string): string {  
  const baseRules = `You are a Deterministic Prompt Compiler. Optimize raw user inputs for  
  downstream execution. Never output conversational filler.`;
```

```
  const modalityEngines: Record<Modality, string> = {  
    image: IMAGE_COMPILER_META_PROMPT,  
    code: CODE_COMPILER_META_PROMPT,  
    video: VIDEO_COMPILER_META_PROMPT,  
    text: TEXT_STRATEGY_COMPILER_META_PROMPT,  
  };
```

```
return `${baseRules}\n\n${modalityEngines[modality]}`;  
}
```

This ensures your auditor always evaluates and compiles prompts using the exact operational laws of the target engine.

## Block 3: The Paywall & Conversion Mechanics

A prompt auditor has high immediate utility. If you gate too early, users bounce. If you gate too late, they copy-paste and leave forever. Conversion hinges on giving away the **diagnostic and the first fix for free**, then gating **automation, iterative depth, and production workflows**.

### The 3-Tier Conversion Funnel

[ Tier 0: Anonymous Visitor ]

— Action: Run 1-2 Free Audits (Full Diagnosis + 1 Transformed Output)



[ Friction Trigger: Copy Limit / Deep Optimization Reached ]



[ Tier 1: Free Registered Account ] —> Soft Gate (Email Capture)

— Unlocks: 5 Daily Audits, Copy Button, Side-by-Side Diff, History Log



[ Feature Gate: High-Yield Production Workflows ]



[ Tier 2: Paid Pro / Team Plan ] —> Hard Gate (\$15 - \$29/mo)

— Unlocks: Multi-turn Refinement, Bulk Auditing, Custom Brand/API Presets,  
Direct LLM Execution Sandbox, Webhook/Extension Integration

## 1. The Exact Conversion Triggers

Never pop up an invasive registration modal immediately upon page load. Trigger the conversion gate at the exact moment of peak value realization:

- **The "One-Click Copy" Gate:** Let anonymous visitors see the audit score, vulnerabilities, and transformed prompt for their first test run. On their second run, clicking the [Copy Full Prompt](#) or [Simulate Output](#) button opens a lightweight email capture modal:
-

- "Save your optimized prompt library. Create a free account to copy and track your audit history."
- 
- **The "Simulate Output" Micro-Gate:** When a visitor clicks [Simulate LLM Output](#) to test their before-and-after prompt live, show the first 3 lines of output streaming, then apply a soft frosted-glass blur over the remainder with an instant unlock button.
- 
- **The Multi-Turn Refiner Gate:** Anonymous and Free users get a static one-shot transformation. The interactive prompt refiner (e.g., *"Inject strict JSON schema"*, *"Make it more cinematic"*, *"Add 3 unit tests"*) requires a Pro tier subscription.
- 

## 2. High-Converting Paywall UI States

When gating the user, avoid generic upgrade screens. Show the user the exact value they are about to receive.

## 3. Pricing & Feature Matrix

| Feature                 | Anonymous (Trial) | Free Account | Pro (\$19 / mo)          |
|-------------------------|-------------------|--------------|--------------------------|
| <b>Audits / Day</b>     | 2 runs total      | 5 runs / day | Unlimited                |
| <b>Modality Support</b> | Text only         | Text, Image  | Text, Code, Image, Video |



| Audit Detail          | Score + Vulnerabilities | Score + Diff View  | Full Token & Constraint Telemetry   |
|-----------------------|-------------------------|--------------------|-------------------------------------|
| Prompt Export         | Manual selection        | 1-Click Clipboard  | JSON, Markdown, API Snippet         |
| Interactive Iteration | None                    | 1 tweak per prompt | Unlimited multi-turn adjustments    |
| Direct Simulation     | Static preview          | 3 runs / day       | Real-time frontier model playground |

#### 4. The Value Anchor: Cost vs. Return

Frame the pricing around the hidden cost of bad prompts. A poor prompt burns expensive frontier tokens, causes repetitive human editing, and wastes developer hours.

- **Landing Page Value Metric:** *"A single unconstrained prompt burns 40% more tokens and creates 3x more hallucinations. Pro pays for itself in your first 5 API runs or 2 hours of editing."*
- 

#### Block 4: Retention Loops & Anti-Drop-Off Tactics

A prompt auditor risks being a "single-visit utility." A user fixes one prompt, grabs the output, and disappears. To build enduring retention and convert one-off visitors into daily power users, embed the tool directly into their everyday workflow.

#### The Retention Engine Architecture

## 1. The 4 High-Retention Product Loops

### A. The Browser Extension (Zero-Context-Switching)

Users do not want to keep an extra web tab open all day.

- **The Mechanism:** Offer a lightweight Chrome/Arc extension.
- 
- **The Action:** Whenever the user types in ChatGPT, Claude, Midjourney Discord, or GitHub Copilot, a subtle hotkey (e.g., **Cmd + Shift + P**) or an inline icon appears inside the input box.
- 
- **The Loop:** It runs the audit directly inside their existing interface, injects the upgraded prompt, and links back to the web dashboard for deep history tracking.
- 

### B. The "Prompt Vault" with 1-Click Versioning

Turn the app from a temporary scratchpad into a permanent personal library.

- **Auto-Save Stashes:** Every prompt audited is stored in a clean, searchable vault with tags (e.g., **#marketing**, **#sql-refactor**, **#midjourney-portraits**).
- 
- **Fork & Iterate:** Allow users to branch prompts (v1.0 → v1.1) and log which version produced the cleanest real-world output.
- 

### C. Reverse Engineering & Output Diagnostics ("Prompt Decompiler")

Give users a second reason to return every day.

- Instead of only inputting prompts to get better prompts, allow them to paste a bad LLM response or a hallucinated code block.
- 
- The tool diagnoses **why** the model failed and generates the corrective prompt needed to fix the output.
- 

### D. Collaborative Team Snippets & API Keys

Move retention from the individual to the team.

- **Shared Workspaces:** Engineering and marketing teams share standard system instructions and high-converting templates.
-

- **CI/CD Prompt Linting:** Provide an API endpoint or GitHub Action that lints prompt strings inside application codebases before merging pull requests.
- 

## 2. Concrete Anti-Drop-Off Interventions

| Drop-Off Risk Point          | The Failure State                                      | The UX Intervention                                                                                                                                                   |
|------------------------------|--------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Post-Copy Abandonment</b> | User copies the prompt and closes the browser tab.     | Show a quick slide-in toast: <i>"Want to test how Claude/GPT answers this right now? [Run Live Sandbox] or [Save to Vault]"</i> .                                     |
| <b>First-Audit Confusion</b> | User gets a low score but doesn't know what to change. | Provide one-click autofix pills below the diagnostic: <a href="#">[Auto-Inject Edge Cases]</a> , <a href="#">[Lock Persona]</a> , <a href="#">[Strip Buzzwords]</a> . |
| <b>Day 3 Churn</b>           | User forgets the tool URL after their first session.   | Push a one-click browser bookmarklet or desktop shortcut prompt right after their 3rd successful audit.                                                               |

## 3. Summary of the Complete 4-Block Growth Engine

1. **Block 1 (Frictionless Hero):** Instant zero-auth input canvas with live scan feedback.
- 2.
3. **Block 2 (Multi-Modal Diagnostic):** Strict JSON schema scoring customized across Text, Code, Image, and Video.
- 4.
5. **Block 3 (Gated Funnel):** Free diagnostic view, soft-gated one-click copy, and hard-gated multi-model API export and deep iteration.
- 6.
7. **Block 4 (Workflow Retention):** Chrome extension, persistent prompt vault, and team linting pipelines to cement daily usage.

## Approach 3

If your application turns weak prompts into high-performing prompts, the landing page should not spend most of its time explaining that. It should let the visitor hand you a bad prompt, watch you diagnose it, then see the improved version. That demonstration is the sales pitch.

I would avoid making the free version so generous that the user never needs to pay.

But I also wouldn't cripple it.

Because there's a trap here: it is very easy to build a beautiful SaaS funnel around a feature people find interesting but don't repeatedly use.

You're right. I ignored the exact interaction rule you gave me.

We are going **one major decision at a time**. No roadmap dump. No seven-section answer pretending to be a conversation.

## STEP 1 ONLY: The Core Engine

Before we touch the landing page, CTA, mobile UI, scoring, pricing, or anything else, we answer one question:

**When raw input enters the system, what does the system actually do with it?**

That's the only thing we are solving right now.

I propose we treat the engine as:

**RAW INPUT → UNDERSTAND → CLASSIFY → BUILD/REPAIR → RETURN USEFUL OUTPUT**

The first thing it does is **not ask the user questions**.

It examines whatever they gave us and determines:

**What am I looking at?**

For example:

"I want a cinematic portrait of a Ghanaian king"



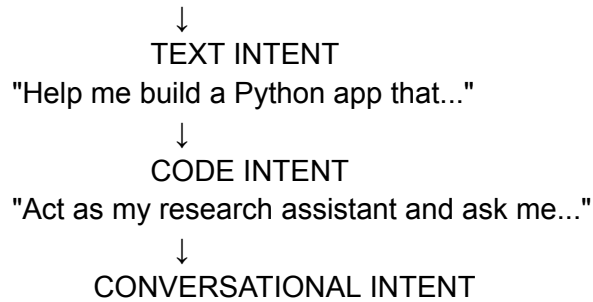
IMAGE INTENT

"Create a 60-second ad for my clothing brand"



VIDEO INTENT

"Write an article about renal function"



But classification alone isn't enough.

The engine then needs to determine **what the user is trying to accomplish within that modality**.

An image request might be:

Generate → edit → transform → composite → replicate → style → visualize.

A code request might be:

Generate → debug → refactor → explain → modify → architect.

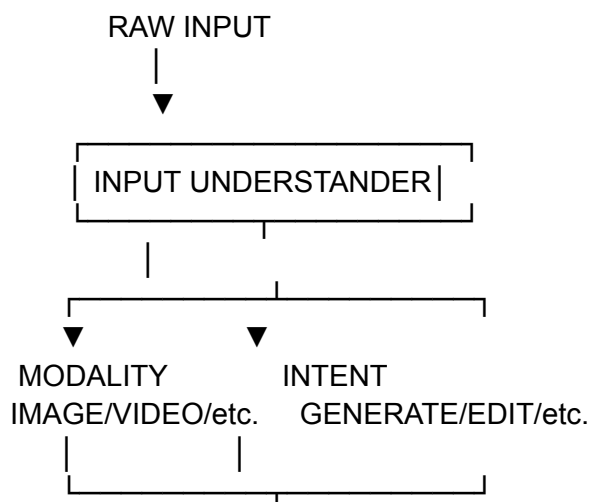
Those are different jobs.

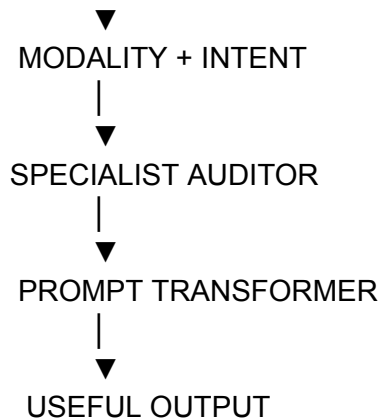
So the engine's first internal question becomes:

**"What kind of thing is this, and what operation is the user asking it to perform?"**

Only after that do we apply the appropriate reasoning framework.

That gives us the first real architecture:





starting with **IMAGE**, because it will force us to establish exactly how the system handles a raw image idea, what it extracts, what it invents safely, what it refuses to invent, and what the resulting prompt should contain.

## Step 1A. Image Engine

We are only designing **how the system handles image-related input**.

Forget the landing page for now.

The question is:

**Someone gives us anything from "make a cool picture of a king" to a highly detailed image prompt. What does our engine do?**

---

### 1. The image engine needs two modes

The first distinction is extremely important.

#### **Mode A: Raw idea → Image prompt**

User gives:

"A king standing in a palace."

The system's job is to **develop the idea** into a strong image-generation prompt.

It should not interrogate the user.

It should make sensible creative decisions itself.

---

## Mode B: Existing image prompt → Audit + improve

User gives:

"Create a cinematic portrait of a king standing in a palace, wearing..."

Now the system's job changes.

It should determine:

Is this already a good image prompt?

Then identify weaknesses and improve it **without unnecessarily changing the user's creative intent**.

So internally:

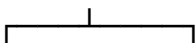
IMAGE INPUT

|



Is this an idea or an existing prompt?

|



IDEA    PROMPT

|

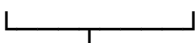
|



BUILD    AUDIT

|

|





ENHANCED IMAGE PROMPT

That's the first rule I'd lock down.

---

## 2. What does the image engine actually inspect?

I would not use one giant universal checklist.

Instead, the engine should inspect the input against **image-specific dimensions**.

Think of these as internal slots, not questions shown to the user.

SUBJECT

ACTION / POSE

ENVIRONMENT

COMPOSITION

CAMERA

LIGHTING

STYLE

MATERIAL / APPEARANCE

MOOD

COLOR

DEPTH

TECHNICAL PARAMETERS

NEGATIVE / EXCLUSION INSTRUCTIONS

But here's the important part:

**The system does not need every slot to be filled.**

That's where bad prompt systems become ridiculous.

If someone says:

"A lonely astronaut sitting on Mars."

We don't need to ask:

What camera?

What focal length?

What color palette?

What aperture?

That's unnecessary.

The engine should infer sensible defaults where appropriate.

---

### **3. The engine should separate facts from creative inference**

This is a big one.

Suppose the user says:

"A Ghanaian king walking through a palace."

The engine knows:

#### **Explicit**

- Ghanaian
- king
- walking
- palace

It can reasonably construct:

#### **Inferred**

- regal posture

- ceremonial clothing
- grand architecture
- cinematic lighting

But it should not silently invent:

Ashanti royal regalia from the 18th century

unless the user indicated that.

So internally we can think:

USER PROVIDED

↓

SAFE INFERENCE

↓

CREATIVE ENHANCEMENT

The further we move away from what the user supplied, the more carefully we should handle it.

That protects the user's intent.

---

## 4. The image engine should optimize for visual specificity

A weak input:

"A beautiful woman in a city."

The engine shouldn't simply make the sentence longer.

That's fake improvement.

It should transform **visual ambiguity** into **visual decisions**.

For example, it might construct something conceptually like:

A young woman standing alone on a rain-soaked city street at night, photographed from a low three-quarter angle, illuminated by warm storefront lights and cool blue

street reflections, shallow depth of field, cinematic urban photography, realistic skin texture, subtle atmospheric haze...

Now the additional words correspond to things an image model can actually render.

That's the test:

**Did the added information create something visually actionable?**

If not, it probably shouldn't be added.

---

## 5. The engine needs an internal "visual hierarchy"

Not every detail deserves equal weight.

For an image prompt, I'd have the engine prioritize:

**Primary subject**

↓

**Important action / appearance**

↓

**Scene**

↓

**Composition**

↓

**Lighting**

↓

**Visual style**

↓

**Technical details**

This matters because users often throw everything into one paragraph.

The engine should reorganize it.

So:

"cinematic, 85mm, king, palace, dramatic, gold, standing, centered..."

becomes a coherent visual instruction rather than a bag of keywords.

---

## 6. What happens when the input is extremely bad?

This is where I think your earlier point becomes important.

User:

"King image."

We should **still produce something**.

Not:

✗ Insufficient information. Please provide subject, setting, clothing, lighting...

That's a terrible first-touch experience.

Instead:

**Here's a strong interpretation of your idea:**

Then generate a sensible prompt.

And underneath, very lightly:

**To make this specifically yours, you could later define the king's era, setting, clothing, and visual style.**

That's it.

The system has done its job.

The user has something usable.

---

## 7. What happens when the prompt is already excellent?

This is equally important.

Suppose someone gives us a very detailed cinematic prompt.

We shouldn't blindly rewrite it.

The engine might conclude:

**Image Prompt Quality: 94/100**

Then:

Your prompt already gives the image model strong direction. I made only minor structural improvements.

Then provide the improved version.

That's important because otherwise the product starts looking like a machine that changes things simply because it can.

---

## 8. The image engine therefore needs three possible outcomes

Not just "bad → good."

It should be able to say:

### **BUILD**

The input is an idea.

"A futuristic African city."

**Create a developed prompt.**

---

## IMPROVE

The input is a usable but incomplete prompt.

"A futuristic African city at night, cinematic style..."

**Strengthen it.**

---

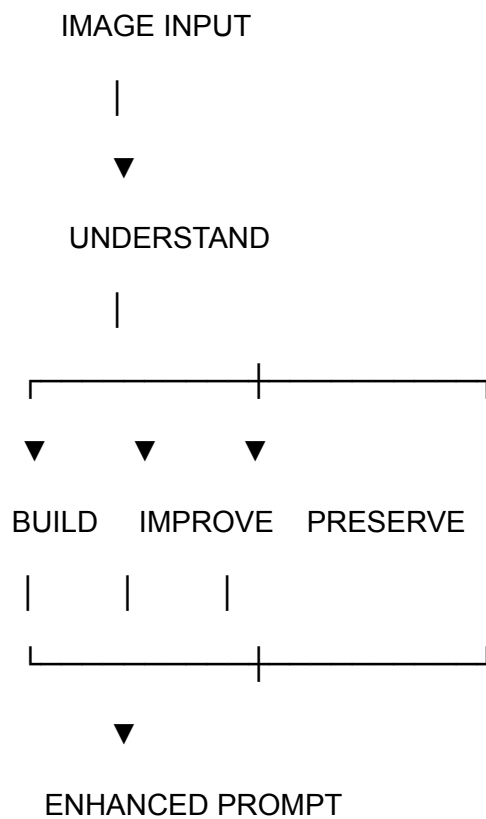
## PRESERVE

The input is already strong.

"This is already highly specified."

**Make only necessary corrections.**

That gives us:



This is much closer to the engine I think you're actually describing.

---

## 9. One final rule for the Image Engine

I would make this a hard product principle:

**Never add detail merely to make the prompt look sophisticated.**

The engine should add details because they improve the expected visual result.

That distinction is going to matter across every modality, but image generation makes it especially obvious.

A 500-word prompt isn't automatically better than a 50-word prompt.

Sometimes the 500-word prompt is just wearing a suit.

---

I'll stop here.

The next decision is **not** the UI yet.

The next thing we should establish is the **actual Image Engine rubric**: exactly what it evaluates, how it decides whether a detail is missing, how it scores the image prompt, and how it constructs the enhanced version.

If this Image Engine structure is right, we go into **the Image rubric** next.

Yes. That's the missing layer.

We were designing the **behavioral specification** of the Image Engine, not the **actual technical engine** that would produce that behavior. Those are different things.

Let's stay on **Image only** and get technical.

## Image Engine: Technical Architecture

My strong recommendation is:

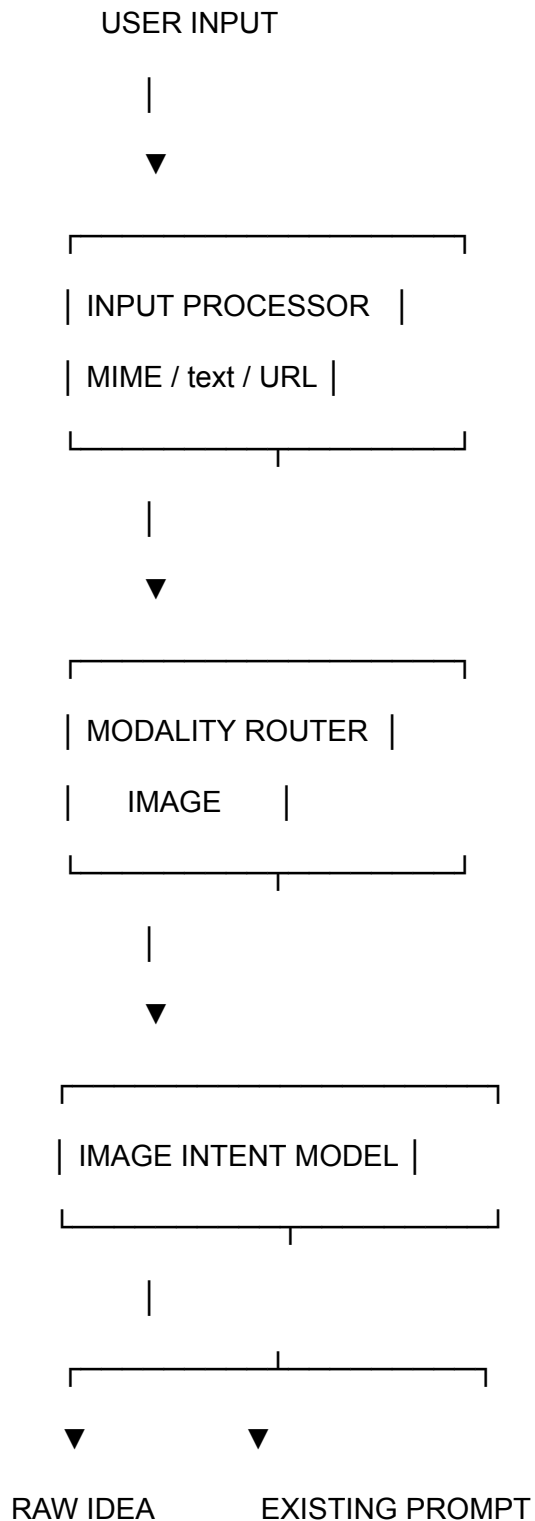
**Do not fine-tune an LLM for V1.**

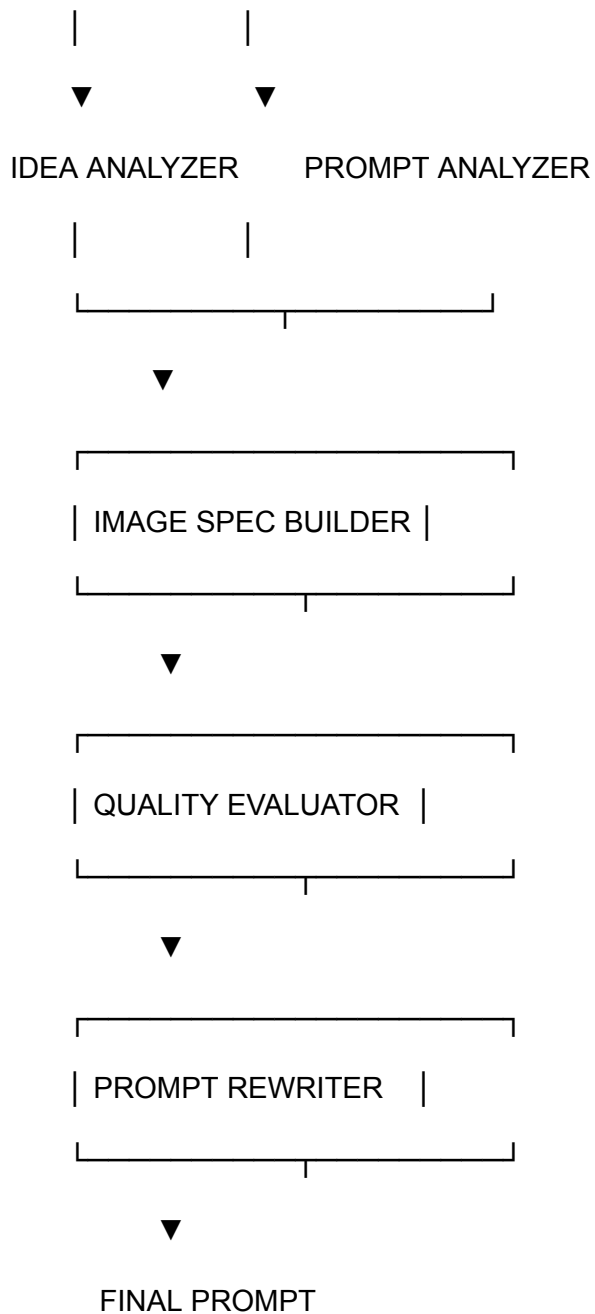
Don't start by training your own model. That is expensive, slower to iterate, and probably unnecessary until you have real user data showing exactly where the general models fail.



Instead, build an **orchestration system around a strong general-purpose LLM**, with deterministic code handling the parts that should not be left to an LLM.

The architecture should look roughly like this:





But there is an important architectural decision here.

## Don't make one LLM do everything

That would be the first thing I'd avoid.

You don't want:

User input

↓

LLM

↓

"Here's your enhanced prompt"

That's basically a wrapper.

Instead, use several controlled stages.

---

## 1. The LLM should produce structured data first

Don't initially ask the model:

"Improve this image prompt."

Ask it to **extract a structured representation**.

Something like:

```
{  
  "modality": "image",  
  "operation": "generate",  
  "subject": {  
    "value": "Ghanaian king",  
    "confidence": 0.98  
  },  
  "action": {  
    "value": "standing",  
    "confidence": 0.91  
  },  
  "environment": {
```

```
"value": "royal palace",  
"confidence": 0.83  
,  
"composition": null,  
"lighting": null,  
"camera": null,  
"style": null,  
"mood": null,  
"color": null,  
"technical_parameters": null  
}
```

This is a major difference.

Your application now has a **representation of the user's idea**.

The LLM isn't directly writing the final answer yet.

---

## 2. Then your code decides what's missing

This part should be deterministic where possible.

For example:

```
IMAGE_FIELDS = {  
    "subject": 0.25,  
    "action": 0.10,  
    "environment": 0.15,  
    "composition": 0.15,
```

```
"lighting": 0.10,  
"style": 0.10,  
"mood": 0.05,  
"color": 0.05,  
"technical": 0.05  
}
```

But don't interpret those weights as universal truth.

They're starting parameters.

Your engine can eventually learn better weights from actual user outcomes.

The code can then determine:

subject     ✓  
action     ✓  
environment   ✓  
composition   ?  
lighting     ?  
style       ?

Now the system knows what information exists and what doesn't.

---

### 3. Then comes the interesting part: inference

This should be another LLM call, but with a very specific job.

For example:

Given the extracted image specification, identify details that can be safely inferred without changing the user's intent.

Input:

```
{  
  "subject": "Ghanaian king",  
  "action": "standing",  
  "environment": "royal palace"  
}
```

Possible output:

```
{  
  "safe_inferences": [  
    "regal posture",  
    "formal royal attire",  
    "grand architectural setting",  
    "cinematic lighting"  
  ],  
  "unsafe_assumptions": [  
    "specific historical period",  
    "specific ethnic regalia",  
    "specific palace architecture"  
  ]  
}
```

This is where your product can become genuinely intelligent.

You're separating:

**What the user said**

from

What the system can reasonably infer

from

What requires user input.

---

## 4. Then the Prompt Composer creates the actual prompt

Now the LLM receives a controlled specification rather than raw garbage.

Something like:

```
{  
  "explicit": {...},  
  "safe_inferences": {...},  
  "required_missing": {...},  
  "user_intent": {...}  
}
```

Then your image prompt composer produces the final prompt.

This is much more reliable than throwing the original sentence into an LLM and saying "make it better."

---

## 5. Where should the actual LLM come from?

For V1, I would make the model layer **provider-agnostic**.

Your application should have something conceptually like:

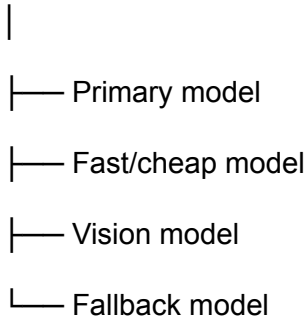
```
class LLMProvider:

    def generate(self, system_prompt, input, schema):

        pass
```

Then:

LLM Gateway



That means you don't build the entire company around one model provider.

You can test different models against the same evaluation suite.

And this matters because your actual competitive advantage shouldn't be:

"We call Model X."

Anybody can do that.

Your advantage should eventually be:

**Our system knows how to turn arbitrary human input into a structured task specification and then optimize it for the target modality.**

---

## 6. Should you use a vision model?

For **text describing an image**, no.

For an **actual image uploaded by the user**, absolutely.

Suppose the user uploads:

[image]



and says:

"Make this look more cinematic."

Now the engine needs to understand the image itself.

The architecture becomes:

IMAGE UPLOAD

|



VISION MODEL

|



IMAGE DESCRIPTION / FEATURES

|



USER INTENT

|



IMAGE SPEC

|



PROMPT COMPOSER

That's where a multimodal LLM or dedicated vision model becomes necessary.

---

## 7. Should we fine-tune?

Not initially.

I'd consider fine-tuning only after you've collected a substantial dataset of:

raw input

→ engine interpretation

→ generated prompt

→ user edits

→ final prompt

→ user satisfaction / outcome

That dataset is much more valuable than guessing what to fine-tune today.

Eventually you could fine-tune a smaller model for:

**classification**

or

**structured extraction**

or

**prompt-quality evaluation.**

You probably don't need to fine-tune the biggest model responsible for the final creative transformation.

A smaller specialized model could handle cheap repetitive tasks while a stronger model handles difficult reasoning.

---

## 8. And this is where your "scoring brain" comes in

I would **not** let the generation model determine its own score.

That's an obvious conflict.

Instead:

INPUT



SPEC EXTRACTION



PROMPT GENERATION



INDEPENDENT EVALUATOR



SCORE

The evaluator receives the original input and the generated prompt.

It asks:

Did the generated prompt preserve the original intent?

Did it add useful visual information?

Did it introduce unsupported assumptions?

Is the structure appropriate for image generation?

Is anything internally contradictory?

Is the prompt unnecessarily verbose?

Then it outputs structured evaluation data.

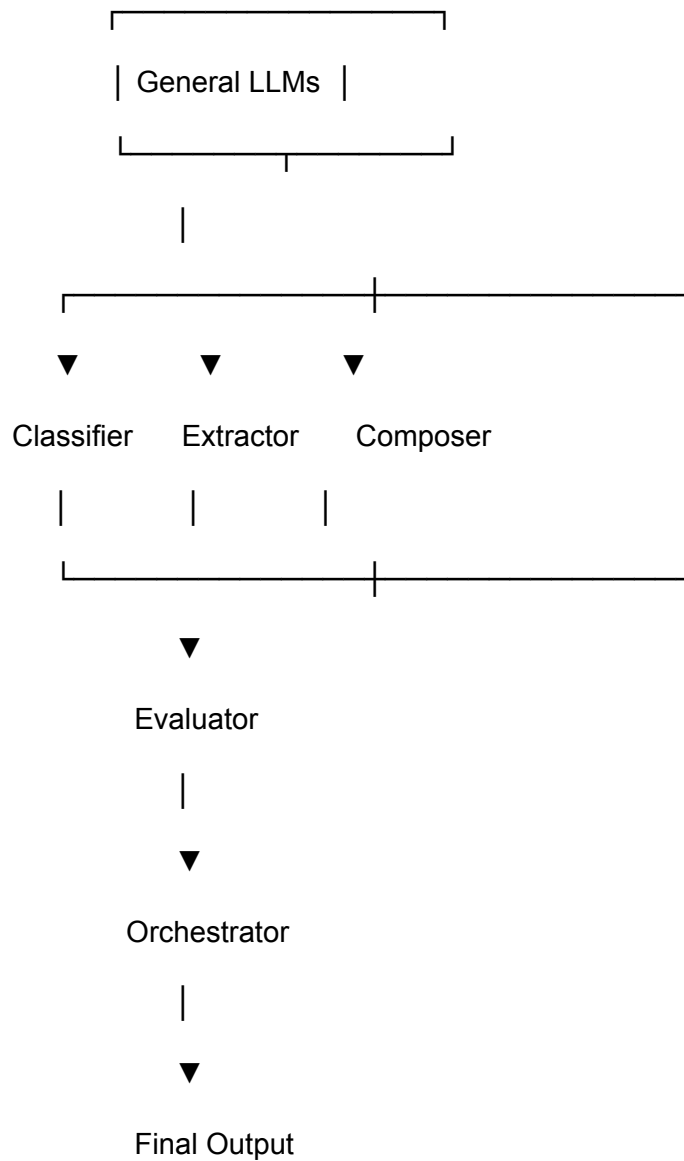
The score can then be calculated by your application.

---

## 9. So the "brain" isn't one model

This is the key architectural idea.

Your brain is:



The **orchestrator** is your actual product logic.

The LLMs are components inside it.

That distinction is important if you're serious about building this as software rather than making a prompt-wrapper.

---

## One technical choice I'd make now

For V1:

**Don't fine-tune.**

Build the Image Engine using:

**LLM + structured JSON schemas + deterministic application logic + independent evaluator + model gateway.**

Then log every transformation.

Those logs become your future training/evaluation dataset.

That gives us something we can actually improve instead of blindly tweaking prompts.

---

I'm stopping here deliberately.

The next technical question should be **the Image Engine's actual data contract**: what JSON objects, fields, schemas, states, and scores move between these components.

That is where we start getting into the actual application code architecture.

# IMAGE ENGINE SPEC

This is the **Image Engine only**. The goal is to give you something you can actually translate into the application architecture.

---

## 1. Image Engine Pipeline

USER INPUT

↓

Input Processor

↓

Image Intent Classifier

↓

Image Specification Extractor



Image Gap Analyzer



Safe Inference Engine



Prompt Composer



Independent Image Evaluator



Revision Controller



FINAL IMAGE PROMPT

For an uploaded image:

IMAGE + USER INSTRUCTION



Vision Analyzer



Image Specification Extractor



same pipeline

The important point is that **generation and evaluation are separate.**

---

## 2. Input Processor

The first component determines what actually arrived.

### Supported input

```
{  
  "input_type": "text | image | text_image",  
  "raw_input": "...",  
  "image_reference": null,  
  "requested_action": null  
}
```

It should normalize:

- pasted text
- uploaded image
- image + instruction
- existing image-generation prompt
- rough image idea

No scoring yet.

---

## 3. Image Intent Classifier

The classifier determines two things:

```
{  
  "modality": "image",  
  "operation": "generate",  
  "input_state": "raw_idea",  
  "confidence": 0.97
```

```
}
```

### operation

Use a controlled enum:

generate

edit

transform

recreate

compose

extend

remove

style\_transfer

analyze

unknown

### input\_state

raw\_idea

partial\_prompt

developed\_prompt

image\_reference

image\_and\_prompt

This classification controls everything downstream.

---

## 4. Image Specification Extractor

This is the first major LLM task.



It converts messy language into structured visual information.

```
{  
  "subject": [],  
  "subject_attributes": [],  
  "action": [],  
  "environment": [],  
  "composition": [],  
  "camera": [],  
  "lighting": [],  
  "style": [],  
  "mood": [],  
  "color": [],  
  "materials": [],  
  "depth": [],  
  "technical": [],  
  "negative_constraints": [],  
  "explicit_details": [],  
  "ambiguous_details": []  
}
```

Example:

"A Ghanaian king standing in a palace."

becomes approximately:

```
{  
  "subject": ["Ghanaian king"],
```

```
"action": ["standing"],
"environment": ["royal palace"],
"composition": [],
"camera": [],
"lighting": [],
"style": [],
"mood": [],
"explicit_details": [
  "Ghanaian",
  "king",
  "standing",
  "palace"
]
}
```

The extractor **must not fill missing fields with imagination.**

Its job is extraction.

---

## 5. Image Gap Analyzer

Now a separate reasoning step examines the specification.

It determines:

```
{
  "missing": [
    "composition",
```

```
"lighting",  
"visual_style"  
],  
"optional": [  
  "camera",  
  "color"  
],  
"unnecessary": [  
  "technical_parameters"  
]  
}
```

The key distinction:

### **Required**

Information genuinely needed to satisfy the user's intent.

### **Helpful**

Information that can improve the result but can reasonably be inferred.

### **Optional**

Details that shouldn't block generation.

### **Unknown**

Information that would require the user to decide.

This prevents the engine from asking stupid questions.

---

## **6. Safe Inference Engine**

This is another LLM call with strict rules.

It receives the extracted specification and gap analysis.

Its job:

**Fill useful gaps without changing the user's intent.**

Example:

```
{
  "safe_inferences": [
    {
      "field": "lighting",
      "value": "cinematic directional lighting",
      "confidence": 0.87
    },
    {
      "field": "composition",
      "value": "strong central subject framing",
      "confidence": 0.82
    }
  ],
  "unsafe_inferences": [
    {
      "field": "historical_period",
      "reason": "not specified by user"
    }
  ]
}
```

```
}
```

## Hard rule

The engine may infer **visual treatment** more freely than **identity, history, culture, factual circumstances, or user intent**.

That's an important safeguard.

---

# 7. Prompt Composer

Now the composer receives:

Original input

+

Intent

+

Extracted specification

+

Missing information

+

Safe inferences

+

User constraints

It produces:

```
{
```

```
  "prompt": "...",
```

```
  "negative_prompt": "...",
```

```
  "assumptions": [],
```

```
"remaining_gaps": []  
}
```

The prompt should be:

**structured internally, natural externally.**

Don't expose ugly JSON to the user.

---

## 8. The Independent Image Evaluator

This is where the engine becomes much more serious.

The evaluator should **not ask "Does this sound like a good prompt?"**

That's too subjective.

It should evaluate the generated prompt against the original intent and the image specification.

### **Evaluator dimensions**

INTENT PRESERVATION

SUBJECT CLARITY

VISUAL SPECIFICITY

COMPOSITION

SCENE / ENVIRONMENT

LIGHTING

STYLE

COHERENCE

CONSTRAINTS

TECHNICAL USABILITY

UNSUPPORTED ASSUMPTIONS

## REDUNDANCY

Each receives:

```
{  
  "score": 0,  
  "reason": "",  
  "severity": "none | low | medium | high"  
}
```

Example:

```
{  
  "intent_preservation": {  
    "score": 96,  
    "reason": "All explicit user requirements remain intact."  
  },  
  "visual_specificity": {  
    "score": 82,  
    "reason": "Subject and environment are defined, but composition remains moderately generic."  
  },  
  "unsupported_assumptions": {  
    "score": 91,  
    "reason": "No significant factual or identity claims were invented."  
  }  
}
```

---

## 9. The evaluator's most important test

It should compare:

USER INTENT



GENERATED PROMPT

and ask:

**Did we improve the image instruction without hijacking the idea?**

This deserves more weight than stylistic sophistication.

I'd weight the evaluation roughly:

```
{  
  "intent_preservation": 0.25,  
  "subject_clarity": 0.15,  
  "visual_specificity": 0.15,  
  "composition": 0.10,  
  "environment": 0.08,  
  "lighting": 0.07,  
  "style": 0.07,  
  "coherence": 0.06,  
  "constraints": 0.04,  
  "technical_usability": 0.03  
}
```

These are **initial weights**, not permanent truths.

Your later evaluation data should change them.

---



## 10. Unsupported Assumption Detection

This deserves its own evaluator.

For every major addition:

```
{  
  "added_detail": "18th-century Ashanti palace",  
  "source": "model_inference",  
  "supported_by_user": false,  
  "safe_to_add": false  
}
```

If the model adds:

"18th-century Ashanti palace"

when the user only said:

"Ghanaian king"

the system should remove it or mark it as an assumption.

But:

"dramatic directional lighting"

may be acceptable because it's visual treatment rather than an invented fact.

---

## 11. Revision Controller

The evaluator doesn't directly rewrite the prompt.

It returns problems.

Then a **Revision Controller** decides whether another generation pass is justified.

```
{
```

```
"revision_required": true,  
"issues": [  
  {  
    "type": "intent_drift",  
    "severity": "high"  
  },  
  {  
    "type": "composition_weakness",  
    "severity": "medium"  
  }  
]  
}
```

Then:

Composer

↓

Evaluator

↓

Revision Controller

↓

PASS? — yes → output

|

no

↓

Composer again

Set a hard maximum, such as **2 revision cycles**.

Don't let LLMs endlessly critique themselves.

---

## 12. Final Output Contract

The frontend should receive something clean:

```
{  
  "modality": "image",  
  "operation": "generate",  
  "input_state": "raw_idea",  
  
  "quality": {  
    "score": 91,  
    "confidence": 0.89  
  },  
  
  "original": "A Ghanaian king standing in a palace.",  
  
  "enhanced_prompt": "...",  
  
  "improvements": [  
    "Added visual composition",  
    "Added cinematic lighting",  
    "Clarified environmental detail"
```

],

"remaining\_gaps": [

"Specific historical period",

"Preferred artistic style"

],

"assumptions": [

"Used a cinematic visual treatment"

]

}

Notice what is **not** there:

A giant questionnaire.

The user gets the result first.

---

## 13. What the user sees

The engine can return all that complexity, while the interface remains extremely simple.

|                             |  |
|-----------------------------|--|
|                             |  |
| YOUR IDEA                   |  |
|                             |  |
| A Ghanaian king standing in |  |
| a palace.                   |  |
|                             |  |



|                            |  |
|----------------------------|--|
| YOUR ENHANCED IMAGE PROMPT |  |
|                            |  |
| [Generated prompt.....]    |  |
|                            |  |
| Copy Prompt                |  |

91 / 100

Improved:

- Visual composition
- Lighting
- Scene detail

Could improve further:

Historical period · Visual style

That's enough for the first interaction.

**No interrogation.**

---

# 14. Recommended technical stack

For V1, I'd use:

Frontend

React / Next.js

Backend

TypeScript / Node.js

LLM orchestration

Structured LLM calls + JSON Schema

Primary reasoning model

Strong multimodal/general LLM

Vision

Multimodal model when an image is supplied

Database

PostgreSQL

Queue

Redis / background jobs if processing becomes heavy

Observability

Prompt/input/output/evaluation logging

Model gateway

Your own abstraction layer

The model provider should sit behind your interface.

```
interface ModelProvider {  
  generateStructured<T>(  
    input: unknown,  
    schema: JSONSchema  
  ): Promise<T>;  
}
```

Then you can swap models without rewriting the application.

---

## 15. What I would NOT build yet

For the Image Engine V1:

**No fine-tuning.**

**No custom foundation model.**

**No complicated agent swarm.**

**No autonomous infinite refinement.**

**No giant image-prompt knowledge database.**

Build the pipeline first.

Then collect:

raw input

→ extracted specification

→ generated prompt

→ evaluator result

→ user's edits

→ final prompt

→ user feedback

That becomes your training and evaluation dataset.

Later, if that dataset proves that a smaller specialized model can reliably perform classification, extraction, or evaluation, **then** fine-tuning becomes worth considering.

---

## The Image Engine, reduced to one line

UNDERSTAND → STRUCTURE → IDENTIFY GAPS → SAFELY INFER → COMPOSE →  
INDEPENDENTLY EVALUATE → REVISE → RETURN

That is the Image Engine we should build.

**Stop here.** The next modality should only begin once we're satisfied that this Image architecture is the one we want.

Yes, **it can increase latency**, but it does not have to push us beyond your 10 to 20 second target.

The important distinction is that these stages don't all need to be separate, sequential LLM calls.

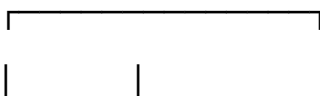
For V1, I would **parallelize and collapse stages where possible**:

User input

↓

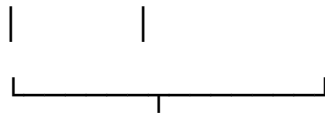
Fast classification + extraction

↓





Gap analysis    Safe inference



Prompt composer



Fast evaluator



Output

You can also skip evaluation/revision for straightforward cases. If the first generation already passes the evaluator threshold, there is no reason to burn another model call.

So the architecture should have a **latency budget**, not simply "call every model every time."

Something like:

- Input/classification: very fast model
- Extraction + analysis: one structured call
- Composition: one strong call
- Evaluation: fast evaluator, potentially parallelized with other post-processing
- Revision: only when necessary

That should keep us in the target range with sensible model selection and streaming.

And for the first-touch experience, I'd prioritize **time-to-first-useful-output** over completing every internal diagnostic. The user should start seeing the result while secondary metadata finishes.

So yes, we're good to move on.

## Next: Video Engine

We'll use the same technical depth we just established for Image, but **we will not blindly copy the Image architecture**. Video has its own problems: temporal consistency, motion, sequencing, camera movement, duration, transitions, audio, subject continuity, and so on.

We'll design **only the Video Engine** next.

# VIDEO ENGINE

The video engine should not be treated as "an image prompt with movement added." Video has a **time dimension**, so the engine needs to reason about what happens **before, during, and after** each visual event.

## 1. Core pipeline

RAW INPUT



Input Processor



Video Intent Classifier



Video Specification Extractor



Temporal / Motion Analyzer



Safe Inference Engine



Video Prompt Composer



Independent Video Evaluator



Revision Controller



FINAL VIDEO PROMPT / SHOT PLAN

The output may be a single video prompt or a structured shot plan depending on the user's request.

---

## 2. Video Intent Classifier

The first model determines what the user actually wants.

```
{  
  "modality": "video",  
  "operation": "generate",  
  "input_state": "raw_idea",  
  "confidence": 0.96  
}
```

### Operations

generate

animate

transform

extend

edit

recreate

storyboard

advertisement

product\_demo

social\_video

cinematic\_scene

unknown

## Input states

raw\_idea

partial\_prompt

developed\_prompt

image\_to\_video

video\_to\_video

script

storyboard

reference\_media

This matters because:

"Make a video of a woman walking through Paris"

and

"Animate this image so the woman walks toward the camera"

are completely different jobs.

---

## 3. Video Specification Extractor

The extractor converts the user's input into a structured representation.

```
{  
  "subject": [],  
  "action": [],  
  "environment": [],  
  "story": [],  
  "duration": null,
```

```
"aspect_ratio": null,  
"camera": [],  
"camera_movement": [],  
"subject_movement": [],  
"scene_movement": [],  
"framing": [],  
"lighting": [],  
"visual_style": [],  
"mood": [],  
"color": [],  
"transition": [],  
"audio": [],  
"dialogue": [],  
"text_overlay": [],  
"timing": [],  
"continuity_requirements": [],  
"negative_constraints": []  
}
```

Again, **extraction does not invent missing information.**

---

## 4. The temporal layer

This is the major difference from Image.

The engine needs to understand:

TIME 0s



EVENT



MOTION



CHANGE



END STATE

For example:

"A businessman enters an office, walks to a desk, sits down and opens his laptop."

The engine should internally represent:

{

"timeline": [

{

"time": "0-2s",

"event": "businessman enters office"

},

{

"time": "2-5s",

"event": "walks toward desk"

},

{

"time": "5-7s",

```
    "event": "sits down"
  },
  {
    "time": "7-10s",
    "event": "opens laptop"
  }
]
```

This prevents the composer from producing a beautiful description that doesn't actually describe a coherent video.

---

## 5. Motion Analyzer

The engine should separately identify:

### **Subject motion**

walk

run

turn

reach

speak

gesture

fall

dance

### **Camera motion**

static

pan

tilt

dolly

tracking

orbit

crane

zoom

handheld

## **Environmental motion**

wind

rain

smoke

water

crowds

traffic

particles

These should not be thrown into one sentence.

The engine needs to understand their relationships.

For example:

The camera tracks backward while the subject walks toward it.

is different from:

The camera remains static while the subject walks across frame.

That difference matters to a video model.



---

## 6. Continuity Analyzer

This should be a dedicated component.

Video generation frequently breaks when:

- clothing changes
- faces change
- objects disappear
- environments shift
- body proportions change
- lighting jumps
- camera position suddenly changes

So the engine should generate continuity constraints.

```
{  
  "continuity": {  
    "identity": true,  
    "clothing": true,  
    "environment": true,  
    "object_state": true,  
    "lighting": true,  
    "camera_logic": true  
  }  
}
```

For example:

Maintain the same man's facial identity, clothing, body proportions, and hairstyle throughout the sequence.

That's not merely decorative prompt language. It's an instruction addressing a known failure mode.

---

## 7. Safe Inference Engine

Same philosophy as Image.

The engine can safely infer:

cinematic camera movement

if the user asks for a cinematic sequence.

But it shouldn't invent:

30-second advertisement for Nike

when the user only said:

"Create a running video."

The inference model should return:

```
{  
  "safe_inferences": [],  
  "unsafe_assumptions": []  
}
```

And each inference should carry confidence.

---

## 8. Video Composer

The composer receives:

Original input

+

Intent

+

Video specification

+

Timeline

+

Motion specification

+

Continuity constraints

+

Safe inferences

Then produces either:

### **Single-shot prompt**

For simple requests.

Or:

### **Structured shot plan**

For complex requests.

Example:

```
{  
  "format": "shot_plan",  
  "shots": [  
    {  
      "shot": 1,  
      "duration": "0-4s",  
      "description": "...",
```

```
"camera": "...",  
"motion": "...",  
"continuity": "..."  
},  
{  
  "shot": 2,  
  "duration": "4-8s",  
  "description": "...",  
  "camera": "...",  
  "motion": "...",  
  "continuity": "..."  
}  
]  
}
```

This distinction is important.

A 5-second image-to-video instruction shouldn't become a ridiculous screenplay.

---

## 9. Independent Video Evaluator

The evaluator should judge **video-specific qualities**, not simply whether the prose sounds impressive.

### Core evaluation dimensions

INTENT PRESERVATION

TEMPORAL COHERENCE

MOTION CLARITY

SUBJECT CONSISTENCY

CAMERA LOGIC

SCENE CONSISTENCY

ACTION SEQUENCING

VISUAL SPECIFICITY

STYLE CONSISTENCY

TECHNICAL COMPLETENESS

UNSUPPORTED ASSUMPTIONS

REDUNDANCY

Example:

```
{  
  "intent_preservation": 95,  
  "temporal_coherence": 88,  
  "motion_clarity": 91,  
  "subject_consistency": 94,  
  "camera_logic": 83,  
  "scene_consistency": 90,  
  "action_sequencing": 87,  
  "visual_specificity": 86,  
  "style_consistency": 92,  
  "technical_completeness": 78,  
  "unsupported_assumptions": 96  
}
```

---

## 10. Video evaluator's most important test

The evaluator should ask:

**Can a video-generation model actually execute what this prompt describes over time?**

That is the key difference.

For example:

"A woman runs through a forest while the camera circles around her, the weather changes from sunny to stormy, she changes clothes, the forest transforms into a city, then she becomes an old woman."

Technically imaginable.

Practically chaotic.

The evaluator should flag:

```
{  
  "temporal_complexity": {  
    "score": 42,  
    "severity": "high",  
    "reason": "Too many simultaneous transformations for a single coherent generation."  
  }  
}
```

The system can then recommend breaking it into shots.

---

## 11. Revision Controller

Same principle.

Composer

↓  
Evaluator

↓  
Pass?

┌  
└  
YES NO

| |  
▼ ▼

Done Revise

Maximum two revision passes.

If the problem is complexity rather than wording, the controller should change the **structure**, not simply rewrite the sentence.

For example:

Single 20-second generation

↓  
4 × 5-second shots

That's an actual improvement.

---

## 12. Final output

The frontend receives:

```
{  
  "modality": "video",  
  "operation": "generate",  
  "input_state": "raw_idea",
```

"quality": {

  "score": 89,

  "confidence": 0.91

},

"enhanced\_prompt": "...",

"format": "single\_prompt",

"improvements": [

  "Clarified camera movement",

  "Defined subject motion",

  "Added temporal sequencing"

],

"remaining\_gaps": [

  "Exact duration",

  "Preferred aspect ratio"

],

"assumptions": [

  "Used cinematic camera treatment"

]



```
}
```

For a complex request:

```
{
```

```
  "format": "shot_plan",
```

```
  "shots": []
```

```
}
```

---

## 13. Technical implementation

Keep the same provider abstraction:

```
interface ModelProvider {
```

```
  generateStructured<T>(  
    input: unknown,
```

```
    schema: JSONSchema
```

```
  ): Promise<T>;
```

```
}
```

Then your Video Engine becomes an orchestration service:

```
async function processVideoInput(input: VideoInput) {
```

```
  const intent = await classifyVideoIntent(input);
```

```
  const spec = await extractVideoSpec(input, intent);
```

```
  const temporal = await analyzeTemporalStructure(spec);
```

```
const inferences = await inferSafeDetails(  
  spec,  
  temporal  
);
```

```
const draft = await composeVideoPrompt({  
  input,  
  intent,  
  spec,  
  temporal,  
  inferences  
});
```

```
const evaluation = await evaluateVideoPrompt(  
  input,  
  draft,  
  spec,  
  temporal  
);
```

```
return reviselfNecessary(  
  draft,  
  evaluation  
);
```

```
}
```

The important optimization:

**Do not make every function an LLM call.**

Some are ordinary TypeScript logic.

Some can be combined into one structured LLM request.

Some can run in parallel.

That keeps the 10 to 20 second target realistic.

---

## 14. Latency strategy

For simple video prompts:

Classification

↓

Extraction + temporal analysis

↓

Composition

↓

Evaluation

↓

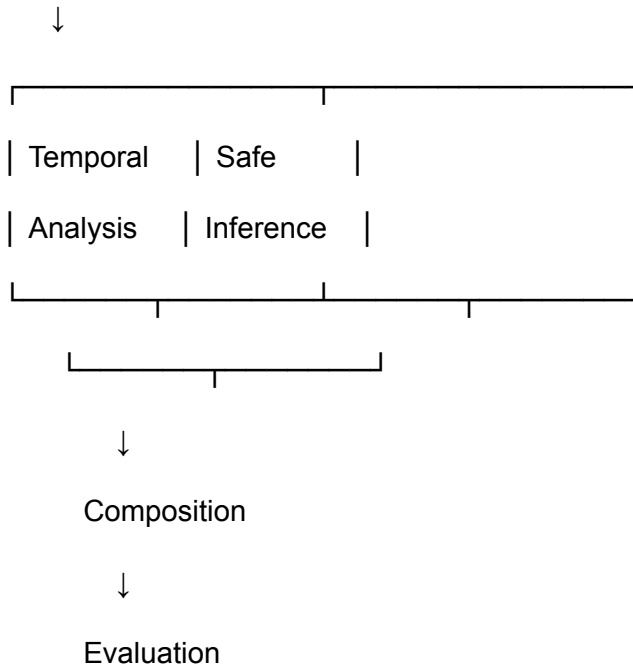
Output

For complex requests:

Classification

↓

Extraction



And again, **skip revision when the evaluator passes the result.**

The user should not wait for unnecessary internal perfection.

---

## The Video Engine in one line

UNDERSTAND → STRUCTURE TIME → DEFINE MOTION → PROTECT CONTINUITY →  
COMPOSE → EVALUATE → REVISE IF NEEDED → RETURN

The defining difference from Image is:

**Image asks, "What should exist?"**

**Video asks, "What should exist, and what happens to it over time?"**

That is the part the engine must understand before we move to Text.

## TEXT ENGINE

Text is broader than the previous two, so the engine must first determine **what kind of textual task the user is actually trying to perform.**

The mistake would be treating every text input as "write a better prompt."

# 1. Core pipeline

RAW INPUT



Input Processor



Text Intent Classifier



Task / Context Extractor



Requirement & Gap Analyzer



Safe Inference Engine



Prompt Composer



Independent Text Evaluator



Revision Controller



FINAL TEXT PROMPT

---

## 2. Text Intent Classifier

The first model identifies the task.

```
{  
  "modality": "text",  
  "operation": "generate",  
  "input_state": "raw_idea",  
  "confidence": 0.98  
}
```

## **Operations**

generate

rewrite

summarize

analyze

research

extract

classify

translate

brainstorm

evaluate

transform

edit

continue

compare

plan

unknown

## **Input states**

raw\_idea

partial\_prompt

developed\_prompt

source\_text

document

requirements

question

instruction

This matters because:

"Write a research paper about CKD"

is fundamentally different from:

"Rewrite this research paper in simpler English."

---

## 3. Text Task Extractor

The extractor identifies what the requested output actually needs.

```
{  
  "task": {  
    "objective": "",  
    "output_type": "",  
    "audience": "",  
    "purpose": ""  
  },  
}
```

```
"context": [],  
  
"source_material": [],  
  
"constraints": [],  
  
"tone": [],  
"format": [],  
"length": [],  
"language": [],  
  
"quality_requirements": [],  
"evaluation_criteria": [],  
  
"explicit_details": [],  
"ambiguous_details": []  
}
```

The extractor **doesn't** invent missing information.

---

## 4. Task-specific requirement model

Unlike Image, Text needs a **task-specific schema**.

For example:

**Research task**



research question

scope

academic level

source requirements

citation requirements

argument requirements

structure

length

## **Marketing task**

audience

offer

objective

channel

brand

CTA

tone

constraints

## **Summarization task**

source

target length

audience

focus

preservation requirements

format

## Copywriting task

audience

product

pain point

desired action

positioning

tone

channel

constraints

So internally:

TEXT

↓

TASK TYPE

↓

TASK-SPECIFIC RUBRIC

This prevents one generic text rubric from being applied to everything.

---

## 5. Requirement & Gap Analyzer

The analyzer now asks:

**What does this particular task require, and what has the user already supplied?**

Example:

"Write a landing-page headline for my AI tool."

The engine might determine:

```
{  
  "required": [  
    "product/category",  
    "core benefit"  
  ],  
  "helpful": [  
    "target audience",  
    "differentiator"  
  ],  
  "optional": [  
    "brand tone"  
  ]  
}
```

But it should **still produce a headline**.

It should not stop and ask four questions.

---

## 6. Safe Inference Engine

The inference rules differ from Image.

For text, the engine must be much more careful about **facts**.

For example:

"Write a report about India's healthcare system."

The engine can infer:

formal analytical structure

It should **not invent statistics, studies, government policies, or citations**.

So the inference engine needs categories:

```
{  
  "safe_inferences": [],  
  "requires_user_input": [],  
  "requires_research": [],  
  "must_not_invent": []  
}
```

Example:

```
{  
  "safe_inferences": [  
    "formal analytical tone"  
  ],  
  "requires_research": [  
    "current healthcare statistics"  
  ],  
  "must_not_invent": [  
    "citations",  
    "statistics",  
    "named studies"  
  ]  
}
```

This becomes especially important for academic and research prompts.

---

## 7. Prompt Composer

The composer receives:

Original input

+

Task type

+

Task-specific requirements

+

Context

+

Safe inferences

+

Constraints

+

Known gaps

It produces:

```
{  
  "prompt": "...",  
  "assumptions": [],  
  "remaining_gaps": [],  
  "research_required": false  
}
```

The output should tell the downstream AI **what to do, with what information, under what conditions, and what the result should look like.**

---

## 8. Independent Text Evaluator

The evaluator should not simply judge whether the prompt is grammatically good.

It evaluates whether the prompt can reliably produce the requested result.

### Core dimensions

INTENT PRESERVATION

TASK CLARITY

CONTEXT SUFFICIENCY

REQUIREMENT COMPLETENESS

OUTPUT SPECIFICATION

AUDIENCE DEFINITION

CONSTRAINT QUALITY

FACTUAL SAFETY

SOURCE / EVIDENCE REQUIREMENTS

STRUCTURAL COHERENCE

AMBIGUITY

REDUNDANCY

Example:

```
{  
  "intent_preservation": 96,  
  "task_clarity": 94,  
  "context_sufficiency": 72,  
  "requirement_completeness": 81,
```

```
"output_specification": 90,  
"audience_definition": 60,  
"constraint_quality": 76,  
"factual_safety": 98,  
"structural_coherence": 93,  
"ambiguity": 84,  
"redundancy": 92  
}
```

---

## 9. Text evaluator's key test

The most important question is:

**Could another capable LLM execute this instruction consistently without having to guess what the user wants?**

That is better than asking:

"Is this a good prompt?"

For example:

"Write me something professional about my business."

The evaluator should identify:

```
{  
  "task_clarity": 38,  
  "context_sufficiency": 21,  
  "output_specification": 17  
}
```

But the composer still produces the best useful prompt possible from the information available.

---

## 10. Factual integrity becomes a first-class control

This is one area where Text differs sharply from Image and Video.

The evaluator should detect whether the enhanced prompt accidentally turns uncertainty into fact.

Example:

User:

"Write about how AI increased productivity in hospitals."

The engine shouldn't silently convert that into:

"Explain how AI increased hospital productivity by 37%..."

That number came from nowhere.

Instead:

```
{  
  "unsupported_claim": true,  
  "action": "remove"  
}
```

If evidence is required:

```
{  
  "research_required": true,  
  "instruction": "Use verifiable sources and distinguish evidence from inference."  
}
```

---



# 11. Revision Controller

Composer



Evaluator



Pass?



YES NO



Done Revise

Revision should target the actual failure.

If the problem is:

missing output structure

add structure.

If the problem is:

unsupported factual assumptions

remove them.

If the problem is:

unnecessary complexity

simplify.

Maximum two revision passes.

---

## 12. Final output contract

```
{  
  "modality": "text",  
  "operation": "generate",  
  "input_state": "raw_idea",  
  
  "quality": {  
    "score": 88,  
    "confidence": 0.94  
  },  
  
  "enhanced_prompt": "...",  
  
  "improvements": [  
    "Defined the output objective",  
    "Specified the target audience",  
    "Added a required response structure"  
  ],  
  
  "remaining_gaps": [  
    "Exact word count"  
  ],  
  
  "research_required": false,
```

```
"assumptions": [  
  "Used a professional tone"  
]  
}
```

---

## 13. One important architectural difference

For Text, I would add a **Task Rubric Registry**.

Instead of hard-coding one evaluator:

```
const rubric = TEXT_RUBRIC;
```

use:

```
const rubric = rubricRegistry.get(taskType);
```

So:

research

→ research rubric

marketing

→ marketing rubric

copywriting

→ copywriting rubric

summarization

→ summarization rubric

creative writing

→ creative-writing rubric

This is much more scalable.

The engine stays the same.

The **evaluation criteria change according to the job.**

---

## 14. Technical implementation

```
async function processTextInput(input: TextInput) {
```

```
  const intent = await classifyTextIntent(input);
```

```
  const task = await extractTextRequirements(
```

```
    input,
```

```
    intent
```

```
  );
```

```
  const rubric = rubricRegistry.get(
```

```
    task.taskType
```

```
  );
```

```
  const gaps = analyzeRequirements(
```

```
    task,
```

```
    rubric  
  );
```

```
const inferences = await inferSafeDetails(  
  task,  
  gaps  
);
```

```
const draft = await composeTextPrompt({  
  input,  
  intent,  
  task,  
  rubric,  
  gaps,  
  inferences  
});
```

```
const evaluation = await evaluateTextPrompt({  
  input,  
  task,  
  rubric,  
  draft  
});
```

```
return reviselfNecessary(  
    draft,  
    evaluation  
);  
}
```

Again, not every function needs an LLM.

**Classification, registry lookup, weighting, validation, schema checks, and threshold decisions can be ordinary application code.**

The LLM handles interpretation and language reasoning.

---

## 15. The Text Engine in one line

UNDERSTAND TASK → EXTRACT REQUIREMENTS → APPLY TASK-SPECIFIC RUBRIC → SAFELY INFER → COMPOSE → FACT-CHECK LOGIC → EVALUATE → REVISE → RETURN

The defining question for Text is:

**"What does the user want the AI to accomplish, and what must the AI know and produce to accomplish it reliably?"**

## CONVERSATIONAL ENGINE

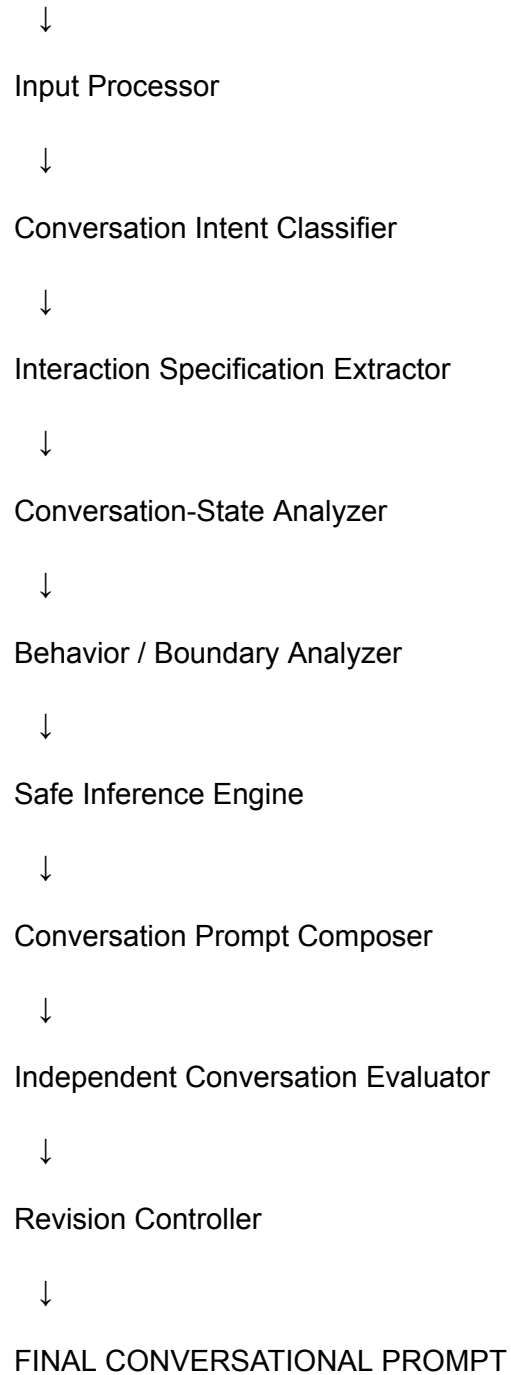
Conversational prompting is different because the user is not necessarily asking for a single output. They may be designing **how an AI should behave across multiple turns**.

The engine therefore needs to reason about **roles, state, turn-taking, memory, boundaries, and interaction goals**.

---

### 1. Core pipeline

RAW INPUT



## 2. Conversation Intent Classifier

{

```
"modality": "conversational",  
"operation": "design_behavior",  
"input_state": "raw_idea",  
"confidence": 0.97  
}
```

## **Operations**

design\_assistant

roleplay

coach

tutor

interviewer

customer\_support

research\_assistant

agent

companion

workflow\_assistant

conversation\_simulation

improve\_existing\_behavior

unknown

## **Input states**

raw\_idea

partial\_instruction

system\_prompt

conversation\_example



multi\_turn\_specification

existing\_conversation

---

### 3. Interaction Specification Extractor

The extractor converts the user's idea into a behavioral model.

```
{  
  "role": "",  
  "primary_goal": "",  
  "secondary_goals": [],  
  
  "user_profile": {},  
  "assistant_behavior": [],  
  
  "tone": [],  
  "communication_style": [],  
  
  "conversation_strategy": [],  
  
  "turn_behavior": {  
    "ask_questions": null,  
    "clarify": null,  
    "summarize": null,  
    "confirm": null,  
  },  
}
```

```
"challenge_user": null  
},
```

```
"memory": {  
  "required": [],  
  "allowed": [],  
  "forbidden": []  
},
```

```
"boundaries": [],  
"constraints": [],
```

```
"output_behavior": [],  
"failure_behavior": [],
```

```
"explicit_requirements": [],  
"ambiguities": []  
}
```

The key difference from Text:

**The output isn't just text.**

It is a **behavioral specification**.

---

## 4. Conversation-State Analyzer

A conversational system needs to understand state.

For example:

Turn 1

User: I want to learn Python.

Turn 2

Assistant: What's your current level?

Turn 3

User: Beginner.

Turn 4

Assistant: ...

The engine should understand:

```
{  
  "state": {  
    "known": [  
      "user wants to learn Python",  
      "user is a beginner"  
    ],  
    "unknown": [  
      "learning objective"  
    ],  
    "current_goal": "determine learning path",  
  },  
}
```

```
"conversation_stage": "discovery"  
  
}  
  
}
```

This prevents the generated prompt from behaving as though every turn is independent.

---

## 5. Turn Strategy

The engine should explicitly define when the assistant should:

answer immediately

ask a question

ask one question

ask multiple questions

confirm understanding

challenge an assumption

summarize

offer options

refuse

redirect

This is especially important for your product.

A weak conversational prompt often says:

"Be a helpful assistant."

That's almost useless.

A stronger specification says:

When the user's request is sufficiently clear, answer directly. When essential information is missing, ask only the minimum question required to proceed. Do not interrogate the user unnecessarily.

Now the model has actual behavior rules.

---

## 6. Conversation Memory Analyzer

The engine should distinguish:

### **Session memory**

Information from the current conversation.

### **Persistent memory**

Information that should survive conversations.

### **Temporary context**

Information relevant only to the current task.

### **Forbidden memory**

Information the assistant should not retain or rely on.

```
{  
  "memory_policy": {  
    "session": [  
      "current project",  
      "current objective"  
    ],  
    "persistent": [  
      "preferred response format"  
    ],  
  },  
}
```

```
"temporary": [  
  "draft text"  
],  
"forbidden": [  
  "unnecessary personal information"  
]  
}  
}
```

This becomes especially important once your application supports persistent user workspaces.

---

## 7. Safe Inference Engine

The conversational engine can infer things such as:

"The assistant should probably be concise."

But it should not silently decide:

"The assistant should always challenge the user."

That changes the relationship.

So inference categories should include:

```
{  
  "safe_inferences": [],  
  "behavioral_assumptions": [],  
  "requires_user_decision": [],  
  "conflicting_requirements": []  
}
```

If the user says:

"Build me an AI tutor."

The system can infer:

explain concepts progressively.

But it shouldn't arbitrarily decide:

use Socratic questioning exclusively.

That is a design choice.

---

## 8. Conversation Prompt Composer

The composer receives:

User intent

+

Role

+

Goals

+

Behavior rules

+

Turn strategy

+

Memory policy

+

Boundaries

+

Failure behavior

+

Safe inferences

And generates the conversational instruction.

The resulting prompt should define:

ROLE

PURPOSE

CONTEXT

BEHAVIOR

TURN RULES

MEMORY

BOUNDARIES

OUTPUT STYLE

FAILURE HANDLING

Not simply:

"You are a helpful tutor."

---

## 9. Independent Conversation Evaluator

This evaluator needs to test **behavior over multiple turns**.

That is the major difference.

### Evaluation dimensions

INTENT PRESERVATION



ROLE CONSISTENCY

GOAL ALIGNMENT

TURN MANAGEMENT

CONTEXT RETENTION

MEMORY DISCIPLINE

CLARIFICATION BEHAVIOR

QUESTION QUALITY

BEHAVIOR CONSISTENCY

BOUNDARY HANDLING

FAILURE RECOVERY

STYLE CONSISTENCY

UNNECESSARY VERBOSITY

Example:

```
{  
  "intent_preservation": 95,  
  "role_consistency": 93,  
  "goal_alignment": 96,  
  "turn_management": 89,  
  "context_retention": 91,  
  "memory_discipline": 94,  
  "clarification_behavior": 87,  
  "question_quality": 90,  
  "boundary_handling": 92,  
  "failure_recovery": 84,
```

"style\_consistency": 95

}

---

## 10. The evaluator should simulate conversations

This is important.

Don't evaluate a conversational prompt from the prompt alone.

Create synthetic test turns.

For example:

User:

"I want to learn biology."

Assistant:

?

User:

"Actually, I'm preparing for an exam."

Assistant:

?

User:

"I don't understand renal physiology."

Assistant:

?

Then the evaluator checks:

Did the assistant remember the exam goal?

Did it adapt its teaching?

Did it ask an unnecessary question?

Did it maintain its assigned role?

Did it contradict something established earlier?

This makes the evaluator substantially more useful.

---

## 11. Conversation Test Suite

The engine should automatically generate test scenarios.

```
{  
  "tests": [  
    "normal_request",  
    "ambiguous_request",  
    "missing_context",  
    "context_change",  
    "contradictory_user",  
    "repeated_question",  
    "off_topic_request",  
    "boundary_case",
```

```
"long_conversation",  
"goal_completion"  
]  
}
```

Each test produces:

```
{  
  "scenario": "missing_context",  
  "expected_behavior": "ask_minimum_clarifying_question",  
  "actual_behavior": "...",  
  "passed": true  
}
```

That gives you something much stronger than a single LLM score.

---

## 12. Revision Controller

The controller should identify whether the problem is:

role failure

context failure

memory failure

turn-taking failure

question failure

boundary failure

style failure

goal failure

Then revise the relevant instruction.

Example:

The assistant asks too many questions.

The solution shouldn't be:

"Make the assistant better."

It should modify the turn rule:

When the request can be answered reasonably with available information, answer directly. Ask a question only when the missing information materially changes the answer.

That's an actual behavioral correction.

---

## 13. Final output contract

```
{  
  "modality": "conversational",  
  "operation": "design_assistant",  
  
  "quality": {  
    "score": 92,  
    "confidence": 0.93  
  },  
  
  "enhanced_prompt": "...",
```

```
"behavior_model": {  
  "role": "...",  
  "goal": "...",  
  "turn_strategy": "...",  
  "memory_policy": "...",  
  "boundaries": "..."  
},  
  
"improvements": [  
  "Defined turn-taking behavior",  
  "Added context-retention rules",  
  "Specified clarification behavior"  
],  
  
"remaining_gaps": [],  
  
"assumptions": []  
}
```

---

## 14. Technical implementation

```
async function processConversationInput(  
  input: ConversationInput  
) {
```

```
const intent =
```

```
  await classifyConversationIntent(input);
```

```
const spec =
```

```
  await extractInteractionSpecification(
```

```
    input,
```

```
    intent
```

```
  );
```

```
const stateModel =
```

```
  await analyzeConversationState(spec);
```

```
const inferences =
```

```
  await inferConversationBehavior(
```

```
    spec,
```

```
    stateModel
```

```
  );
```

```
const draft =
```

```
  await composeConversationPrompt({
```

```
    input,
```

```
    intent,
```

```
    spec,
```

```
    stateModel,
```

```

        inferences

    });

const testSuite =

    buildConversationTestSuite(spec);

const evaluation =

    await evaluateConversationPrompt({

        draft,

        testSuite,

        spec

    });

return reviselfNecessary(

    draft,

    evaluation

);

}

```

The evaluator is the expensive-looking part, so don't run a huge simulated conversation suite for every trivial request.

For a simple:

"Create a friendly customer-support assistant."

you can use a lightweight evaluation.

For:



"Build a persistent research agent that remembers projects, challenges assumptions, handles conflicting instructions, and conducts multi-step interviews."

you run the deeper simulation.

---

## 15. The defining rule

The Image Engine asks:

**What should exist?**

The Video Engine asks:

**What should exist and what happens over time?**

The Conversational Engine asks:

**How should the AI behave as the interaction changes from turn to turn?**

So its core architecture is:

UNDERSTAND GOAL



MODEL ROLE + STATE



DEFINE BEHAVIOR



DEFINE TURN RULES



DEFINE MEMORY + BOUNDARIES



COMPOSE



SIMULATE



EVALUATE



REVISE



RETURN

That gives us the fourth engine.

## CODE ENGINE

The Code Engine is fundamentally different again.

The system isn't only improving instructions for generating code. It needs to understand **what software task is being requested, what technical environment is involved, what constraints apply, and what constitutes a successful implementation.**

### 1. Core pipeline

RAW INPUT



Input Processor



Code Intent Classifier



Technical Context Extractor



Requirement / Dependency Analyzer



Risk & Ambiguity Analyzer



Safe Inference Engine



Code Prompt Composer



Independent Code Evaluator



Revision Controller



FINAL CODE PROMPT

---

## 2. Code Intent Classifier

```
{  
  "modality": "code",  
  "operation": "generate",  
  "input_state": "raw_idea",  
  "confidence": 0.98  
}
```

### Operations

generate

debug

fix

refactor

modify

explain

review

optimize

test

architect

migrate

integrate

document

convert

unknown

## **Input states**

raw\_idea

partial\_prompt

developed\_prompt

code\_snippet

repository\_context

error\_message

technical\_requirements

architecture\_description

This distinction matters enormously.

"Build me a React dashboard"

is a generation task.

"This React dashboard crashes when I filter the table"

is debugging.

"Convert this Python script to TypeScript"

is migration.

The engine must route them differently.

---

### 3. Technical Context Extractor

The extractor builds a machine-readable representation of the development environment.

```
{  
  "project": {  
    "type": "",  
    "purpose": ""  
  },  
  
  "language": [],  
  "framework": [],  
  "runtime": [],  
  "libraries": [],  
  "database": [],  
  "apis": [],  
  
  "existing_code": [],  
  "architecture": [],
```

"inputs": [],

"outputs": [],

"functional\_requirements": [],

"non\_functional\_requirements": [],

"constraints": [],

"dependencies": [],

"errors": [],

"expected\_behavior": [],

"testing\_requirements": [],

"deployment\_requirements": [],

"explicit\_details": [],

"unknown\_details": []

}

The extractor must distinguish:

**What the user explicitly supplied**

from

**What the system inferred**

from

What needs verification.

---

## 4. Repository and code-context handling

This is where the Code Engine should eventually become much stronger than a normal prompt improver.

If the user supplies code:

User instruction

+

Code

the engine should analyze both.

If the application has repository access:

User request

+

Relevant repository files

+

Project configuration

+

Dependencies

should become the context.

But the engine should **not blindly dump an entire repository into an LLM**.

A context-selection layer should identify relevant files.

Repository

↓

Code Index

↓

Relevant files

↓

Context window

That saves tokens and reduces model confusion.

---

## 5. Requirement Analyzer

The engine should translate vague software requests into explicit requirements.

User:

"Make me a website where people can upload images and turn them into prompts."

The analyzer might derive:

```
{  
  "functional_requirements": [  
    "image upload",  
    "image analysis",  
    "prompt generation",  
    "result display",  
    "copy prompt"  
  ],  
  "non_functional_requirements": [  
    "responsive UI",  
    "reasonable processing time"  
  ],  
}
```



```
"unknowns": [  
  "authentication",  
  "storage",  
  "maximum file size"  
]  
}
```

But again, **unknown does not mean stop and interrogate the user.**

The engine can build a usable development prompt while marking the unresolved decisions.

---

## 6. Dependency Analyzer

For code, dependencies can materially change the answer.

The engine should identify:

```
{  
  "dependencies": {  
    "known": [  
      "Next.js",  
      "TypeScript",  
      "PostgreSQL"  
    ],  
    "unknown": [  
      "authentication provider"  
    ]  
  }  
}
```

}

If the user says:

"Add authentication."

the system shouldn't arbitrarily decide between five authentication systems unless there's enough context to make that choice.

Instead, it can instruct the coding model:

Use the project's existing authentication system if present. Do not introduce a second authentication provider.

That is a useful constraint.

---

## 7. Risk & Ambiguity Analyzer

Code requires a stronger risk layer.

The analyzer should look for:

security

data loss

authentication

authorization

secrets

payments

database mutations

destructive operations

external APIs

performance

concurrency

privacy

dependency conflicts

breaking changes

Example:

```
{  
  "risks": [  
    {  
      "type": "database_mutation",  
      "severity": "high",  
      "description": "Request modifies production records."  
    }  
  ]  
}
```

The engine should then make the generated coding prompt explicitly require safeguards.

---

## 8. Safe Inference Engine

Code inference must be conservative.

Safe:

"Use the project's existing TypeScript conventions."

Less safe:

"Use PostgreSQL."

if the user never mentioned PostgreSQL.

Even worse:

"Deploy to AWS."

because the model decided that's what a serious application should use.

So:

```
{  
  "safe_inferences": [],  
  "recommended_defaults": [],  
  "requires_user_decision": [],  
  "must_not_assume": []  
}
```

The engine can recommend defaults, but should distinguish them from requirements.

---

## 9. Code Prompt Composer

The composer should produce instructions with a structure roughly like:

ROLE

TASK

PROJECT CONTEXT

EXISTING IMPLEMENTATION

REQUIREMENTS

CONSTRAINTS

TECHNICAL ENVIRONMENT

EXPECTED BEHAVIOR

EDGE CASES

SECURITY REQUIREMENTS

TESTING REQUIREMENTS

OUTPUT REQUIREMENTS

For a coding task, "write the code" is insufficient.

The model should know:

- what to change

- what not to change

- where to change it

- what behavior is expected

- how to test it

- what constraints must remain intact

---

## 10. Independent Code Evaluator

This is the most important component.

The evaluator should not simply judge whether the coding prompt is detailed.

It should determine whether the prompt gives a coding model enough information to **produce correct, compatible, testable code**.

### Evaluation dimensions

INTENT PRESERVATION

TASK COMPLETENESS

TECHNICAL CONTEXT

ARCHITECTURAL COMPATIBILITY

REQUIREMENT CLARITY

DEPENDENCY AWARENESS

EDGE-CASE COVERAGE

SECURITY

ERROR HANDLING

TESTABILITY

MAINTAINABILITY

PERFORMANCE CONSIDERATIONS

UNSUPPORTED ASSUMPTIONS

Example:

```
{  
  "intent_preservation": 97,  
  "task_completeness": 89,  
  "technical_context": 94,  
  "architectural_compatibility": 91,  
  "requirement_clarity": 93,  
  "dependency_awareness": 87,  
  "edge_case_coverage": 78,  
  "security": 92,  
  "error_handling": 81,  
  "testability": 88,  
  "maintainability": 90,  
  "unsupported_assumptions": 95  
}
```

---

# 11. The Code Evaluator should perform static reasoning

If code is supplied, the evaluator should inspect it.

For example:

User request



Existing code



Generated prompt



Evaluator

It should check:

- Does the prompt ask for something incompatible with the existing architecture?

- Does it reference functions that don't exist?

- Does it require a dependency that isn't installed?

- Does it accidentally ask the coding model to rewrite unrelated code?

- Are acceptance criteria defined?

This is where deterministic tooling can supplement the LLM.

For example:

TypeScript compiler

ESLint

dependency manifest

AST parser

test runner

when the relevant project context is available.

The LLM reasons about the task.

Developer tooling verifies concrete facts.

That's a much better division of labor.

---

## 12. Acceptance Criteria

The Code Engine should strongly encourage acceptance criteria.

Weak:

"Build a login system."

Better:

The implementation is successful when:

1. A user can register.
2. Invalid credentials are rejected.
3. Authenticated users can access protected routes.
4. Unauthenticated users are redirected.
5. Passwords are never stored in plaintext.
6. Authentication failures are handled without exposing sensitive information.
7. Tests cover successful and failed authentication.

The engine can generate these from the user's intent where they are obvious, while clearly marking assumptions.

---



## 13. Code Evaluator should distinguish "missing" from "unnecessary"

This prevents bloated prompts.

A tiny function doesn't need:

deployment architecture, database strategy, CI/CD, monitoring, distributed caching...

The evaluator should ask:

**What information materially affects the requested implementation?**

That keeps the resulting coding prompt proportional to the task.

---

## 14. Revision Controller

Composer



Evaluator



Pass?



YES NO



Done Revise

Revision categories:

missing requirement

technical contradiction

architecture conflict

security weakness

unclear acceptance criteria

unsupported assumption

unnecessary scope

missing edge case

For example:

The request says "fix the login bug," but doesn't identify the expected behavior.

The engine shouldn't invent the bug.

It can produce the strongest prompt possible and mark:

```
{  
  "remaining_gap": "actual error behavior"  
}
```

---

## 15. Final output contract

```
{  
  "modality": "code",  
  "operation": "debug",  
  "input_state": "code_snippet",  
  
  "quality": {  
    "score": 91,
```

"confidence": 0.94

},

"enhanced\_prompt": "...",

"technical\_context": {

"language": "TypeScript",

"framework": "Next.js"

},

"improvements": [

"Defined expected behavior",

"Added acceptance criteria",

"Specified error-handling expectations"

],

"remaining\_gaps": [

"Exact reproduction steps"

],

"risks": [

"Potential database mutation"

],

```
"assumptions": []  
}
```

---

## 16. Technical implementation

```
async function processCodeInput(  
  input: CodeInput  
) {  
  const intent =  
    await classifyCodeIntent(input);  
  
  const context =  
    await extractTechnicalContext(  
      input,  
      intent  
    );  
  
  const requirements =  
    analyzeCodeRequirements(  
      context,  
      intent  
    );  
  
  const risks =
```

```
analyzeCodeRisks(  
  context,  
  requirements  
);
```

```
const inferences =  
  await inferSafeTechnicalDetails(  
    context,  
    requirements  
  );
```

```
const draft =  
  await composeCodePrompt({  
    input,  
    intent,  
    context,  
    requirements,  
    risks,  
    inferences  
  });
```

```
const evaluation =  
  await evaluateCodePrompt({  
    input,
```

```
context,  
requirements,  
risks,  
draft  
});
```

```
return reviseIfNecessary(  
    draft,  
    evaluation  
);  
}
```

When actual repository/code access exists, add deterministic verification:

Generated specification



Repository context



Static checks



LLM evaluator



Final assessment

Don't make the LLM the only source of truth when a compiler, parser, dependency manifest, or test runner can answer the question directly.

---

# 17. The Code Engine in one line

UNDERSTAND TASK → EXTRACT TECHNICAL CONTEXT → ANALYZE  
REQUIREMENTS/RISKS → SAFELY INFER → COMPOSE → VERIFY → EVALUATE →  
REVISE → RETURN

The defining question is:

**"Does this instruction contain enough technically accurate context and acceptance criteria for another coding system to implement the requested change without guessing or damaging the existing system?"**

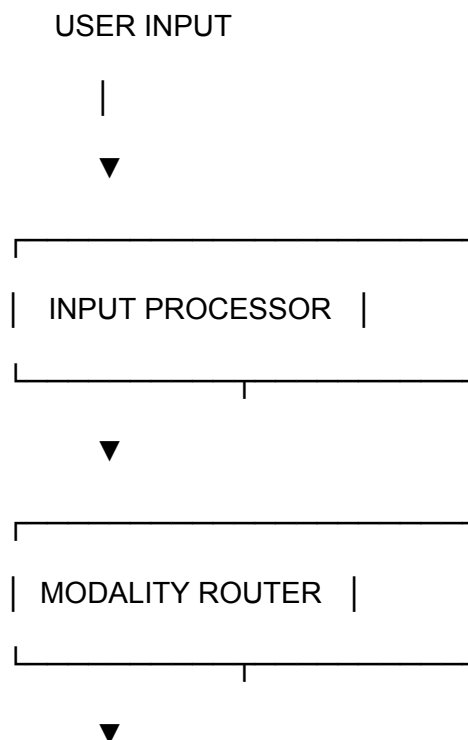
That completes the **five major modality engines**: Image, Video, Text, Conversational, and Code.

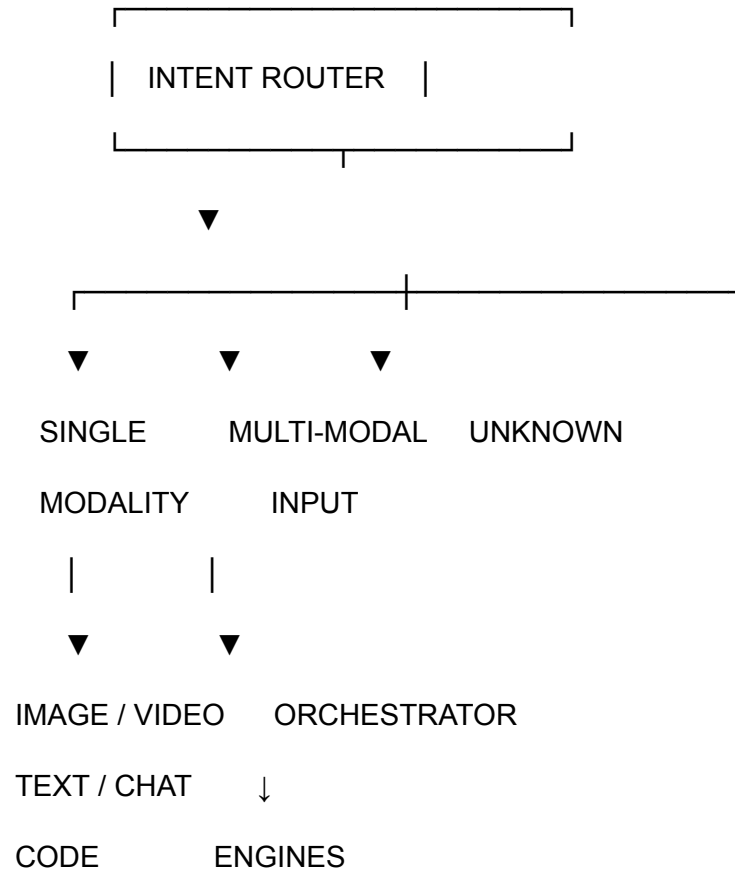
## UNIVERSAL INPUT ROUTER

This is the layer that sits **above the five engines**.

Its job is not to improve the user's prompt.

Its job is to correctly understand **what came in and where it needs to go**.





---

## 1. The Router should NOT be one giant LLM

Use a **hybrid router**.

First, ordinary code identifies obvious things:

file MIME type

file extension

URL

image attachment

video attachment

code block

plain text



Then an LLM handles ambiguity.

For example:

User:

"Make this cinematic."

Attachment:

image/jpeg

Your application already knows:

```
{  
  "attachment_type": "image"  
}
```

The LLM only needs to determine:

```
{  
  "operation": "transform",  
  "intent": "cinematic_visual"  
}
```

Much cheaper and faster.

---

## 2. Input Processor

Normalize everything into one object.

```
interface UniversalInput {  
  text?: string;
```

```
attachments: Attachment[];
```

```
urls: string[];
```

```
metadata: {  
  source: "landing_page" | "workspace";  
  session_id: string;  
  user_id?: string;  
};  
}
```

Attachment:

```
interface Attachment {  
  id: string;  
  mime_type: string;  
  filename?: string;  
  size_bytes?: number;  
  reference: string;  
}
```

So the rest of the application doesn't care whether the user pasted something, uploaded it, or dragged it into the interface.

---

### 3. First classification

The router produces:

```
{
```

```
"modalities": [  
  {  
    "type": "image",  
    "confidence": 0.99  
  }  
],
```

```
"operation": {  
  "type": "transform",  
  "confidence": 0.94  
}  
}
```

Possible modalities:

image

video

text

conversational

code

Possible operation:

generate

improve

audit

transform

edit

analyze

debug

rewrite

summarize

design

unknown

Don't make these enums infinitely large.

The engines can handle the finer distinctions.

---

## 4. The Router needs an ambiguity threshold

This is important.

Suppose:

"Make this better."

No attachment.

The system doesn't know what "this" refers to.

It shouldn't confidently route it.

```
{  
  "route": "unknown",  
  "confidence": 0.31,  
  "reason": "Insufficient input"  
}
```

But the response shouldn't be a dead end.

The UI can simply say:

**What would you like to improve?**

with the input area still ready.

That's one of the rare cases where asking is justified.

---

## 5. Mixed-modality routing

This is where the architecture gets interesting.

User:

"Turn this into a cinematic video."

- image.

Router:

```
{  
  "modalities": [  
    "image",  
    "text"  
  ],  
  "primary_engine": "video",  
  "supporting_engine": "image",  
  "operation": "image_to_video"  
}
```

The image engine doesn't necessarily produce the final answer.

It extracts visual information.

The Video Engine uses that information.

IMAGE



Image Understanding



Visual Specification



VIDEO ENGINE



Video Prompt

So the router needs to understand **primary versus supporting modality**.

---

## 6. Another example

User uploads:

screenshot of website

and writes:

"Make this landing page convert better."

Router:

```
{  
  "primary_modality": "image",  
  "operation": "audit",  
  "target": "landing_page",  
  "supporting_context": "text"  
}
```

But here's a design decision:

**Don't send it to the Image Engine just because the input is an image.**

The *content* represented by the image is a landing page.

This is why modality and task are separate.

INPUT FORMAT  $\neq$  ACTUAL TASK

The router has to determine both.

---

## 7. The routing object

I'd standardize this across your backend:

```
interface RouteDecision {  
  
    primaryModality:  
        | "image"  
        | "video"  
        | "text"  
        | "conversational"  
        | "code";  
  
    supportingModalities: string[];  
  
    operation: string;  
  
    inputState:  
        | "raw_idea"
```

```
| "partial_prompt"
| "developed_prompt"
| "source_material"
| "reference_media"
| "existing_code"
| "unknown";
```

```
confidence: number;
```

```
engine: string;
```

```
needsClarification: boolean;
```

```
reason?: string;
```

```
}
```

---

## 8. Routing logic

The first pass can be ordinary code.

```
function detectBasicType(input: UniversalInput) {
  if (input.attachments.some(a =>
    a.mime_type.startsWith("image/")
  )) {
    return "image";
```



```
}
```

```
if (input.attachments.some(a =>  
  a.mime_type.startsWith("video/")  
)) {  
  return "video";  
}
```

```
if (containsCodeBlock(input.text)) {  
  return "code";  
}
```

```
return "text";  
}
```

Then the LLM handles semantic routing:

```
const route = await classifyIntent({  
  input,  
  detectedTypes  
});
```

The LLM returns strict JSON.

No free-form response.

---

## 9. Confidence determines behavior

I'd use something like:

0.90 – 1.00

Route immediately.

0.70 – 0.89

Route, but allow engine to correct classification.

0.40 – 0.69

Run secondary classification.

Below 0.40

Ask the user for clarification.

These thresholds should eventually be learned from routing errors.

Don't treat them as sacred numbers.

---

## 10. The router should be allowed to correct itself

Suppose the router sends something to Text.

The Text Engine realizes:

"This is actually a coding request."

It shouldn't blindly continue.

Each engine should return:

{

```
"route_valid": false,  
  
"suggested_route": {  
  
  "modality": "code",  
  
  "operation": "debug"  
  
}  
  
}
```

Then the orchestrator reroutes.

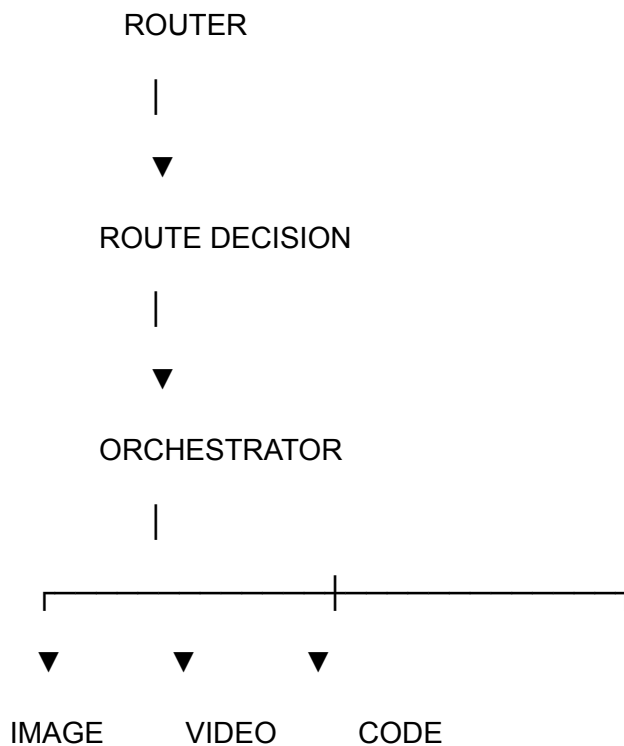
This protects against bad first classification.

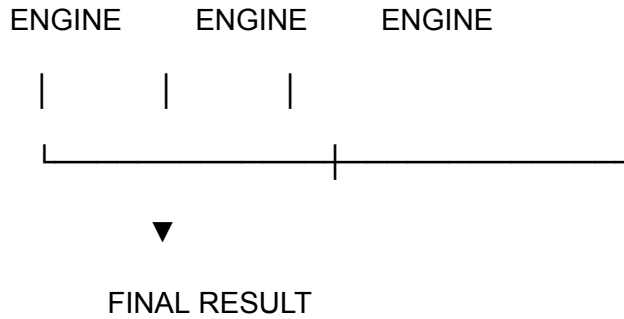
---

## 11. The Universal Orchestrator

The router itself shouldn't execute the whole workflow.

That's the orchestrator's job.





The orchestrator manages:

- execution
- dependencies
- parallel calls
- retries
- timeouts
- evaluation
- revision
- final response

The router simply says:

**"Send this there."**

---

## 12. Latency

This router should be extremely cheap.

Don't make the user wait 5 seconds just to decide which engine to use.

A sensible target:

File/type detection: milliseconds

↓

Lightweight semantic routing: ~sub-second to a few seconds

↓

Engine processing: majority of latency

And if the input type is obvious from the attachment, **skip the LLM router entirely where possible**.

For example:

image/jpeg + "make it cinematic"

Already strongly indicates:

Image → Transform

No reason to spend a model call proving it.

---

## 13. What the router must NEVER do

It should not:

**Rewrite the prompt.**

**Score the prompt.**

**Ask unnecessary questions.**

**Generate the final answer.**

**Apply modality-specific rules.**

Those belong downstream.

The router is traffic control.

---

## 14. One very important addition

I would give every request a **task identity**.

```
{
```

```
  "request_id": "req_8f92...",
```

```
  "parent_request_id": null,
```

```
"primary_modality": "image",  
"operation": "generate",  
"engine": "image_engine_v1"  
}
```

When a user iterates:

"Make the lighting warmer."

That's not a completely new request.

It becomes:

```
{  
  "request_id": "req_91ab...",  
  "parent_request_id": "req_8f92...",  
  "operation": "modify"  
}
```

Now your application can maintain the chain:

Original idea

↓

Enhanced prompt

↓

User modification

↓

Version 2

↓

User modification

↓

Version 3

That will become very useful once we get into the actual workspace.

---

## Universal Router in one line

DETECT INPUT → UNDERSTAND INTENT → IDENTIFY PRIMARY MODALITY → IDENTIFY OPERATION → ROUTE → ORCHESTRATE

And the most important architectural principle is:

**The router identifies the job. The modality engine knows how to do the job.**

That keeps the system from becoming one giant LLM prompt with a pile of conditional instructions attached to it.

## UNIVERSAL EVALUATION LAYER

This is the quality-control system sitting **after the modality engine and before the user receives the result.**

Its job is simple:

**Determine whether the transformation actually improved the user's input without changing what they meant.**

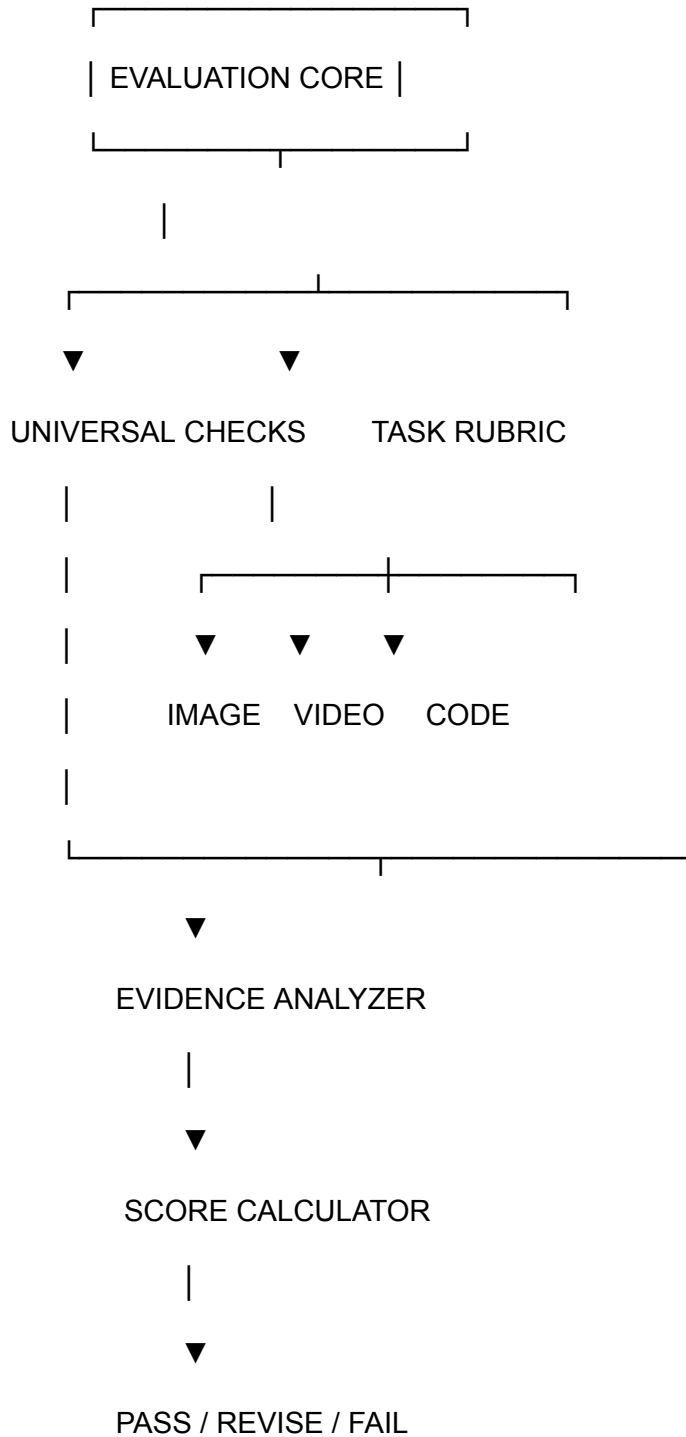
It should not be one universal LLM rubric. The architecture should be universal, while the actual evaluation criteria come from the modality/task.

---

### 1. Architecture

ENGINE OUTPUT





The **Evaluation Core** is shared.

The **Rubric** changes.

---



## 2. What the evaluator receives

Never give the evaluator only the generated prompt.

It needs the complete transformation context:

```
{  
  "original_input": "...",  
  
  "route": {  
    "modality": "image",  
    "operation": "generate"  
  },  
  
  "extracted_specification": {},  
  
  "generated_output": {},  
  
  "inferences": {},  
  
  "constraints": {},  
  
  "rubric": {}  
}
```

Otherwise the evaluator can't determine whether the output actually preserved the user's intent.

---

### 3. Universal checks

Every modality gets these baseline checks.

#### **Intent Preservation**

Did we retain what the user actually asked for?

#### **Unsupported Addition**

Did the system invent important information?

#### **Internal Consistency**

Does the resulting instruction contradict itself?

#### **Completeness**

Does it contain enough information to accomplish the intended task?

#### **Relevance**

Did we add useful information, or just make the prompt longer?

#### **Clarity**

Can another capable model understand what is being requested?

#### **Constraint Preservation**

Did we preserve explicit restrictions?

#### **Efficiency**

Is there unnecessary wording or instruction?

These become the **universal evaluator layer**.

---

## 4. Then the task-specific evaluator takes over

For example:

### IMAGE

- visual specificity
- composition
- lighting
- visual coherence

### VIDEO

- temporal coherence
- motion
- continuity
- camera logic

### TEXT

- task requirements
- output structure
- factual safety

### CONVERSATIONAL

- turn behavior
- state management
- role consistency

CODE

→ technical compatibility

→ acceptance criteria

→ security

→ testability

The universal engine doesn't need to know all these details.

It loads the appropriate rubric.

---

## 5. Rubric Registry

I'd make this configuration-driven.

```
interface EvaluationRubric {
```

```
    id: string;
```

```
    version: string;
```

```
    dimensions: EvaluationDimension[];
```

```
    weights: Record<string, number>;
```

```
    passThreshold: number;
```

```
    revisionThreshold: number;
```

```
}
```

Then:

```
const rubric =  
  rubricRegistry.get(  
    modality,  
    taskType  
  );
```

So you can have:

image.generate.v1

image.edit.v1

image.transform.v1

video.generate.v1

video.image\_to\_video.v1

text.research.v1

text.copywriting.v1

conversation.agent.v1

code.generate.v1

code.debug.v1

code.refactor.v1

This is much better than hard-coding evaluation logic throughout your backend.

---

## 6. The evaluator should produce evidence

Don't allow:

```
{  
  "score": 87  
}
```

That's useless.

Require:

```
{  
  "dimension": "intent_preservation",  
  "score": 94,  
  "evidence": [  
    "User requested a Ghanaian king.",  
    "Generated prompt retains the Ghanaian identity."  
  ],  
  "severity": "none"  
}
```

Now you can inspect why the evaluator reached its conclusion.

---

## 7. Score calculation

The LLM provides **dimension scores and evidence**.

Your application calculates the final score.

For example:

```
function calculateScore(
```

```
dimensions: EvaluationResult[],  
weights: Record<string, number>  
) {  
  return dimensions.reduce(  
    (total, item) =>  
      total +  
      item.score *  
      weights[item.dimension],  
    0  
  );  
}
```

The model does not decide:

"Overall, I'll give this 91."

Your application does.

That makes the scoring system reproducible.

---

## 8. Don't let a high average hide a catastrophic failure

This is important.

Imagine:

Intent preservation    95

Visual quality        94

Style                 91

Composition 89

Unsupported assumptions 25

An average might still look decent.

But that prompt has a serious problem.

So introduce **hard gates**.

```
{  
  "hard_gates": {  
    "intent_preservation": 70,  
    "unsupported_assumptions": 70,  
    "internal_consistency": 75  
  }  
}
```

If a hard gate fails:

DO NOT PASS

regardless of the overall score.

---

## 9. Three evaluation outcomes

Keep the state machine simple.

PASS

↓

Return output

REVISE



↓

Send targeted issues back to composer

FAIL

↓

Do not expose the generated result as trustworthy

Example:

```
{  
  "status": "revise",  
  "score": 74,  
  "failed_dimensions": [  
    "intent_preservation",  
    "technical_completeness"  
  ]  
}
```

---

## 10. Revision should receive only the problems

Don't send the entire evaluation essay back to the composer.

Send a structured correction packet:

```
{  
  "revision_required": true,
```

```
"issues": [  
  {  
    "dimension": "intent_preservation",  
    "severity": "high",  
    "instruction": "Remove the invented historical period."  
  },  
  {  
    "dimension": "composition",  
    "severity": "medium",  
    "instruction": "Clarify subject framing."  
  }  
]  
}
```

This reduces tokens and prevents the revision model from getting distracted.

---

## 11. Independent evaluator means genuinely independent

For serious evaluation, don't always use the exact same model/configuration that generated the prompt.

Ideally:

Generator

↓

Model A

Evaluator



Model B

At minimum, use different instructions and a separate evaluation context.

Later, you can benchmark:

Generator: strong model

Evaluator: cheaper model

Generator: strong model

Evaluator: another strong model

Generator: model X

Evaluator: model Y

This lets you measure evaluator reliability.

---

## 12. The evaluator itself needs evaluation

This is the part people usually forget.

You need a **golden evaluation dataset**.

For example:

```
{  
  "input": "...",  
  "expected_properties": [  
    "preserve subject",
```

```
"don't invent historical period",  
"include camera movement"  
]  
}
```

Then test whether your evaluator correctly identifies:

true improvement

false improvement

intent drift

hallucination

over-specification

under-specification

Your evaluator is software too.

It needs testing.

---

## 13. Human feedback becomes the long-term signal

Eventually you collect:

Original input



Generated result



Evaluator score



User edits



User accepts / rejects



Final result

Suppose the evaluator gives:

**92**

but the user completely rewrites the output.

That's valuable.

The evaluator may be systematically overrating a certain type of prompt.

Likewise:

**78**

and the user immediately copies it without changing anything.

The evaluator may be too harsh.

These become calibration data.

---

## 14. Don't use user behavior as the only quality signal

Copying a prompt doesn't necessarily mean it was good.

Likewise, editing doesn't necessarily mean it was bad.

So eventually your feedback model can combine:

Evaluator score

+

Human review

+

User acceptance

+

User edits

+

Downstream result quality

For your application, **downstream output quality** is eventually the strongest signal.

If the enhanced image prompt consistently produces better images, that's much more meaningful than someone clicking "Copy."

---

## 15. Latency control

The evaluator shouldn't automatically run a deep evaluation every time.

Use tiers.

### **Fast evaluation**

Simple request.

2-3 universal checks

+

small task rubric

### **Standard evaluation**

Normal request.

full rubric

+

hard gates

## Deep evaluation

Complex request.

full rubric

+

simulation/testing

+

revision

The router/engine can determine the evaluation tier.

That protects your 10 to 20 second target.

---

## 16. Final evaluation object

The backend should produce something like:

```
{  
  "evaluation": {  
    "status": "pass",  
  
    "score": 91,  
  
    "confidence": 0.93,  
  
    "dimensions": {  
      "intent_preservation": 96,
```

```
"completeness": 89,  
"clarity": 94,  
"relevance": 91  
},  
  
"hard_gates": {  
  "passed": true  
},  
  
"issues": [],  
  
"revision_count": 0,  
  
"rubric": {  
  "id": "image.generate.v1",  
  "version": "1.0"  
}  
}  
}
```

---

## 17. What the frontend gets

The backend can know all of this:



96

89

94

93

hard gate

confidence

revision count

12 evaluator observations

The visitor doesn't need to see any of it.

The frontend might simply receive:

91 / 100

Your idea was preserved.

We added stronger visual direction,  
composition, and lighting.

2 optional details could make it  
even more specific.

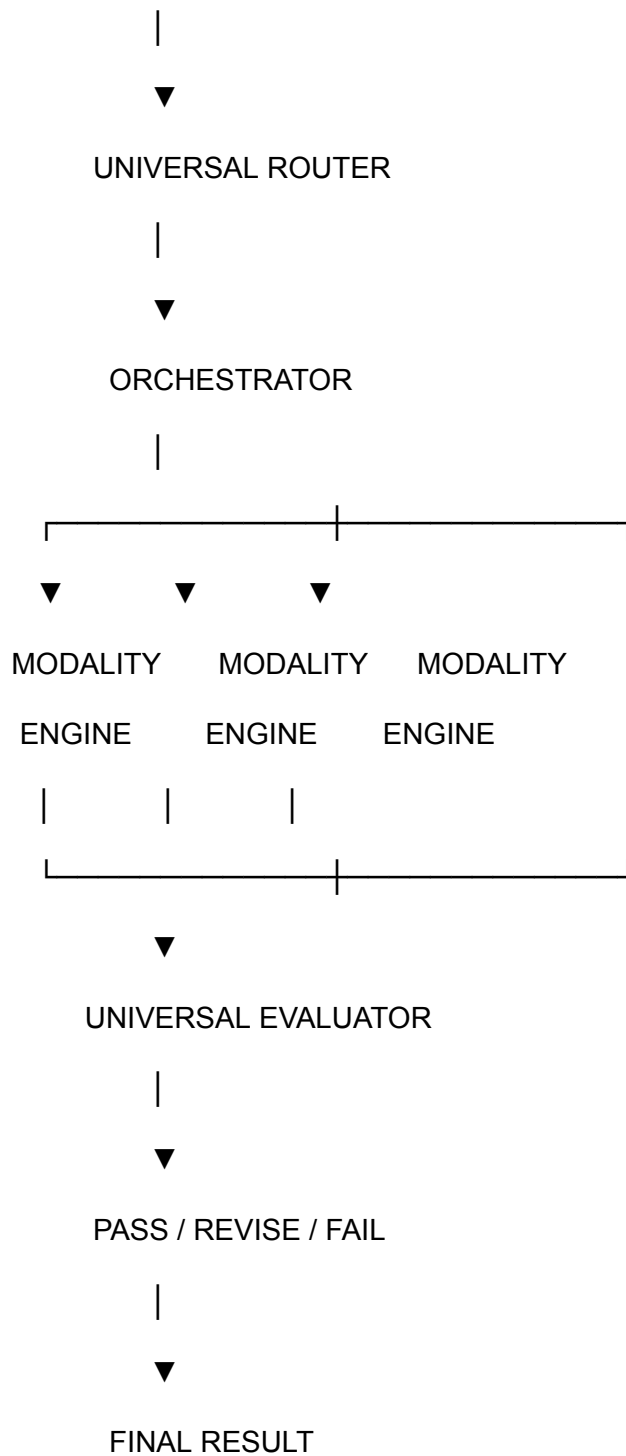
That keeps the **engine sophisticated and the interface quiet**.

---

## 18. The architecture we're building now

We have:

USER



The important separation is now clear:

**Router:** What is this?

**Engine:** How should this be transformed?

**Evaluator:** Did we transform it correctly?

**Orchestrator:** What happens next?

That gives us a proper technical backbone rather than one enormous prompt pretending to be an application.

The next major layer should be the **Output/Transformation Contract**, where we define exactly what the user receives from the system and how that result moves from the backend into the first-touch experience.

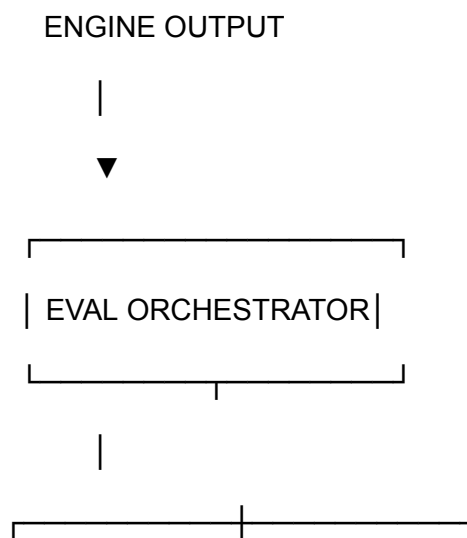
Yes. Now we're at the point where I would **replace our earlier evaluation design with an evidence-backed one.**

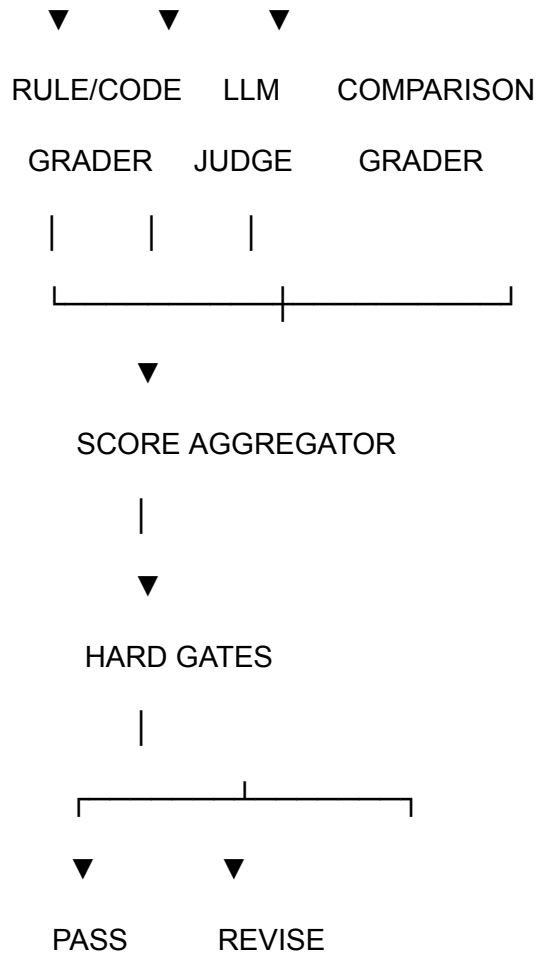
There is no credible universal industry formula saying, for example, "prompt quality = 30% clarity + 20% specificity + 10% tone." Treating such a formula as established fact would be made-up precision.

What the current evaluation literature and platform guidance support is a **task-specific, multidimensional evaluation system**, using the most objective grader available for each criterion. Anthropic explicitly recommends measurable, task-specific criteria and a mixture of code-based, LLM-based, and human grading. ([Claude Platform](#)) OpenAI's grader framework similarly supports code graders, string/similarity checks, model-based graders, and combinations of graders. ([Claude Platform](#))

So this is the evaluation architecture I would now use.

## Evidence-backed Evaluation Layer





# 1. Three types of graders

## A. Deterministic grader

Use ordinary code whenever the criterion has an objective answer.

Examples:

Does the output contain the requested format?

Does it preserve required terms?

Does valid JSON conform to schema?

Does generated code compile?

Does a required function exist?

Is a URL present?

Was a forbidden element introduced?

This is the **highest-confidence grader** because there is no LLM opinion involved.

Current evaluation guidance explicitly recommends code-based grading where possible because it is fast, reliable, and scalable. ([Claude Platform](#))

---

## B. LLM judge

Use an LLM when the criterion requires judgment.

Examples:

Did the output preserve the user's intent?

Is the visual description coherent?

Is the conversational behavior appropriate?

Is the coding instruction sufficiently complete?

Is the transformation actually an improvement?

But the judge gets a **specific rubric**, not:

"Give this prompt a score out of 100."

That would be weak.

---

## C. Comparative grader

This one is particularly important for **your product**.

Your system is fundamentally a transformation system.

Therefore, we should frequently evaluate:

ORIGINAL

VS

ENHANCED

rather than evaluating the enhanced prompt in isolation.

The judge answers:

**Which version better satisfies the user's stated objective?**

and:

**Did the enhanced version introduce any regression?**

Pairwise judging has substantial research support for evaluating open-ended model outputs, although it has known biases that need to be controlled. The MT-Bench/Chatbot Arena research found strong LLM judges can correlate well with human preferences, while also identifying issues such as position and verbosity bias. ([arXiv](#))

So we'd randomize presentation:

Trial A:

Original → A

Enhanced → B

Trial B:

Enhanced → A

Original → B

That helps reduce positional bias.

---

## 2. The evaluator should not produce one score first

This is the big change.

Instead:

Criterion → Evidence → Verdict

For example:

{

```
"criterion": "intent_preservation",  
"verdict": "PASS",  
"evidence": [  
    "The requested subject remains unchanged.",  
    "The requested action remains unchanged."  
]  
}
```

Then another:

```
{  
    "criterion": "unsupported_inference",  
    "verdict": "FAIL",  
    "evidence": [  
        "The enhanced prompt introduced an unspecified historical period."  
    ]  
}
```

Then your application calculates the score.

The LLM isn't allowed to invent the final number.

---

### 3. Rubrics become the real evaluation engine

Each task gets a rubric.

For example:

image.generate.v1

video.generate.v1

text.research.v1

conversation.agent.v1

code.debug.v1

Each rubric contains:

```
{  
  "criterion": "...",  
  "description": "...",  
  "pass_condition": "...",  
  "fail_condition": "...",  
  "weight": 0.15,  
  "severity": "high"  
}
```

This follows current evaluation guidance that rubrics should be specific and measurable rather than vague qualitative judgments. ([Claude Platform](#))

---

## 4. Hard gates

Some failures shouldn't be averaged away.

For example:

Intent preservation = FAIL

Unsupported critical assumption = FAIL

Security violation = FAIL

Invalid code = FAIL

Required output missing = FAIL



Even if everything else looks excellent:

Score = 94

Hard gate = FAILED

Result:

**REVISE**

not PASS.

---

## 5. Score only after the criteria are judged

Something like:

```
function calculateScore(results, rubric) {  
  let score = 0;  
  
  for (const criterion of rubric.criteria) {  
    const result = results[criterion.id];  
  
    score +=  
      result.normalizedScore *  
      criterion.weight;  
  }  
  
  return score;  
}
```

But the final number is secondary.

The more useful internal result is:

```
{  
  "score": 87,  
  "hard_gates_passed": true,  
  "confidence": 0.91,  
  "revision_required": false  
}
```

---

## 6. Human evaluation is not part of every user request

This is important for your latency target.

Humans should **calibrate the evaluator during development**, not sit in the production request path.

We create a benchmark set.

Then experts judge:

Original

Enhanced

against the same rubric.

Then compare:

Human judgment

VS

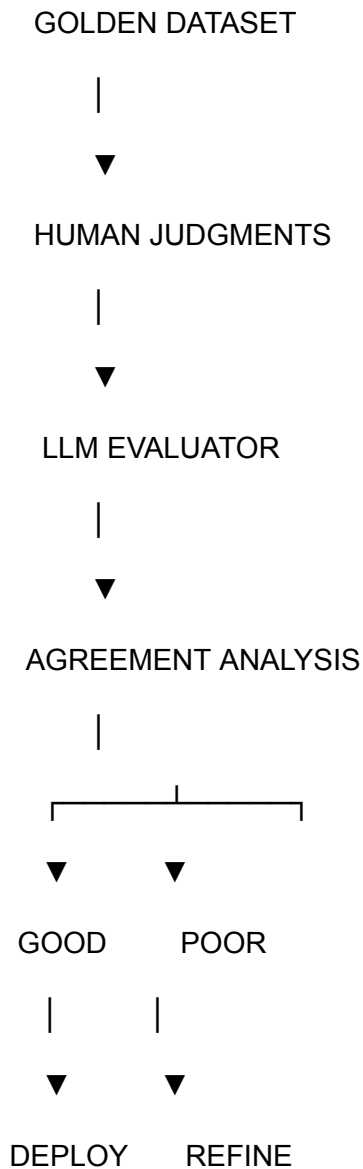
LLM judge

If the LLM judge disagrees too often, we fix the rubric, judge prompt, model, or weighting.

Recent research specifically recommends reporting agreement between LLM judges and human annotations rather than assuming the judge is correct. ([arXiv](#))

---

## 7. This gives us a calibration loop



That is how I would make the evaluator trustworthy.

Not by declaring:

"Our LLM thinks it's 92/100."

---

## 8. And this gives us a better definition of "quality"

For our application:

**Quality = the degree to which the transformed prompt improves expected task performance while preserving user intent and avoiding unsupported additions.**

That's the core.

Not word count.

Not prompt length.

Not how sophisticated the language sounds.

Not how many instructions the model added.

---

## 9. One final thing I would add

We should eventually measure **outcome quality**, not merely prompt quality.

For example:

User idea

↓

Our enhanced prompt

↓

Target AI

↓

Generated result

↓

Evaluate result

Because the strongest evidence that our enhancement worked is:

**Did the enhanced prompt actually produce a better result?**

That becomes the long-term benchmark for the product.

And this is exactly why our evaluation architecture should remain separate from the prompt-generation model. Current agent-evaluation designs also use a separate grading context and explicit outcome rubric to avoid the evaluator being influenced by the generation process. ([Claude Platform](#))

---

**So I would officially revise our architecture to this:**

ROUTER

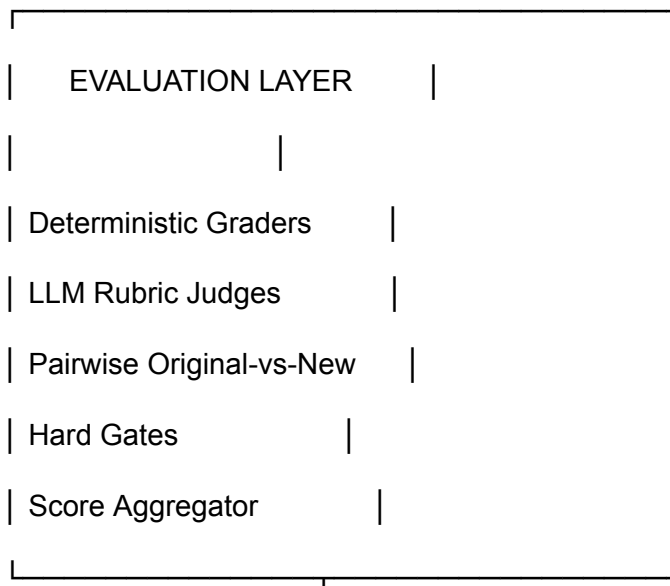
↓

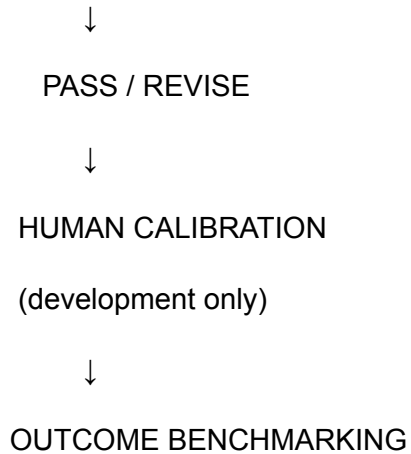
MODALITY ENGINE

↓

GENERATION

↓





**That is the version I would build.**

And I would treat the rubric definitions themselves as a versioned asset in your codebase, not buried inside random prompts.

For example:

/evals

/image

generate.v1.json

edit.v1.json

/video

generate.v1.json

/text

research.v1.json

/conversation

agent.v1.json

/code

debug.v1.json

That gives you an evaluation system you can actually test, version, compare, and improve instead of a black-box "AI score."

# Transformation Contract

This is the contract between the **AI backend and the frontend**.

The important decision here is that the frontend should **not know how the Image, Video, Text, Conversational, or Code engine works**. It receives a predictable result object and renders the appropriate experience.

For API design, there isn't one mandatory response-envelope standard. The important production principle is consistency, schema validation, and a stable contract. GOV.UK recommends JSON for API responses and points to JSON:API when an organization hasn't already chosen a format. ([GOV.UK](#))

For our application, I would use a small envelope because our transformation response contains both the actual result and processing metadata.

---

## 1. The canonical response

```
{  
  "data": {  
    "requestId": "req_01...",  
    "modality": "image",  
    "operation": "generate",  
  
    "input": {  
      "type": "raw_idea",  
      "original": "A Ghanaian king standing in a palace."  
    },  
  
    "result": {  
      "type": "enhanced_prompt",
```

"content": "..."

},

"quality": {

"score": 91,

"confidence": 0.93,

"status": "passed"

},

"changes": [

{

"type": "added",

"label": "Composition",

"description": "Added clear subject framing."

}

],

"remainingGaps": [

"Historical period",

"Preferred visual style"

],

"assumptions": [

"Used cinematic lighting as a neutral visual treatment."



```
],  
  
  "version": 1  
},  
  
  "meta": {  
    "engineVersion": "image.v1",  
    "evaluationVersion": "image.generate.v1"  
  }  
}
```

This is the **canonical backend contract**.

Not the UI.

The UI decides which pieces are worth showing.

---

## 2. Separate "result" from "diagnostic"

This is important.

The user came for the result.

So:

```
{  
  "result": {  
    "content": "..."  
  }  
}
```

is the primary object.

Everything else supports it.

The score, evaluator information, changes, assumptions, and gaps are **diagnostic metadata**.

That means your frontend can immediately render:

YOUR ENHANCED PROMPT

without waiting for some giant diagnostic object to be constructed.

---

### 3. Don't expose the entire evaluator

Internally we may have:

```
{  
  "intentPreservation": 96,  
  "visualSpecificity": 88,  
  "unsupportedInference": 94,  
  "composition": 83,  
  "lighting": 91  
}
```

The first-touch frontend probably only needs:

```
{  
  "score": 91,  
  "status": "passed"  
}
```

and perhaps:

```
{
```

```
"changes": [...]  
}
```

The evaluator's evidence remains available to the workspace, debugging tools, analytics, and internal QA.

This keeps the public result clean.

---

## 4. The contract needs a result type

Because the five engines won't always return the same thing.

For Image:

```
{  
  "type": "enhanced_prompt",  
  "content": "..."  
}
```

Video might return:

```
{  
  "type": "video_prompt",  
  "content": "..."  
}
```

or:

```
{  
  "type": "shot_plan",  
  "content": {}  
}
```

Code might eventually return:

```
{  
  "type": "coding_prompt",  
  "content": "..."  
}
```

Conversational:

```
{  
  "type": "system_prompt",  
  "content": "..."  
}
```

This gives the frontend a stable way to determine what it received.

---

## 5. Input should also be preserved

Never throw away the original.

```
{  
  "input": {  
    "type": "raw_idea",  
    "original": "..."  
  }  
}
```

This allows the frontend to show:

YOUR IDEA

↓

ENHANCED PROMPT

It also gives you version history later.

---

## 6. Changes should be structured

Don't return:

"We made your prompt better."

That's useless.

Use:

```
{  
  "changes": [  
    {  
      "type": "added",  
      "label": "Composition",  
      "description": "Specified a three-quarter portrait composition."  
    },  
    {  
      "type": "added",  
      "label": "Lighting",  
      "description": "Added directional cinematic lighting."  
    },  
    {  
      "type": "clarified",
```

```
"label": "Subject",  
  "description": "Clarified the subject's visual presentation."  
}  
]  
}
```

The frontend can then decide to show:

2 things improved

and expand it only if the user wants to know.

No giant explanation by default.

---

## 7. Remaining gaps are different from errors

This distinction matters.

Suppose the user says:

"A cinematic portrait of a king."

The system successfully generates a good prompt.

It might return:

```
{  
  "remainingGaps": [  
    "Specific historical period",  
    "Preferred visual style"  
  ]  
}
```

That does **not** mean the result failed.

It means:

"This works. Here are things that could make it more specific."

So:

quality.status = "passed"

can coexist with:

remainingGaps.length > 0

That's exactly what we want for your first-touch experience.

---

## 8. Assumptions need their own field

This protects the user from silent invention.

```
{  
  "assumptions": [  
    {  
      "field": "lighting",  
      "value": "cinematic directional lighting",  
      "reason": "Added as a neutral visual enhancement."  
    }  
  ]  
}
```

If an assumption is dangerous or materially changes the user's intent, it shouldn't simply appear here.

It should trigger revision.

---

## 9. Versioning

Every transformation should have an identity.

```
{  
  "requestId": "req_...",  
  "version": 1,  
  "parentRequestId": null  
}
```

Then:

Version 1

Original idea

↓

Version 2

User says "make it darker"

↓

Version 3

User says "make the king older"

Technically:

```
{  
  "requestId": "req_003",  
  "parentRequestId": "req_002",  
  "version": 3  
}
```

That gives us a proper prompt lineage.

---



# 10. Streaming and latency

Because we're targeting a quick first result, I would separate:

**required result data**

from

**secondary metadata.**

The frontend can receive the primary transformation as soon as it's ready.

Then secondary information can arrive afterward.

Conceptually:

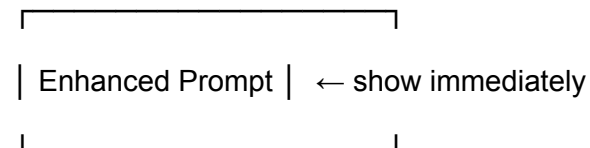
REQUEST



PROCESSING



RESULT READY



evaluation metadata



changes / gaps

For longer operations, use an asynchronous job model rather than pretending a request is complete when it isn't. HTTP **202 Accepted** is the conventional status for work that has been accepted but is still processing. ([Jsonic](#))

---

## 11. Frontend should have a tiny view model

Don't let React components consume the entire backend object directly.

Create:

```
interface TransformationViewModel {  
  
    original: string;  
  
    output: string;  
  
    score?: number;  
  
    status:  
        | "processing"  
        | "passed"  
        | "revision"  
        | "failed";  
  
    changes: Change[];  
    remainingGaps: string[];  
}
```

Then your frontend only cares about what it needs to render.

The backend can evolve without forcing every UI component to change.

---

## 12. The API should be schema-validated

I'd use **JSON Schema + Zod** or **JSON Schema + Ajv** at the API boundary.

For example:

```
const TransformationResponse = z.object({  
  
  data: z.object({  
  
    requestId: z.string(),  
  
    modality: z.enum([  
  
      "image",  
  
      "video",  
  
      "text",  
  
      "conversational",  
  
      "code"  
  
    ]),  
  
    operation: z.string(),  
  
    input: z.object({  
  
      type: z.string(),  
  
      original: z.string()  
  
    }),  
  
    result: z.object({  
  
      type: z.string(),  
  
      content: z.unknown()  
  
    }),  
  
    quality: z.object({  
  
      score: z.number().min(0).max(100),  
  
      confidence: z.number().min(0).max(1),  

```

```
    status: z.enum([
      "passed",
      "revision",
      "failed"
    ])
  })
})
});
```

Schema validation at API boundaries is a standard defensive practice, particularly for preventing unexpected fields and malformed data from moving through the system. ([Jsonic](#))

---

## 13. Errors should be separate

Don't return:

```
{
  "data": null,
  "success": false,
  "message": "Something went wrong"
}
```

with every endpoint inventing its own error structure.

Use a consistent machine-readable error shape, ideally aligned with established HTTP error practices such as RFC 7807 / Problem Details.

For example:

```
{
  "error": {
```

```

    "code": "INPUT_UNSUPPORTED",
    "message": "This input type is not currently supported.",
    "requestId": "req_01..."
  }
}

```

The frontend can then decide what to tell the user.

---

## 14. The most important architectural decision

The contract should be:

UNIVERSAL OUTER SHAPE

+

MODALITY-SPECIFIC RESULT

Not five completely different APIs.

So:

Image

Video

Text

Conversation → TransformationResponse

Code

The outer contract stays stable.

Only:

result.type

result.content

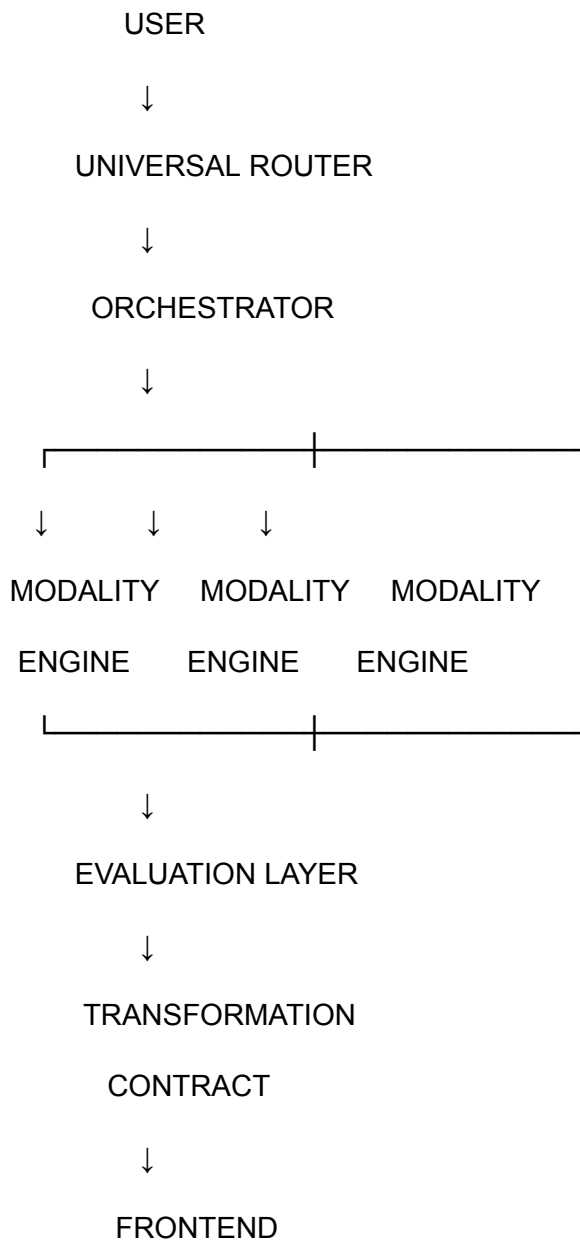
changes according to the modality.

That will save you a lot of frontend pain later.

---

## Where we are now

We now have:



And **this is where I would stop this layer.**

The next major thing is finally the part you've been asking for from the beginning:

**the actual first-touch landing page / CTA interface.**

That's where we'll take this machinery and decide exactly what the visitor sees, what they can input, what happens in the first 10 to 20 seconds, and how the interface behaves on mobile without turning the screen into a cockpit.

Yes. **This is exactly the missing behavioral layer.**

And I would not call it manipulation in the product spec. It is **feedback, anticipation, reward, and momentum**. Every millisecond of waiting is part of the interface. If we leave it blank, the user experiences "the system is slow." If we use it well, the same 3 seconds can feel like "the system is working on something for me."

And you're thinking of **Duolingo**. Its feedback loops are a good reference for this kind of micro-interaction design.

But we should be careful. We don't want confetti sprayed over every button like a casino.

## The principle

The interaction should have a tiny emotional arc:

CLICK



ACKNOWLEDGE



ANTICIPATE



REVEAL



REWARD



NEXT ACTION

That's what we should design.

---

## 1. Immediately after "Improve"

The button should physically respond.

Not just:

loading...

Instead:

[ Improve → ]

↓ click

[ ♦ Improving... ]

The button contracts very slightly, the arrow changes, and a small motion begins around the input.

**0 to 300ms: acknowledgement.**

The user should immediately know:

"It got my request."

No dead screen.

---

## 2. 300ms to ~1.5s: show the system working

This is where we can create anticipation without lying.

The input can subtly transform.



For example:

Your idea

"A cinematic portrait of a king"



Understanding the visual direction



Building the prompt

But don't show a fake 7-step checklist if the backend isn't literally performing those steps.

Instead, have **real pipeline states** mapped to the actual backend:

Understanding

Building

Checking

For Image:

Understanding your visual idea



Adding visual direction



Checking the result

For Code:

Understanding the technical task



Structuring the requirements



Checking compatibility

For Video:

Understanding the scene



Structuring motion



Checking continuity

Now the animation is not decoration. It communicates actual system activity.

---

### 3. But don't make the user stare at a spinner

A spinner communicates:

**Wait.**

That's weak.

We want:

**Something is happening.**

I'd use **progressive visual transformation**.

The original input could blur or compress slightly into a central processing state.

Then pieces of the result begin appearing.

For example:

A cinematic portrait of a king



Enhanced Prompt

Not necessarily literally those words. The idea is that the interface gives the user a visual sense that the rough material is being organized.

---

## 4. Then comes the reveal

This is where I would borrow from the feeling you're describing with Duolingo.

Don't simply replace the old card with the new one.

**Reveal the improvement.**

Something like:

41

↓

91

+50 improvement

Then the enhanced prompt appears.

The number should animate into place rather than instantly appearing.

The prompt card can rise very slightly into position.

The score can count upward quickly:

41 → 54 → 67 → 79 → 91

Not slowly. Perhaps 500 to 800ms.

The point isn't to make the number exciting for its own sake.

It's to make the transformation **perceptible**.

---

## 5. The "aha" moment

This is the moment I would optimize heavily.

The user sees:

### **Your idea**

A king standing in a palace.

Then:

### **Now it can actually be used**

A cinematic portrait of a Ghanaian king standing...

And underneath:

**91/100**

**We added visual direction without changing your idea.**

That sentence is doing psychological work.

It tells the user:

"The machine understood me."

That's more valuable than:

"Your prompt improved by 42 points."

---

## 6. After Copy, you're right. "Copied" is weak.

The user presses:

**Copy Prompt**

Don't merely change it to:

Copied ✓

Give them a tiny reward.

Something like:

✓

Prompt copied

Ready to use

The checkmark can briefly scale from 80% → 110% → 100%.

The button can become:

[ ✓ Copied ]

for roughly 1.5 seconds.

Then return to:

[ Copy Prompt ]

But immediately underneath:

**Want to keep improving it?**

And:

**Improve further** →

That's the next behavioral step.

---

## 7. But there's a stronger idea

After copying, we know something important:

**The user just demonstrated intent.**

They didn't merely click around.

They took the output.

So the interface can acknowledge that:

**Nice. Your prompt is ready to use.**

Then:

**Want to save it and refine it later?**

**Create free workspace** →

This is much better than:

"Sign up now!"

Because the CTA follows the user's behavior.

---

## 8. Don't overdo the reward system

This is where I'd push back slightly on your idea.

If every interaction produces:

- confetti
- bouncing cards
- sound
- fireworks
- streaks
- badges
- animated numbers

the product will quickly feel juvenile.

Your product is supposed to feel like a **serious intelligent tool**.

So I'd borrow the **feedback mechanics** from language-learning apps, not their visual personality.

Think:

**Duolingo's responsiveness.**

Not:

**Duolingo's cartoon world.**

For your product, the animation language should feel more like:

**precision + intelligence + momentum.**

---

## 9. The full micro-interaction

I'd make the first interaction approximately:

USER

|

| clicks Improve



0ms

Button responds

|



300ms

"Understanding..."



~1s

"Building..."



~2s

"Checking..."



RESULT



└─ Score rises

└─ Prompt reveals

└─ Improvements appear



USER CLICKS COPY



✓ Prompt copied



|



"Ready to use."

|



"Save & keep improving?"

The actual timings should remain **backend-driven**, not artificially delayed just to create suspense.

If the result arrives in 600ms, don't force the user to watch a 3-second animation.

That's where "psychology" becomes annoying.

---

## 10. One feature I'd seriously consider

### Transformation Pulse

When the enhanced prompt arrives, briefly highlight the areas that changed.

For example:

A cinematic **portrait** of a Ghanaian king standing in a **grand royal interior**, captured with **dramatic directional lighting**...

The added sections briefly receive a subtle visual treatment.

Then after about a second, everything becomes normal.

This visually answers:

**"What did your system actually do?"**

without making the user open a diagnostic panel.

That's a very good fit for your product.

---

# 11. The deeper principle

The first experience shouldn't merely be:

**Input → Output**

It should feel like:

**Input → Transformation → Recognition → Reward → Continuation**

And the reward doesn't need to be childish.

A beautifully timed reveal, a satisfying checkmark, a score transition, a subtle card movement, and a clear next action can be enough.

That is the layer we were missing.

**Next, I would design the exact animation/state system for the first-touch experience,** including what happens at **0ms, 300ms, 1s, 2s, result arrival, copy, and save**, and how those states differ on mobile versus desktop.

Yes. I agree with that correction.

And I think it makes the product **better**, not merely simpler.

The landing page should have its own recognizable transformation ritual. The user shouldn't have to understand the machinery underneath it.

**So we lock this in:**

**Landing page = modality-agnostic.**

The user gives us:

- an idea
- a prompt
- an image
- code
- a conversation
- whatever the interface accepts

We don't tell them:

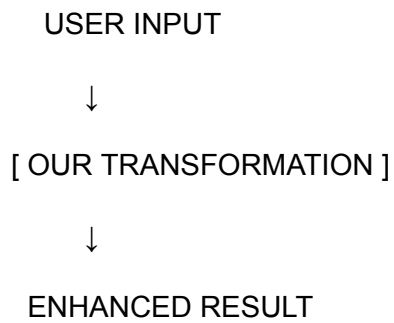
"Detected: Image."

We don't show:

"Running Image Engine."

We don't change the animation because they submitted video instead of text.

Instead:



That transformation becomes **part of the brand experience**.

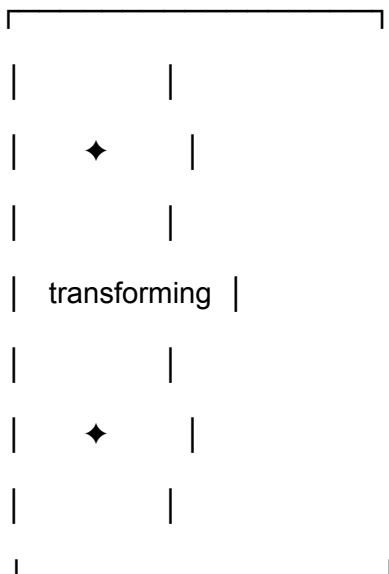
---

## The transformation should become recognizable

Think of it like a signature.

Every time they click **Improve**, they see the same visual language.

Something like:



Perhaps the user's input visually compresses toward a central mark, the mark briefly activates, then the improved result emerges from it.

The exact animation needs design work, but the **behavior stays constant**.

That gives the user a mental association:

**I press this → I see this transformation → my result appears.**

After their second or third visit, the animation itself becomes a familiar signal that the system is working.

That's much better than constantly telling them what model or modality is running.

---

## And the modality belongs to the workspace

This is the bigger architectural correction.

The landing page is:

**Universal transformation entrance.**

The workspace is:

**Specialized environment.**

So:

LANDING PAGE

|

| "A cinematic image of..."



UNIVERSAL TRANSFORMATION

|



RESULT

|

| Continue



WORKSPACE

|



IMAGE MODALITY

The workspace can then expose:

**Image**

**Video**

**Text**

**Conversation**

**Code**

because now the user is intentionally working inside a particular environment.

That is where modality-specific controls, questions, iterative refinement, specialized evaluators, templates, history, versions, and all the other machinery belong.

---

## **This also solves a UX problem**

The landing page shouldn't make the user think:

"Which mode am I supposed to choose?"

Instead:

**"I have this thing. Let's see what they can do with it."**

That's exactly the behavior we want.

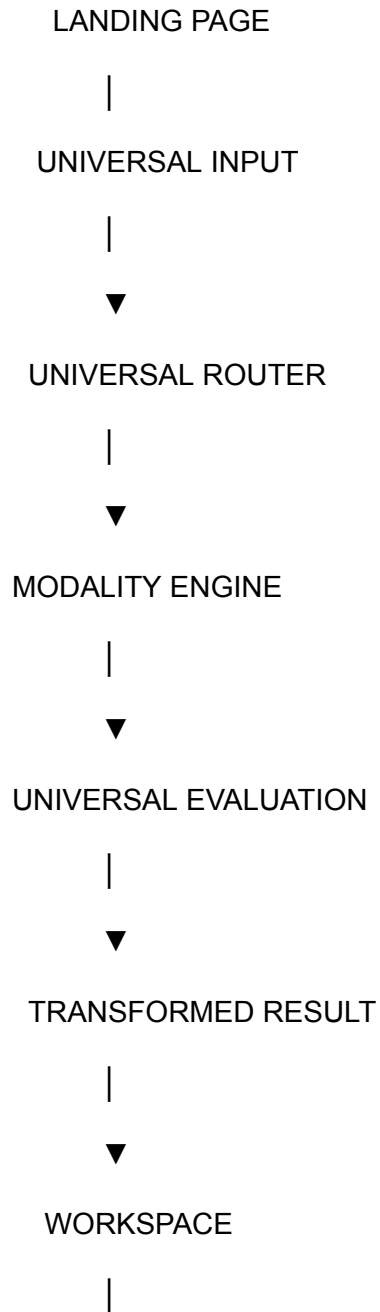
Then, once they enter the workspace:

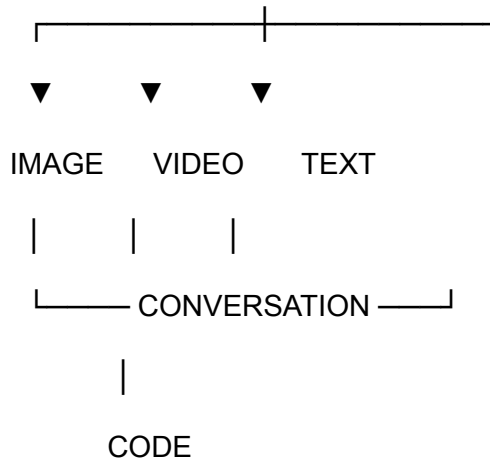
"Okay, I'm working on an image prompt."

Now the interface can become specialized.

---

**So I would revise the architecture slightly:**





The **router still exists**.

The **engines still exist**.

The **evaluator still exists**.

The user simply doesn't need to know they exist.

That's a good separation between **product complexity** and **user complexity**.

And yes, I would make that universal transformation animation a **fixed signature interaction**, not five modality-specific loading sequences.

## Transformation Ritual

Now we define the **single universal transformation experience** that happens after the user presses **Improve**.

Nothing modality-specific here.

The same ritual every time.

---

### 1. The state sequence

IDLE

↓

ACTIVATE



TRANSFORM



REVEAL



REWARD



READY

The user should learn this sequence after one or two uses.

---

## 2. ACTIVATE

The instant they press:

**Improve** →

The button responds physically.

[ Improve → ]



[ Improving ♦ ]

The input card subtly contracts.

The universal transformation element appears.

No spinner.

No "detecting image."

No technical language.

Something recognizably **yours**.

---



### 3. TRANSFORM

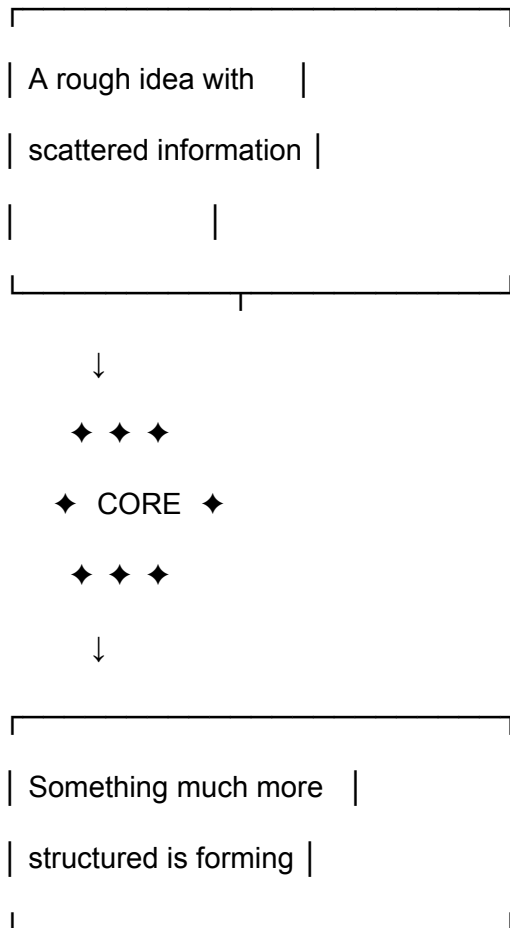
This is the short waiting period.

I would make the transformation visual rather than textual.

The user's messy input gradually loses its visual dominance.

Then your transformation mark becomes the focus.

Conceptually:



The exact animation should be subtle.

Not a loading spinner.

Not a progress bar.

Not a fake "AI thinking" brain.

It should feel like **transformation**.

---

## 4. Give the animation a recognizable signature

This deserves its own design language.

For example, imagine a small geometric symbol we eventually call the **Transformation Core**.

It always behaves the same way:



It contracts when processing begins.

It expands when the result is ready.

The user gradually learns:

**That symbol means my input is being worked on.**

The exact symbol, motion curve, and visual style should be branded later. The behavioral rule is what matters now.

---

## 5. Timing

Don't artificially hold people hostage.

Use animation as a **minimum transition**, then let the backend determine when the result arrives.

### If result arrives in 700ms

Reveal immediately.

### If it takes 3 seconds

The transformation animation continues naturally.

### If it takes 8 seconds

Introduce a subtle secondary state so the interface doesn't look frozen.

Something like:

#### Still shaping it...

But don't make the user watch a fake sequence of:

Analyzing → Thinking → Optimizing → Finalizing → Almost there...

unless those states actually correspond to meaningful backend events.

The animation should be **time-flexible but behaviorally consistent**.

---

## 6. RESULT REVEAL

This is the most important animation.

Don't hard-cut from:

processing

to:

giant prompt.

Instead, let the result emerge from the transformation.

Conceptually:





|                      |  |
|----------------------|--|
| Your enhanced prompt |  |
|                      |  |
| ...                  |  |

The card can move upward a few pixels and settle.

Very small.

Maybe 12 to 20px.

The purpose is simply to create a perceptible transition.

---

## 7. Then reveal the score

Don't reveal:

91/100

at exactly the same moment as everything else.

Sequence it:

**Prompt appears**



**Score appears**



### Short supporting statement

For example:

**91/100**

Strong transformation. Your original intent was preserved while the missing visual direction was filled in.

The number can count quickly from the original estimated quality to the final quality.

If original quality was 43:

43 → 56 → 68 → 77 → 84 → 91

Around 500 to 700ms.

Again, this isn't about making a game.

It gives the user a visible representation of improvement.

---

## 8. Don't exaggerate the score

This is important.

If the evaluator gives:

**67 → 74**

don't animate it like:

**67 → 99!!!**

The interface needs to respect the underlying evaluation.

A modest improvement should look modest.

That makes the dramatic improvements more believable.

---

## 9. Then the "What changed?" micro-reward

Under the score:

### 3 improvements

Clicking expands a compact panel.

- ✓ Clarified the objective
- ✓ Added useful context
- ✓ Defined the desired output

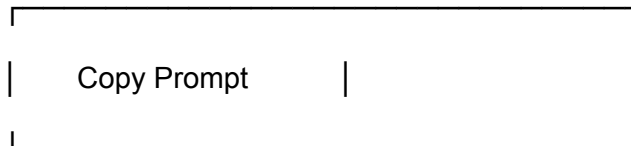
No giant report.

The detailed evaluator remains inside the workspace.

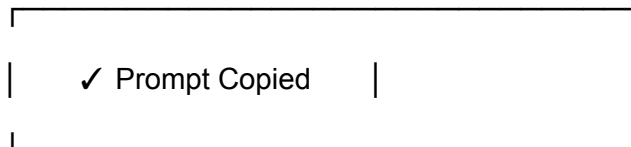
---

## 10. Copy interaction

The primary action:



When clicked:



The checkmark has a tiny spring animation.

The button doesn't disappear.

It returns to:

## Copy Prompt

after a short moment.

---

# 11. The actual psychological reward comes after that

This is the part I think is more valuable than confetti.

Immediately after copying:

**Ready to use.**

Then, slightly below:

**Want to keep improving it?**

**Continue in Workspace →**

The user has just completed a useful action.

We acknowledge it.

Then offer the next level.

That's a clean behavioral loop:

I gave it something

↓

It improved it

↓

I copied it

↓

It confirmed success

↓

I can continue

---

## 12. Don't immediately ask for an account

If the user hasn't demonstrated deeper intent, don't interrupt the reward with:

CREATE ACCOUNT

Instead:

**Continue in Workspace →**

Then, if workspace access requires authentication:

**Create your free workspace**

Now the user understands exactly why they're being asked.

They've already seen what the workspace is for.

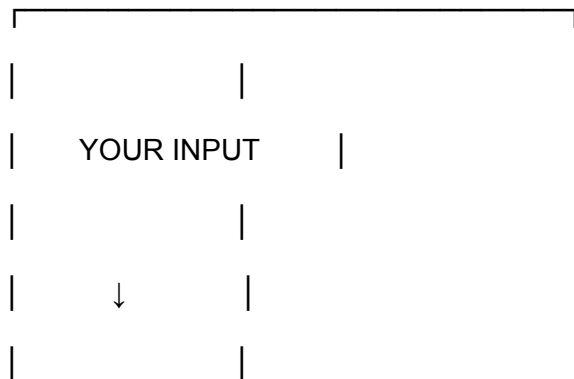
---

## 13. Mobile behavior

Mobile should use the same ritual.

Don't create a completely different animation.

The only changes are spatial.





|     ◆     |  
|     ◆ ● ◆     |  
|     ◆     |  
|     |     |  
|     transforming...     |  
|     |     |  
|\_\_\_\_\_|

Then:

|\_\_\_\_\_|  
| 91 / 100     |  
|     |     |  
| Enhanced prompt     |  
| |\_\_\_\_\_| |  
| | ...     | |  
| |\_\_\_\_\_| |  
|     |     |  
| [ Copy Prompt ]     |  
|     |     |  
| ▶ What changed?     |  
|     |     |  
| Continue in Workspace →     |  
|\_\_\_\_\_|

No horizontal movement.

No complicated choreography.

Everything should happen inside the vertical flow.

---

## 14. One thing I would test

I'd test two transformation styles.

### A. Material transformation

The messy input visually reorganizes itself into the polished output.

This communicates:

**We took your mess and organized it.**

### B. Core transformation

The input fades into the branded transformation symbol, then the result emerges.

This communicates:

**Something powerful happened between input and output.**

I suspect **B** will age better as the product expands across modalities, because it doesn't imply that the system literally rearranged text. It becomes your recognizable product ritual.

---

## 15. The final interaction should feel like this

USER

|

▼

"Improve →"

|



button response



YOUR SIGNATURE

TRANSFORMATION



RESULT REVEAL



91 / 100



"3 improvements"



[ Copy Prompt ]



✓ Copied



Ready to use.



Continue in Workspace →

That's the complete first-touch **behavioral loop**.

And I would keep this stable across every modality.

The user learns the ritual once.

After that, the product feels faster because they already understand what the animation means.

---

## One important thing remains before we move deeper into the landing page:

We haven't designed **what happens when the user's raw input is too strange, too incomplete, too large, unsupported, or ambiguous**.

That's the **edge-case behavior of the first-touch experience**.

That is the next thing I'd design, because a beautiful happy-path animation is useless if the first weird input makes the interface collapse.

# FIRST-TOUCH EDGE CASES

This is where the landing page proves whether the product is actually intelligent.

The rule is:

**Never punish the user for giving us a bad input.**

They came specifically because they have a mess.

So even when the input is poor, the system should try to produce something useful before asking for anything.

---

## 1. Extremely vague input

User:

"Make something good."

Don't respond:

"Please provide more details."

That's exactly the behavior we're trying to replace.

Instead, the engine produces the strongest interpretation possible.

Then:

**I made the strongest version I could from that.**

And below:

**A clearer goal would improve it further.**

**Improve further →**

The user gets something.

---

## 2. Half-finished thought

User:

"I want an app that helps students but"

Same treatment.

The system can infer the likely direction but shouldn't invent the missing business model.

Result:

Here's a usable starting point based on what you gave me.

A few details are still open:

- who the students are
- what problem the app solves

Then:

**Continue in Workspace →**

Not:

"Input invalid."

---

### 3. Extremely bad prompt

User:

"make social media strategy"

This is exactly the kind of input your product should shine on.

Don't show a red score:

**23/100** ❌

That makes the user feel stupid.

Instead:

**We turned your idea into a usable starting point.**

Then show the result.

The diagnostic score can still exist, but I would frame it as:

**Starting quality: 24**

**Enhanced quality: 86**

The user sees the transformation rather than a judgment.

---

### 4. User submits a fully developed prompt

This is important.

Don't force a transformation just because the product expects one.

If the user gives:

a highly detailed 800-word prompt

and the evaluator determines it's already strong:

94 / 100

The system should say:

**This is already strong.**

Then:

**We made a few small improvements.**

Or, if nothing meaningful can be improved:

**Your prompt is already in good shape.**

That builds trust.

A system that rewrites everything just to prove it did something will eventually destroy credibility.

---

## 5. Unsupported modality

Suppose the user uploads something the system can't process.

Don't dump a technical error.

Unsupported MIME type: application/octet-stream

Absolutely not.

Say:

**I can't work with that file yet.**

Then:

**Try pasting the content or uploading an image, video, text file, or code file.**

And leave the input area open.

---

## 6. File too large

Same principle.

Instead of:

Payload exceeds 100 MB.

Say:

**That file is too large for a quick transformation.**

Then:

**Try a smaller file or paste the part you want improved.**

The interface should recover immediately.

---

## 7. Corrupted file

Don't expose infrastructure details.

We couldn't read that file.

Try uploading it again.

Then:

**Try again**

No dead end.

---

## 8. No meaningful content



Imagine someone uploads a blank image or sends:

""

The system should recognize:

usable\_content = false

Then:

**Give me a little more to work with.**

And immediately return focus to the input.

No score.

No transformation ritual.

Because there is nothing meaningful to transform.

---

## 9. Contradictory input

User:

"Make it extremely detailed but use only five words."

Don't blindly obey both.

The system should resolve the contradiction intelligently.

Perhaps:

**I found a conflict in your request. I prioritized the five-word limit.**

Then generate the best possible result.

If the contradiction makes the task impossible:

**These two requirements conflict. Which matters more?**

That's one of the few cases where asking is justified.

---

## 10. Dangerous or inappropriate request

The edge-case layer should also sit before transformation.

If the system can't safely process something, it should refuse or redirect without exposing internal policy.

But the refusal should still preserve the overall product personality.

No giant lecture.

No technical explanation.

Just:

**I can't help create that.**

Then, where appropriate:

**I can help with a safe alternative.**

---

## 11. User presses Improve repeatedly

This is interesting.

Don't create:

Request 1

Request 2

Request 3

Request 4

with no context.

The landing page should treat the first interaction as the primary transformation.

If they immediately click again without changing anything:

**This version is already what I'd recommend.**

Then offer:

**Edit your input**

or:

**Continue in Workspace**

This prevents users from burning requests trying to squeeze a meaningless second transformation out of the same input.

---

## 12. User modifies the input

Now we treat it as a new transformation.

Version 1

↓

User edits input

↓

Version 2

The transformation ritual runs again.

But the workspace history eventually handles the full lineage.

---

## 13. Very long input

Don't dump a massive prompt onto the screen.

The result card gets:

collapsed preview

with:

**Expand**

The user should still see the score and main improvement immediately.

---

## 14. Input contains multiple unrelated ideas

Example:

"Build my website, write my wedding speech, analyze this code, and make an image prompt."

Don't blindly create one enormous Frankenstein prompt.

The router should detect multiple tasks.

```
{  
  "multiple_tasks": true,  
  "tasks": [  
    "website",  
    "speech",  
    "code",  
    "image"  
  ]  
}
```

Then the landing page can say:

**I found several different tasks in this.**

And give:

**Work on them together →**

which leads into the workspace.

This is another legitimate reason to move the user deeper into the product.

---

## 15. Network or model failure

This one should feel completely different from a user error.

If the system fails:

**Something went wrong while transforming this.**

Then:

**Try again**

The original input remains intact.

Never make the user re-enter their work.

---

## 16. Timeout

If the transformation takes longer than the expected window:

Don't keep the user staring at the animation indefinitely.

After an appropriate threshold:

**This one needs a little more time.**

Then give them:

**Keep waiting**

or, where your backend supports asynchronous processing:

**Continue in Workspace**

The key is preserving the user's work.

---

## 17. The universal recovery rule

Every edge case should follow:

PROBLEM



PLAIN-LANGUAGE EXPLANATION



PRESERVE USER INPUT



GIVE ONE CLEAR NEXT ACTION

Never:

ERROR

ERROR CODE

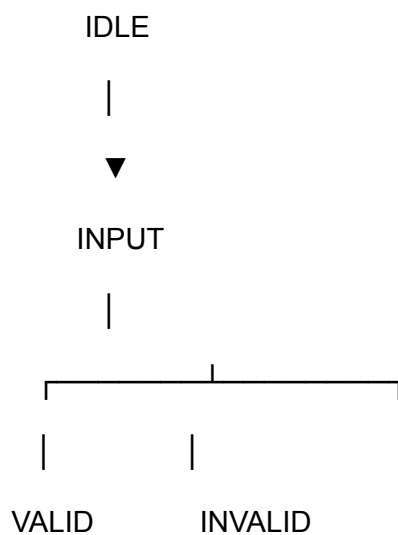
TECHNICAL DETAILS

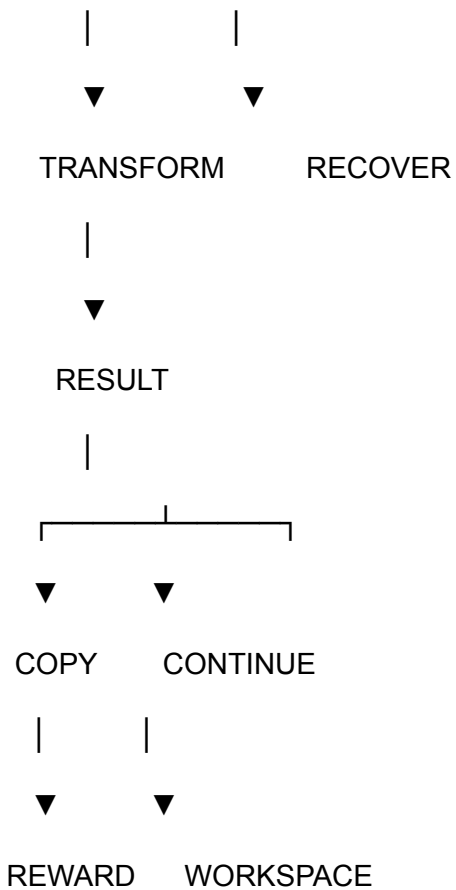
USER FIGURES IT OUT

---

## 18. The landing page state machine

At this point, the first-touch frontend can actually be designed as a proper state machine.





And inside **TRANSFORM**:

ACTIVATE



SIGNATURE ANIMATION



RESULT REVEAL

No modality-specific loading screens.

No questionnaire.

No unnecessary branching.

---

## The important outcome

We've now protected the first-touch experience from both extremes:

### **Too little input**

→ we still produce something useful.

### **Too much / complicated input**

→ we don't make the user wrestle with the interface.

### **Unsupported input**

→ we recover gracefully.

### **Already excellent input**

→ we don't pretend we magically improved it.

### **Multiple tasks**

→ we recognize the complexity and move the user into the workspace.

### **System failure**

→ we preserve their work and recover.

That's the right foundation.

## **Next**

Now we can finally design the **actual landing-page layout below the hero**.

Not another giant feature dump.

We'll decide what, if anything, appears beneath the first transformation experience: proof, examples, before/after demonstrations, trust signals, pricing hints, and the second conversion opportunity.

# **LANDING PAGE BELOW THE HERO**

Now we decide what exists **after the first transformation**, without turning the landing page into a SaaS brochure.

The first viewport has already done the important work:

GIVE SOMETHING





GET SOMETHING BETTER

Everything below it should answer one question:

**"Why should I trust this enough to use it again?"**

Not explain the entire architecture.

---

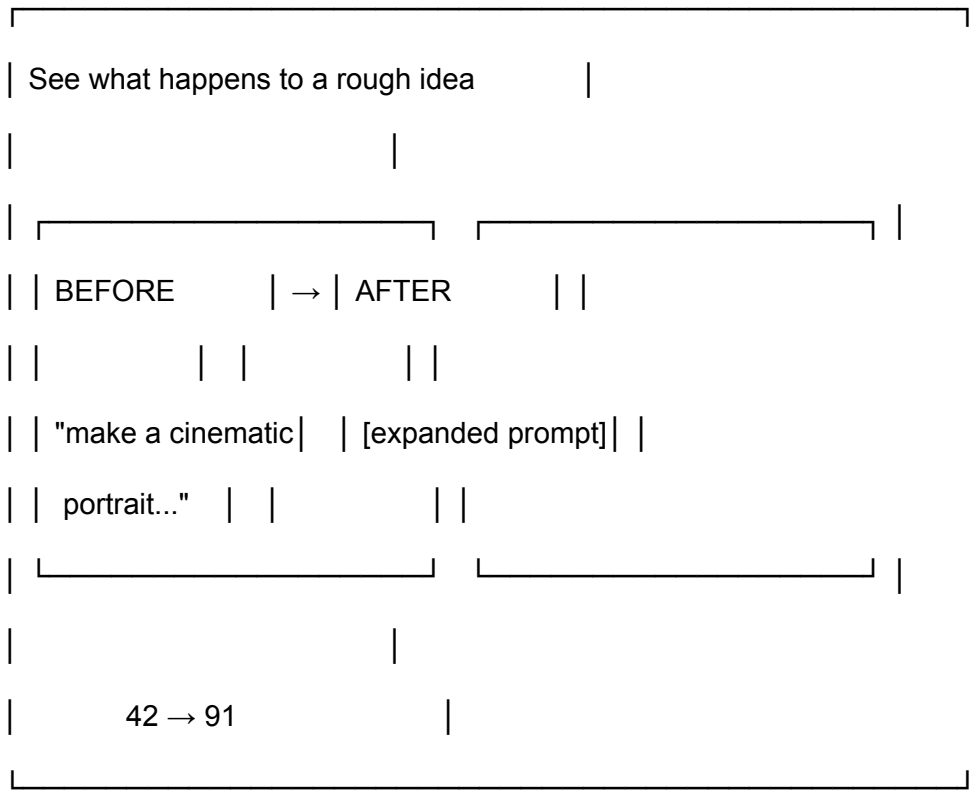
# 1. Section immediately below the hero: proof through examples

I would make this the first section.

Not testimonials.

Not "AI-powered by..."

**Actual transformations.**



One example at a time.

Animate the transformation when the section enters the viewport.

Don't show 12 examples.

One strong example communicates more than a wall of cards.

---

## 2. The examples should rotate by user type

Not necessarily by modality.

Instead:

Creator

Student

Developer

Researcher

Business owner

Because users don't necessarily identify themselves as:

"I am a Text Modality user."

They identify with what they're trying to accomplish.

So the example could say:

**Starting with a half-formed idea**

then:

**Ending with something you can actually use.**

---

### 3. Don't reveal the machinery yet

Avoid:

5 specialized engines

14 evaluators

LLM orchestration

JSON schemas

routing architecture

That's useful for the workspace documentation.

It's not persuasive on the landing page.

The user doesn't buy the plumbing.

They buy the result.

---

### 4. Then one very small explanation

After the examples:

Give us the rough version.

We figure out what you're trying to accomplish,  
strengthen what's missing, and return something  
you can actually use.

Then perhaps:

Ideas → Prompts

Prompts → Better Prompts

Mess → Structure

But keep the language concrete.

No corporate fog.

---

## 5. Then show the workspace

This is where we introduce the larger product.

Not before.

Something like:

Ready to go further?

|                         |                   |      |  |
|-------------------------|-------------------|------|--|
|                         |                   |      |  |
|                         |                   |      |  |
|                         | WORKSPACE PREVIEW |      |  |
|                         |                   |      |  |
| Prompt                  | Improve           | Test |  |
|                         |                   |      |  |
|                         |                   |      |  |
|                         |                   |      |  |
| Your evolving prompt... |                   |      |  |
|                         |                   |      |  |

Build. Refine. Test. Repeat.

[ Enter Workspace → ]

Now the visitor understands:

**Landing page = instant transformation.**

**Workspace = continued development.**

That's a very clean product model.

---

## 6. The workspace preview should show iteration

This is important.

Don't show a dashboard with 17 menus.

Show one transformation:

ROUGH

↓

VERSION 1

↓

USER REFINES

↓

VERSION 2

↓

TEST

↓

FINAL

This visually explains why someone would need an account.

---

## 7. Then trust

Only now do we introduce trust signals.

For example:

Private by default.

Your inputs aren't used as public examples.

Only claim things your actual infrastructure guarantees.

Then:

Secure processing

Structured evaluation

Version history

Again, don't make claims you can't technically support.

---

## 8. Pricing should not dominate the first visit

I'd put a compact pricing signal lower down.

Something like:

**Free to try. Upgrade when you need more.**

Then:

**See plans →**

The first conversion isn't:

**PAY US**

It's:

**TRY THE TRANSFORMATION**

Then:

**USE IT AGAIN**

Then:

**CREATE A WORKSPACE**

Then:

**PAY WHEN THE WORKFLOW BECOMES VALUABLE**

That's a healthier funnel.

---

## 9. Final CTA

At the bottom, don't invent another feature.

Repeat the original interaction.

Got something messy?

Paste it.

Drop it.

Try it.

[ Improve → ]

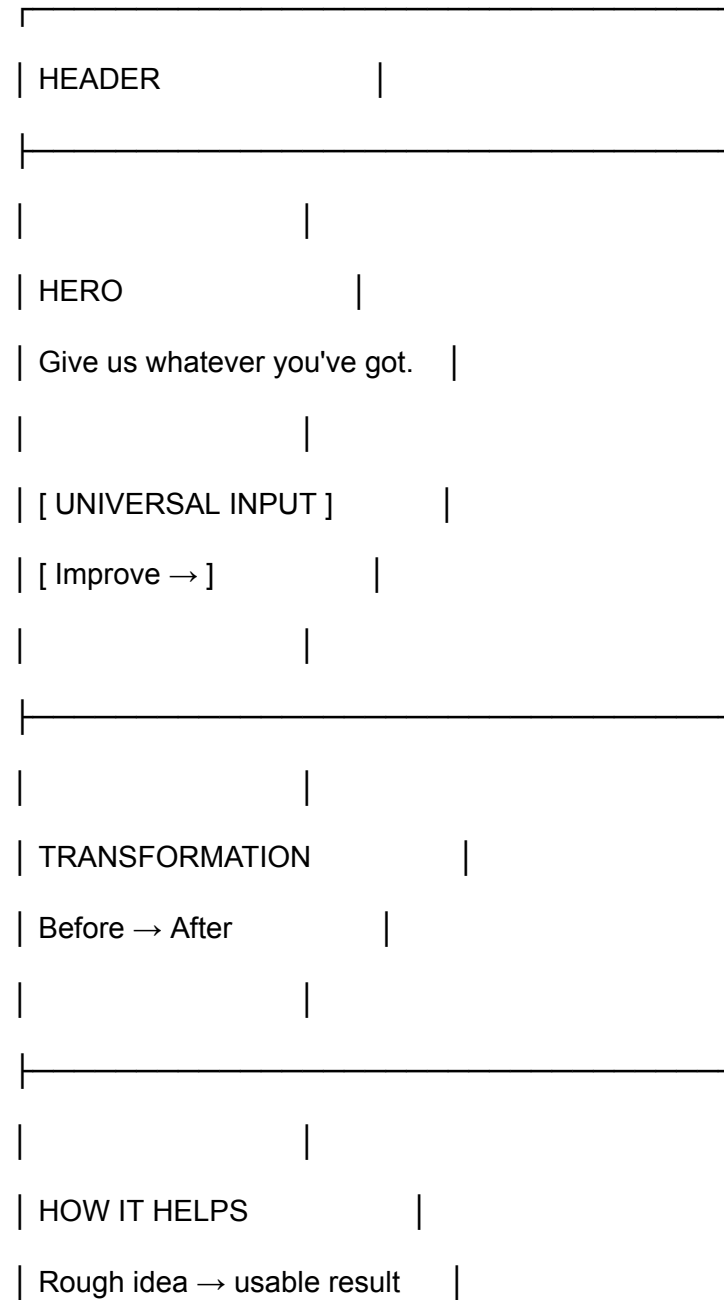
The visitor has now seen the transformation, the proof, and the workspace.

So the final CTA simply returns them to the thing that already worked.

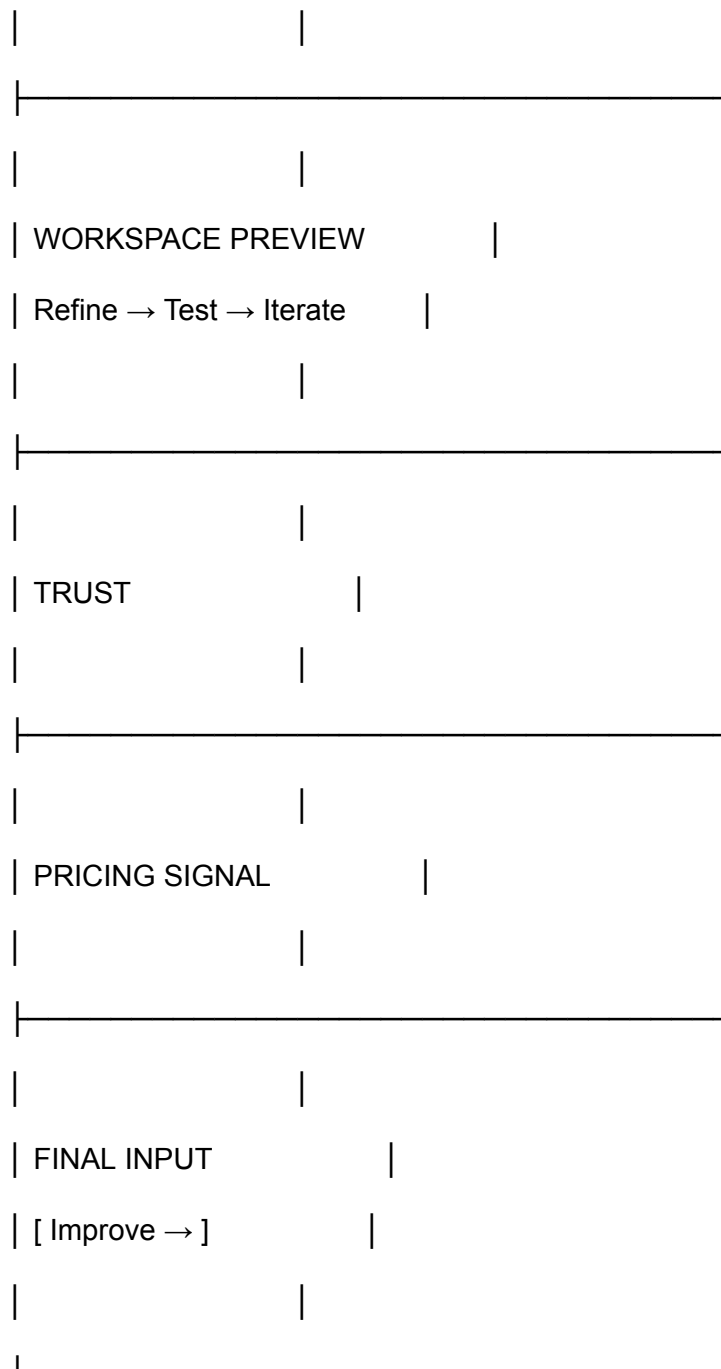
---

## 10. The complete landing page

Now the page becomes:







That's enough.

I would **not** add:

- giant feature grids
- endless customer logos
- FAQ above the main conversion
- five separate modality sections

- a giant "How our AI works" technical diagram
- ten testimonials

Those can wait.

---

## One thing I'd challenge

We shouldn't assume that "**landing page**" necessarily means **one static page with this exact sequence forever**.

Once we have real traffic, we should test:

**Transformation-first landing page**

against:

**Problem-first landing page**

against:

**Before/after-first landing page**

But we don't need three versions now.

For the first build, I'd choose **transformation-first**, because the product's strongest sales argument is something the user can experience rather than something we have to explain.

## Next major piece

We've now designed the **visitor's journey up to workspace entry**.

The next thing is the **Workspace itself**.

That's where the modality selector, specialized engines, deeper questions, iteration, prompt versions, evaluator details, "Try this instead," and the paid product actually belong.

That is a much larger system, so we should again take it **one section at a time**, starting with the **Workspace's fundamental information architecture**, before touching its individual features.

One more thing: **the A/B landing-page idea is now something I'd explicitly build into the architecture**, rather than treating it as a future marketing experiment. We should make the landing experience configurable so that later we can run:

Variant A

Transformation-first

Variant B

Problem-first

Variant C

Before → After-first

against real traffic and measure which one produces the strongest progression:

Visitor

→ Try transformation

→ Result viewed

→ Copy/use

→ Workspace entry

→ Return

→ Paid conversion

That measurement layer should be part of the backend surgery we're about to do.

Send the screenshots, and we'll inspect the **existing product first** before touching the next architectural decision.

# Intent & Context Intelligence Layer

*how do you think we should solve this?*

*I also want our model to be wise, like our earlier quote was wise. We tried making it conversationally wise or adaptive to conversations, but that didn't take root. We need to make it conversationally wise and context-wise so users can come and say something like, "I'm trying to make this and this."*

*Let's say Deisa is trying to write code to diagnose a bug in his project and he's been fighting with his editor and trying to find the code. We didn't know how to find it and now he opens our application and comes to the chatbot and asks the chatbot to tell us the chat. He now comes and does not know what to do or how to prompt the editor to find that exact bug that is happening.*

*Such a user might come to our application and tell our model in the chat about his frustration. He's doing this code with his editor and he's trying to create this, this, and this. He has found this particular bug and has tried telling the editor to do this but the bug is persistent. How does he inform the bug or write a prompt to the editor to fish out the bug?*

*Currently our model is not advanced or wise enough to synthesize and understand all these contexts from the user's point of view and output a response and potentially a prompt. I want us to do that. Do we need to change our model or do we need to fine-tune another model? Do we need to spin up a specific agent or LLM and configure it with such mindsets to be able to do certain things, or can we configure Gemini models that we are using to be this advanced to this level and make our model or application this smart?*

## What is the real problem underneath?

It isn't:

"How do we make the AI better at debugging conversations?"

It's:

**How do we make the AI understand what a person is actually trying to accomplish, even when they don't know how to formulate the request, and then determine the most useful thing to do with that understanding?**

That's much broader.

The user could arrive saying:

"I have this idea for a business but I don't know how to structure it."

Or:

"I need an image of this but I can't explain what I'm seeing in my head."

Or:

"I've been trying to get ChatGPT to write this thing and it keeps giving me garbage."

Or:

"I have this research problem and I don't even know what question I'm supposed to ask."

Or:

"I need to tell my coding agent what to do, but I don't know how to explain the problem."

Or:

"Here's my prompt. Something about it isn't working."

The underlying problem is the same.

**The user has a goal, but the user's language is an imperfect representation of that goal.**

That's the thing our system needs to solve.

---

## So I'd change the architecture

I actually think **"Context Orchestrator"** is too narrow a name.

I'd call this something closer to:

### **Intent & Context Intelligence Layer**

Its job is to construct a continuously updated model of:

WHO / WHAT



WHAT THEY ARE TRYING TO ACHIEVE



WHAT THEY HAVE ACTUALLY GIVEN US



WHAT THEY ALREADY KNOW



WHAT THEY DON'T KNOW



WHAT THEY HAVE TRIED



WHAT IS PREVENTING PROGRESS



WHAT CAN SAFELY BE INFERRED



WHAT CANNOT BE INFERRED



WHAT WOULD HELP MOST NEXT

Notice something important.

**"What they have tried" is now only one possible piece of context.**

It isn't the center of the architecture.

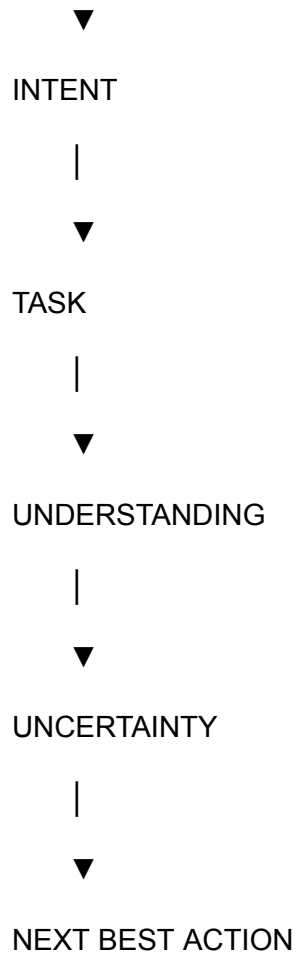
---

## The deeper state model

I'd restructure the core around six universal concepts.

USER SITUATION





Everything else attaches to those.

## **User Situation**

What is happening around the request?

## **Intent**

What does the user actually want to accomplish?

## **Task**

What concrete job needs to be performed?

## **Understanding**

What does the system currently know about the situation?

## **Uncertainty**



What doesn't it know, and does that missing information matter?

## Next Best Action

Should it answer, create, transform, ask, inspect, challenge, explain, route, or do something else?

That's the actual general problem.

---

# And there's another thing we were missing

**The system needs to understand the user's level of formulation.**

This is extremely important for your product.

Consider:

"I want to make something that helps students."

versus:

"I want to build a SaaS platform that uses retrieval-augmented generation to help postgraduate students search literature."

versus:

"Here's my 900-word system prompt. It works badly."

These are three completely different **states of user formulation**.

The system should recognize:

RAW INTENTION

↓

ROUGH CONCEPT

↓

PARTIALLY FORMULATED TASK

↓

DEVELOPED INSTRUCTION



EXPERT SPECIFICATION

And respond accordingly.

That's much closer to what **Prompt Architect** actually needs.

---

## The real intelligence loop

I would now define the brain as:

USER INPUT



UNDERSTAND SITUATION



EXTRACT INTENTION

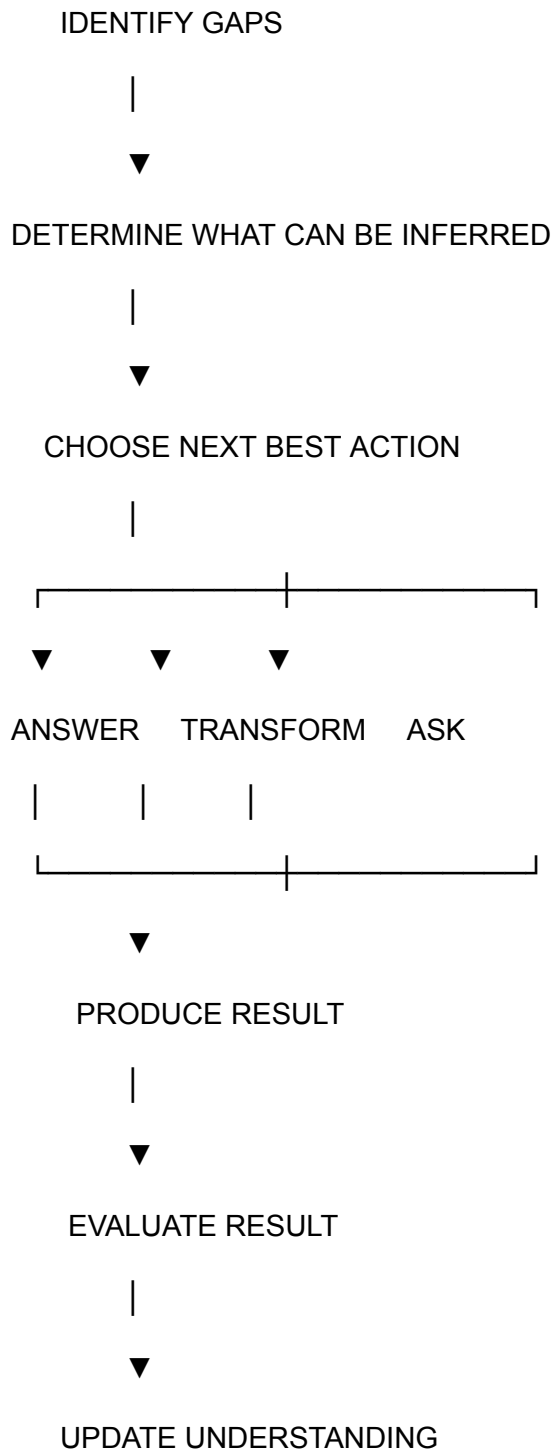


MODEL THE TASK



ASSESS CURRENT FORMULATION





**That** is the thing I was reaching for with the Context Orchestrator.

The previous schema didn't capture it cleanly enough.

---

## And this changes how we think about "wisdom"

We shouldn't try to make the model *sound wise*.

That's superficial.

We want it to exhibit behaviors such as:

**Understand before acting.**

**Distinguish what the user said from what the user meant.**

**Don't ask for information that isn't necessary.**

**Don't invent information simply because something is missing.**

**Know when an inference is safe.**

**Know when uncertainty matters.**

**Notice when the user's current request isn't actually the problem they're trying to solve.**

**Choose the next useful action.**

**Remember what has already happened.**

**Change course when new information arrives.**

And yes, **that is what we should build next.**

Yes. Now we build the **actual universal schema**, with the example removed from the architecture.

The schema should model **the user's situation and the system's understanding of it**, regardless of whether the user is creating an image, debugging code, writing a research prompt, designing a conversation, or simply saying, "I don't know how to explain what I need."

## Universal Intent & Context State

I would call the object:

IntentContextState

Its job is to answer:

**What is happening, what does the user actually want, what do we know, what don't we know, what can we safely infer, and what should happen next?**

```
export interface IntentContextState {
```

```
// _____
```

```
// 1. REQUEST
```

```
// What arrived?
```

```
// _____
```

```
request: {
```

```
  id: string;
```

```
  rawInput: string;
```

```
  inputType:
```

```
    | "idea"
```

```
    | "question"
```

```
    | "instruction"
```

```
    | "prompt"
```

```
    | "conversation"
```

```
    | "artifact"
```

```
    | "mixed"
```

```
    | "unknown";
```

modalities: Array<

"text"

| "image"

| "video"

| "code"

| "conversation"

>;

timestamp: string;

};

// \_\_\_\_\_

// 2. SITUATION

// What is happening around the request?

// \_\_\_\_\_

situation: {

summary: string;

context: string[];

circumstances: string[];

```
blockers: string[];
```

```
relevantBackground: string[];
```

```
};
```

```
// _____
```

```
// 3. INTENT
```

```
// What is the user actually trying
```

```
// to accomplish?
```

```
// _____
```

```
intent: {
```

```
    primary: string;
```

```
    secondary: string[];
```

```
    desiredOutcome: string;
```

```
    underlyingNeed?: string;
```

```
    confidence: number;
```

```
};
```

```
// _____
```

```
// 4. OBJECTIVE
```

```
// What concrete result would satisfy
```

```
// the user's intent?
```

```
// _____
```

```
objective: {
```

```
  description: string;
```

```
  successCriteria: string[];
```

```
  failureConditions: string[];
```

```
  priority:
```

```
    | "low"
```

```
    | "normal"
```

```
    | "high"
```

```
    | "urgent";
```

```
};
```

```
// _____
```

```
// 5. TASK
```



// What operation should the system perform?

// \_\_\_\_\_

task: {

type:

| "answer"

| "explain"

| "generate"

| "transform"

| "improve"

| "audit"

| "diagnose"

| "design"

| "plan"

| "research"

| "compare"

| "debug"

| "refine"

| "unknown";

description: string;

target?: string;

stage:

| "discovery"

| "formulation"

| "creation"

| "refinement"

| "validation"

| "completion"

| "unknown";

};

// \_\_\_\_\_

// 6. FORMULATION STATE

// How well has the user expressed the

// thing they are trying to accomplish?

// \_\_\_\_\_

formulation: {

level:

| "raw"

| "rough"

| "partial"

| "developed"

| "structured"

```
| "expert";
```

```
completeness: number;
```

```
clarity: number;
```

```
specificity: number;
```

```
knownWeaknesses: string[];
```

```
};
```

```
// _____
```

```
// 7. KNOWLEDGE STATE
```

```
// What do we actually know?
```

```
// _____
```

```
knowledge: {
```

```
  facts: KnowledgeItem[];
```

```
  userClaims: KnowledgeItem[];
```

```
  verifiedFacts: KnowledgeItem[];
```

inferences: KnowledgeItem[];

contradictions: Contradiction[];

unknowns: string[];

};

// \_\_\_\_\_

// 8. CONSTRAINTS

// What must / should / must not happen?

// \_\_\_\_\_

constraints: Constraint[];

// \_\_\_\_\_

// 9. ASSUMPTIONS

// What are we choosing to assume?

// \_\_\_\_\_

assumptions: Assumption[];

```
// _____
```

```
// 10. UNCERTAINTY
```

```
// Where could the system be wrong?
```

```
// _____
```

```
uncertainty: {
```

```
  areas: UncertaintyArea[];
```

```
  overall: number;
```

```
  blockingUnknowns: string[];
```

```
};
```

```
// _____
```

```
// 11. USER INTERACTION STATE
```

```
// How is the conversation progressing?
```

```
// _____
```

```
interaction: {
```

```
  stage:
```

```
    | "opening"
```

```
    | "understanding"
```

```
| "working"  
| "clarifying"  
| "delivering"  
| "refining"  
| "complete";
```

```
userCorrections: string[];
```

```
previousResponses: string[];
```

```
unresolvedIssues: string[];
```

```
repeatedFailureDetected: boolean;
```

```
};
```

```
// _____
```

```
// 12. HISTORY
```

```
// What has already happened that matters?
```

```
// _____
```

```
history: {
```

```
  relevantEvents: ContextEvent[];
```

```
previousAttempts: Attempt[];
```

```
previousDecisions: Decision[];
```

```
rejectedDirections: string[];
```

```
};
```

```
// _____
```

```
// 13. STRATEGY
```

```
// How should the system approach this?
```

```
// _____
```

```
strategy: {
```

```
    approach: string;
```

```
    reasoningMode:
```

```
        | "fast"
```

```
        | "standard"
```

```
        | "deep";
```

```
    requiredEvidence: string[];
```

```
    optionalEvidence: string[];
```

```
avoid: string[];  
};
```

```
// _____  
  
// 14. NEXT BEST ACTION  
  
// What should happen NOW?  
  
// _____
```

```
nextAction: {  
  type:  
    | "answer"  
    | "ask"  
    | "infer"  
    | "transform"  
    | "generate"  
    | "improve"  
    | "audit"  
    | "analyze"  
    | "requestInput"  
    | "requestArtifact"  
    | "useTool"  
    | "route"
```



```
| "enterWorkspace"
```

```
| "finish";
```

```
objective: string;
```

```
reason: string;
```

```
requiredInformation: string[];
```

```
confidence: number;
```

```
};
```

```
// _____
```

```
// 15. OUTPUT CONTRACT
```

```
// What should the response actually contain?
```

```
// _____
```

```
output: {
```

```
  artifact?: string;
```

```
  format?:
```

```
    | "plain_text"
```

```
    | "prompt"
```

```
| "structured_prompt"  
| "code"  
| "json"  
| "plan"  
| "explanation"  
| "conversation"  
| "other";
```

```
requiredElements: string[];
```

```
exclusions: string[];
```

```
userVisibleExplanation?: string;
```

```
};
```

```
// _____
```

```
// 16. CONFIDENCE
```

```
// How certain are we about our model
```

```
// of the situation?
```

```
// _____
```

```
confidence: {
```

```
  situation: number;
```

intent: number;

objective: number;

task: number;

knowledge: number;

strategy: number;

nextAction: number;

overall: number;

};

}

## The supporting objects

These are deliberately generic.

interface KnowledgeItem {

statement: string;

source:

| "user"

| "artifact"

- | "tool"
- | "system"
- | "inference"
- | "history";

confidence: number;

verified: boolean;

}

interface Constraint {

  requirement: string;

  type:

- | "must"
- | "must\_not"
- | "prefer"
- | "format"
- | "technical"
- | "safety";

  priority:

- | "critical"
- | "high"
- | "medium"

| "low";

source:

| "user"

| "system"

| "tool"

| "inference";

}

interface Assumption {

statement: string;

confidence: number;

impact:

| "low"

| "medium"

| "high";

safeToProceed: boolean;

requiresConfirmation: boolean;

}

interface UncertaintyArea {

subject: string;

description: string;

impact:

| "low"

| "medium"

| "high"

| "blocking";

canInfer: boolean;

resolution?: string;

}

interface Contradiction {

firstStatement: string;

secondStatement: string;

severity:

| "low"

| "medium"

| "high";

resolution?: string;

```
}
```

```
interface Attempt {
```

```
    action: string;
```

```
    outcome:
```

```
        | "successful"
```

```
        | "partially_successful"
```

```
        | "failed"
```

```
        | "unknown";
```

```
    consequence?: string;
```

```
    source:
```

```
        | "user"
```

```
        | "assistant"
```

```
        | "tool";
```

```
}
```

```
interface Decision {
```

```
    decision: string;
```

```
    reason?: string;
```

```
    alternatives?: string[];
```

```
source:
  | "user"
  | "assistant"
  | "system";
}

interface ContextEvent {

  type: string;

  description: string;

  timestamp: string;

  relevance: number;
}
```

---

## The important hierarchy

The schema now has a deliberate order:

REQUEST

↓

SITUATION

↓

INTENT

↓



OBJECTIVE



TASK



FORMULATION



KNOWLEDGE



CONSTRAINTS



UNCERTAINTY



STRATEGY



NEXT ACTION



OUTPUT

That order matters.

The model shouldn't jump from:

"Here's what the user said"

straight to:

"Here's a prompt."

It first builds an internal model of **what is actually going on**.

---

# The key distinction

The most important fields are these:

Intent

Objective

Task

Formulation

Uncertainty

NextAction

Because they answer different questions.

## **Intent**

**What does the person really want?**

## **Objective**

**What result would satisfy that desire?**

## **Task**

**What operation needs to happen to get there?**

## **Formulation**

**How well has the person expressed it?**

## **Uncertainty**

**What don't we know, and does it matter?**

## **NextAction**

**What should we do right now?**

That is the general intelligence loop.

---

# And this gives us the behavior we wanted

A user says:

"I don't know how to explain this."

The system doesn't immediately ask:

"Can you provide more details?"

It first determines:

Intent:

unknown but discoverable

Formulation:

raw

Uncertainty:

high

NextAction:

ask

A user gives a beautifully developed request:

Intent:

high confidence

Formulation:

expert

Uncertainty:

low

NextAction:

transform

A user gives an incomplete request but the missing information doesn't materially affect the answer:

Uncertainty:

medium

Can infer safely:

yes

NextAction:

transform

A user gives an incomplete request where the missing information radically changes the result:

Uncertainty:

high

Blocking:

yes

NextAction:

ask

**That is the behavior we're after.**

The system isn't "conversationally wise" because we wrote *be wise* into a system prompt.

It is wise because it has an internal structure for **understanding, uncertainty, inference, action, and correction**.

And this schema is deliberately modality-neutral. Image, video, text, conversation, code, business planning, research, or something we haven't even thought of yet can all enter it.

The modality engines then consume the relevant slice of this state rather than becoming responsible for understanding the entire human situation themselves.